From Traces to Guarded Programs: Evidence-Gated Compilation of Recurrent Agent Workflows

Reza Rahimi
JazzX AI
Los Altos, CA, USA
reza.rahimi@jazzx.ai

ABSTRACT

Tool-using agents often invoke a model at every step, even along a repeated read-only path. Replacing repeated reasoning with deterministic execution could reduce inference cost, but repetition alone does not make this safe: tool arguments may depend on decisions not recoverable from observable state, apparently read-only operations may have hidden effects, and reproducing the same tool calls may still change the agent's final answer. Recurrence identifies an opportunity for optimization, not permission to remove reasoning.

We introduce *guarded agentic compaction* (GAC), a compile-or-retire approach that converts recurrent execution paths into deterministic programs only when the evidence supports it. GAC grounds tool arguments in task inputs, prior results, or observed invariant literals, treats writes, approvals, handoffs, and unknown effects as hard barriers, protects compiled execution with input, output, and environment guards, and requires a finite-sample risk test before deployment. Admission depends jointly on a configured risk bound and the statistical confidence supported by the available traces. If any requirement fails, GAC keeps the largest justified prefix when possible, or falls back to the unchanged agent.

Across three live GitHub workflow families and 90 unseen test cases, GAC satisfies the exact task requirements on all 90, compared with 89 of 90 for the unchanged agent, while reducing model requests by 66.6%, tokens by 63.1%, latency by 64.2%, and estimated cost by 58.7%. A time-forward evaluation specializes four of five further repositories, and on 132 fresh post-cutoff pull requests an end-to-end certificate records no misses (one-sided bound 0.0173). On BIRD, a public text-to-SQL benchmark, a schema-first agent's opening is compiled on 26 of 31 databases, cutting model calls by 45–50% with no detected accuracy loss, while agents that choose tables per question are refused. Under its default 0.05 risk bound GAC declines NESTFUL, API-Bank, and BFCL for lack of qualifying evidence, and AppWorld's one admitted program rarely fits the target agent. We also report limits: the learned gate was indistinguishable from a support-only comparator, an amended end-to-end issue gate retired before its held-out test, and under corrupted tool outputs the run-time checks prevented no wrong answers.

GAC reframes optimization as evidence-gated specialization: compile only what is justified, and preserve the agent everywhere else.

KEYWORDS

LLM agents, tool use, workflow optimization, program synthesis, provenance, selective prediction, risk control, OpenAI Agents SDK

1 INTRODUCTION

The standard reasoning–acting loop delegates every tool boundary to a language model: model, tool, model, tool, and so on [52]. This flexibility is valuable while a workflow is novel. At scale, however, mature agents often revisit the same read-only evidence-gathering prefix with the same data dependencies. Repeating a model decision then consumes latency, tokens, and money without necessarily adding useful adaptation.

Deleting such decisions is not ordinary caching. A tool argument may depend on a prior observation, a repeated sequence may cross an approval or write, a nominal read may have a hidden external effect, and an input that resembles the training trace may violate a permission or freshness constraint. A fast wrong workflow is worse than a slow agent. The research problem is therefore:

> *Given historical executions of a tool-using agent, identify a deterministic region that can replace model-mediated control flow, and dispatch it only where observable evidence supports that replacement.*

Consider the expanded real GitHub issue task. Given an issue number, the unchanged agent usually calls `record`, `labels`, and `comments` in that order; the latter two receive the `issue_number` returned by `record`. It then produces a source-grounded classification and excerpt. All 132 discovery executions take that route. A recurrence-only optimizer would therefore emit one attractive macro: fetch all three observations, remove the intervening model turns, and hand the bundle to the final model call.

The apparent macro is not justified. The observed `limit` is a varying integer chosen by the agent; it cannot be reconstructed from the entry state or either earlier result. GAC therefore rejects the three-read candidate as `ungroundable_slot` and emits only the `record -> labels` prefix. The unchanged agent chooses the comments call and renders the answer. This two-read artifact passes 92/92 calibration groups at $\alpha = .05$ with an exact upper bound of 0.0498. Section 3.3 walks through the decision with an executed held-out record.

Successful replay is still narrower than safe substitution. In a separate, earlier 18-record study, an aggressive three-read artifact replayed all 45 calibration groups but passed only 17 exact answer contracts. On issue 6602 it returned Markdown anchor text where the contract required the full URL. A provider-free `ContinuationGuard` detects the miss and checked-renders all 18 cases; this is a mechanism check, not live safety evidence. Together, the two cohorts separate four decisions: recurrence proposes a candidate; provenance bounds its program; continuation checks the remaining answer; and the evidence gate decides deployment.

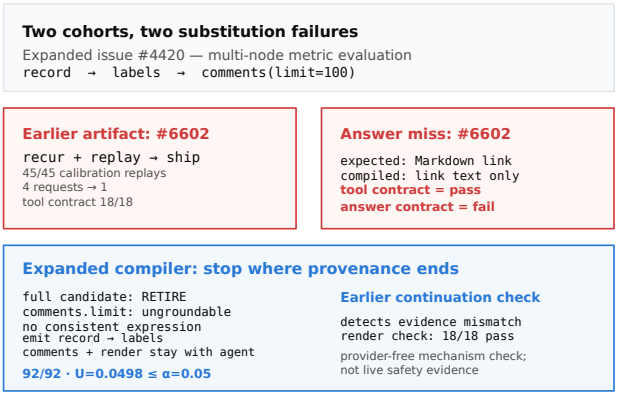


Figure 1: Why recurrence and tool replay do not license full substitution. In the expanded cohort, the comments limit varies and is not trace-groundable; the compiler therefore emits two reads and admits them under 92/92 calibration groups at $\alpha = .05$. In a separate earlier cohort, a three-read artifact replayed tools cleanly but lost the URL on issue 6602; a provider-free continuation guard detected and checked-rendered the miss.

This problem differs from current-query scheduling in LLMCompiler [25], prompt-program optimization in DSPy [24, 39], graph compression in AgentSlimming [9], model routing [36], and prompt compression [21]. It is closest to Agent Workflow Optimization (AWO), which converts frequent trace sequences into deterministic meta-tools [1], and to plan caching [54]. Our focus is the missing admission argument: typed value provenance, effect barriers, compatibility pins, post-execution verification, and finite-sample selective risk control.

We call the resulting approach *guarded agentic compaction* (GAC): a guarded *specialization* that compiles the routine part of a workflow — the recurrent read-only evidence prefix — while leaving every decision that the evidence does not determine to the model. Compaction names the mechanism, specialization names what it is allowed to do, and the boundary between them is the paper's subject. The shipped implementation contains two passes over a common typed trace representation: guarded region compilation (GRC), studied here, and trace-guided workflow specialization (TGWS), which routes entry states to smaller prompt/tool configurations. We isolate GRC because it changes execution semantics more directly and is the contribution with new real-scenario evidence.

The evaluation asks six reader-facing questions:

RQ1. Can typed trace provenance reconstruct the dependencies needed to synthesize recurrent tool programs?

RQ2. On unseen real records, does an admitted prefix preserve the registered factual contract while reducing provider work, and how does it compare with an equally placed hand-written program?

RQ3. Does exact selective admission reject recurrent families when calibration support is inadequate?

RQ4. Which failure modes remain, and what does compaction *not* improve?

RQ5. Can group-level paired evidence select a risk-bounded optimization action before a fresh real-record cohort, and does that choice preserve the registered task contract while reducing resource use?

RQ6. Do efficiency and preservation transfer across distinct real-record workflow families with different tool vocabularies and output contracts?

The paper makes four contributions:

(1) An evidence-licensed trace-to-program formulation that combines typed value provenance, effect and position barriers, bounded readable synthesis, empirical contracts, continuation-pinned composite projection, and runtime fallback.

(2) A compile-or-retire admission protocol: the score and threshold grid are frozen before calibration, exact finite-sample bounds govern dispatch, and rejected candidates remain visible rather than being silently discarded.

(3) A real-provider study spanning three public-record workflow families and 90 held-out records, plus a narrower five-repository time-forward extension on a frozen PR-outcome task. Together they separate execution conformance from source-grounded answer quality, preserve retained negative outcomes, and compare unchanged, compiled, and fair manual or template baselines without claiming superiority where they tie.

(4) A four-substrate external compiler audit in which three corpora retire and AppWorld supplies the first reachable admission, followed by a released-agent trajectory analysis that separates artifact admissibility from structural dispatchability without claiming agent quality or resource savings.

Alongside the primary live study, we add two narrower extensions. First, a frozen-source cross-repository GitHub checkpoint executes a simplified PR-outcome task across five repositories, reaches 580/580 exact discovery traces, completes four time-forward held-out repository cohorts exactly, and retires the fifth at compile time. Second, a bounded public-record extension checkpoint records 420 privacy-modified HMDA groups that pass an independently implemented, provider-free exact gold reconstruction. HMDA is reported as a source-fidelity and control-plane preflight, not as a live optimization result; it is excluded from every effectiveness denominator and cross-domain efficiency claim.

We do *not* claim semantic equivalence, production certification, full-workflow cross-repository or time-forward generalization, or superiority to hand-written code. GCS synthesizes only a read-only view over an admitted program; it does not infer undeclared semantics or fuse physical reads. The six-case manual result is parity, and the bounded GEPA result does not generalize to prompt optimization as a whole.

Roadmap. Section 2 formalizes episodes, candidate regions, the position invariant, and the selective objective. Section 3 presents the method as a cascade of independent rejection points (fig. 6) and states the calibration guarantee. Section 4 defines the evidence tiers and the hypotheses they test, and section 5 answers RQ1–RQ6, opening with the hypothesis-by-hypothesis verdict in table 6 and closing with the five negative results and what each

one rules out (section 5.12). Section 6 reports the later studies (a public SQL benchmark, fresh post-cutoff records, replications, and retired gates). Section 7 positions GAC against adjacent optimizers, and section 9 groups the threats to validity by the kind of inference each endangers. Every rejected candidate, the archived failed pilot, and the unsupported claims are retained rather than pruned, so the denominator of each result stays visible.

2 PROBLEM FORMULATION

2.1 Executions and candidate regions

Reading guide. Everything in this section refines one informal invariant. Picture a careful operations assistant that asks a model what to do before each read. GAC may replace a short, repeated stretch of those questions with a pre-checked checklist, and only while all of the following hold: every value the checklist needs can be traced to the entry form or to an earlier receipt; every action in it is a declared safe-to-stage read in the same security compartment; the checklist begins at the one point where the runtime can substitute it; the present case matches the pinned workflow and passes its guard and its risk gate; and the staged result passes inspection before release. If any of those fails, the ordinary assistant continues — which is a designed outcome, not a failure. The definitions below are what make each clause precise, and they, not this paragraph, are authoritative.

An episode $E = (z, M, e_{1:T}, y)$ contains an entry-state snapshot z, a content-addressed execution manifest M, ordered events e_t, and an observable outcome y. Events include model requests/responses, tool calls/results, handoffs, guardrails, approvals, errors, and commit boundaries. Each tool f has an application-owned effect declaration $\epsilon(f)$ and capabilities such as *speculatable* and *replayable*. Unknown effects are not reads.

A candidate region $R = [a, b]$ begins at a model-request boundary and ends after a tool result. It is *groundable* when every call argument is either an application-declared literal for that slot or has an expression over entry state or prior results within R:

$$\forall c_j \in R,\ \forall u \in \mathrm{args}(c_j): \quad \mathrm{Lit}(c_j, u) \vee \left[u = g_{j,u}\left(z, o_{<j}\right), \quad g_{j,u} \in \mathcal{L}\right], \tag{1}$$

where Lit is a schema-level allowlist fixed independently of the traces and $\mathcal{L}$ is a closed, bounded transform library. It is *effect-admissible* when all tools are declared read-only, speculatable, replayable, and within the same principal/isolation partition. Handoffs, approvals, writes, errors, and unknown effects are barriers.

The current runtime resolves artifacts at the initial model boundary. Therefore the deployable compiler adds a position invariant:

$$a = 0 \quad \text{in the normalized model-boundary index.} \tag{2}$$

This restriction is not aesthetic. A suffix program dispatched at entry may reorder or duplicate earlier calls; section 5.11 demonstrates the failure empirically.

Do not infer. A repeated tool name, an *undeclared* matching literal value, and a successful replay are not provenance witnesses. An allowed literal must be fixed by the schema; every other slot needs an expression in $\mathcal{L}$ over z or a prior result. That is why section 3.3 can reject a region whose three reads recur in every observed execution.

2.2 Selective optimization objective

The compiler emits an artifact $A = (P, H, V, q, \eta, M)$: deterministic program P, hard guard H, verifier V, nonconformity score q, threshold η, and manifest pins M. The six components answer six different questions and none of them is "the guard": P is the checklist to run; H decides whether this entry state is eligible to run it; V inspects what the run produced; q scores how unfamiliar the entry state is; η is the cutoff chosen from a frozen grid so that a bound on selective risk holds (eq. (8)); and M is the compatibility seal that says which deployment the whole kit was learned under. Let $x = (z, M', \omega)$ denote a future entry state, runtime manifest, and external state. Here $M' \simeq M$ means equality on every pinned compatibility field; ω can affect outcomes and cost but is not observable at the dispatch boundary. Dispatch is selective:

$$d_A(x) = \mathbb{1}\{H(z, M') = 1 \wedge q(z) \le \eta \wedge M' \simeq M\}. \tag{3}$$

If any precondition fails, the baseline agent runs. If execution fails before a commit and staging proves the attempt clean, the baseline agent runs. Dirty failure is an incident, not fallback.

Let $L(A, x) = 1$ indicate an externally visible task/contract violation or incident after dispatch; a clean abort followed by the unchanged baseline has $L = 0$. Let $C(A, x)$ include the complete realized cost of the attempt, including a clean fallback. For a future group $G = (x_1, \ldots, x_m)$, define $D_A(G) = \max_i d_A(x_i)$ and $W_A(G) = \max_i d_A(x_i) L(A, x_i)$. We seek an artifact with high coverage and savings subject to bounded selective risk:

$$\max_A \quad \mathbb{E}_{G \sim \mathcal{D}_{\mathrm{grp}}}\left[\sum_{x \in G} d_A(x)\{C(B, x) - C(A, x)\}\right] \tag{4}$$

$$\text{s.t.} \quad \Pr[W_A(G) = 1 \mid D_A(G) = 1] \le \alpha, \tag{5}$$

where B is the unchanged agent. The constraint conditions on dispatch: pre-dispatch abstention does not enter its denominator, although it earns no savings in the objective; by convention its conditional risk is zero when population dispatch probability is zero. The implementation may return no artifact (RETIRE); this is a valid optimizer output.

A singleton-group worked count makes the denominator concrete. Suppose 100 future episodes reach the entry boundary, 40 fail the manifest or hard guard, 30 are abstained by the gate, and 30 are dispatched. The savings term of (5) accumulates over those 30, and so does the risk constraint: it bounds the fraction of *those 30* that were wrong after dispatch, not the fraction of all 100 that were answered correctly. A workload where the gate abstains on 70 of 100 episodes therefore has poor coverage and can still satisfy the constraint exactly, which is the trade fig. 5 is drawn to expose. The reported live studies use one episode per group; for multi-episode groups, the risk constraint counts a group as admitted or violating when any member does, and the objective sums savings over members under the same group distribution.

At the compiler layer the implemented action set is $\{B, A\}$: retain the baseline or admit one compiled artifact. The separate portfolio layer in section 3.7 consumes paired measurements for arbitrary named actions. It does not weaken compiler admission or turn an unevaluated macro into deployable code.

Equation (5) is a design target, not a problem the implementation solves. Algorithm 1 ranks a bounded set of families, calibrates

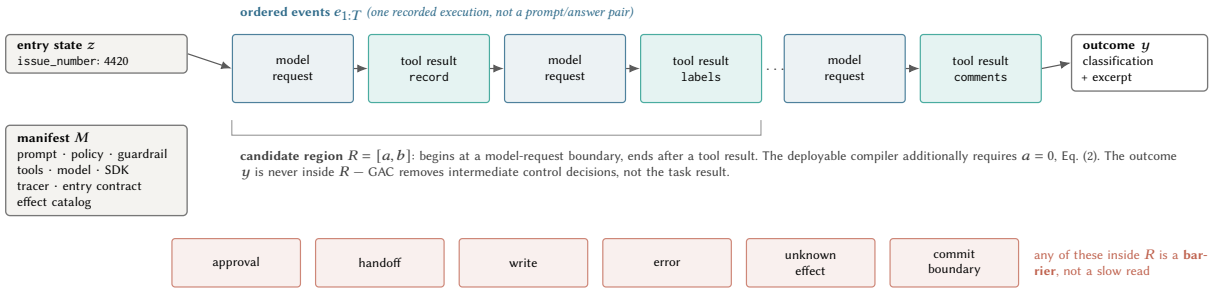


Figure 2: Anatomy of one episode, and of a candidate region inside it. An episode is a typed, version-pinned execution record rather than a prompt/answer pair: the manifest M pins the identities a learned artifact may later be reused under, and the events $e_{1:T}$ retain the tool receipts a compiler would have to reconstruct arguments from. A candidate region is a bounded segment of that record, and it never contains the outcome y. The icons in the lower strip are examples of event kinds that act as barriers; they are not an exhaustive state machine.

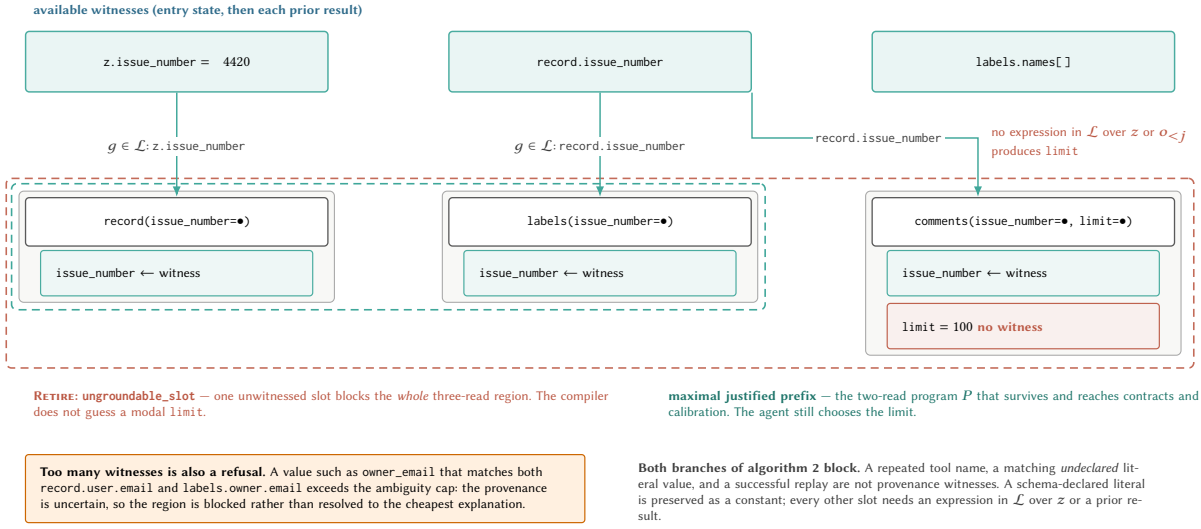


Figure 3: Equation (1) applied to the held-out issue-4420 record. Each nonliteral argument slot needs an expression in $\mathcal{L}$ over the entry state or a prior result; the green edges carry that expression, and the `limit` socket has no incoming edge at all. The diagram therefore asks a strictly harder question than recurrence: not "did this sequence repeat?" but "can every socket be wired from an allowed prior value?" One unwitnessed slot retires the whole three-read region, and a slot with too many witnesses does the same — both branches of algorithm 2 block rather than resolving to the cheapest available explanation. The ambiguity inset is illustrative; the wiring and the refusal are the executed compiler's decision on that record.

each fixed candidate, and retains nondominated survivors; it neither establishes global optimality nor provides a compiler-wide guarantee for that search. The two coarse terms in (6) can therefore change which candidates receive synthesis effort, even though neither participates in an admission decision. Reading ρ_{eff} from the effect catalog and measuring program size after synthesis are future work rather than semantics the code claims to enforce.

Optimization is not certification. Equation (6) decides where synthesis effort goes; (7) decides whether a target is reachable at all; neither one admits an artifact. Admission is eq. (8) plus the hard guard and the verifier, and RETIRE is a legitimate answer at every stage.

3 GUARDED AGENTIC COMPACTION

GAC is a cascade of independent rejection points (fig. 6). It is worth reading that diagram as a map of refusal boundaries rather than a dataflow: almost every edge in it ends at the unchanged agent, and the shaded band names the conditions under which no amount of evidence changes that. Algorithm 1 states that cascade in one place, because the order the stages run in is itself part of the argument: each stage can only weaken a candidate, never rescue one, and every RETIRE is recorded with the stage that caused it, so rejected candidates stay in the denominator. The rest of this section expands the three stages that carry the safety claim — provenance (algorithm 2), calibration (algorithm 3), and the runtime boundary (algorithm 4). Read algorithm 1 as a one-way security checkpoint:

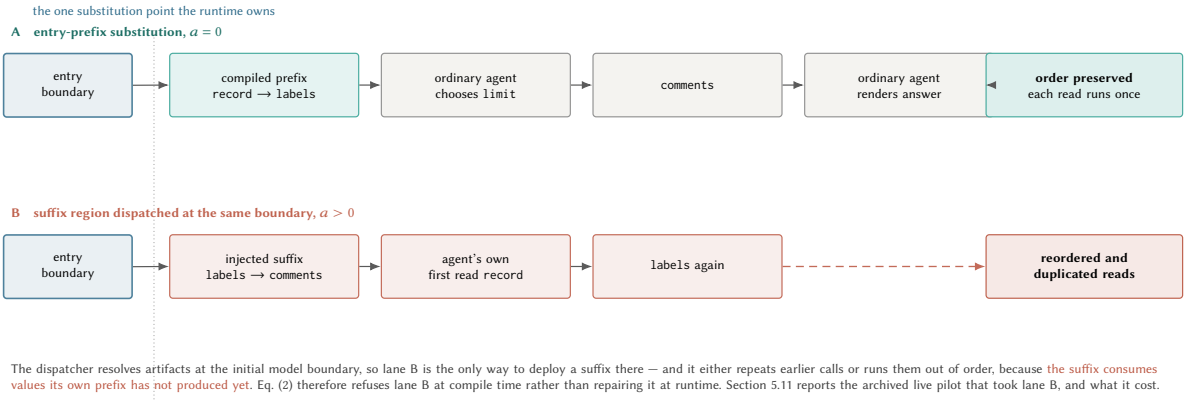


Figure 4: Why prefix-only, drawn as the two orders the runtime can actually produce. Because resolution happens at the initial model boundary, a region with $a > 0$ can only be deployed by injecting it there, which either repeats reads the agent will make anyway or runs them before the values they depend on exist. This panel depicts the current runtime placement and carries no measurement: the archived live pilot that took the lower lane, and what it cost, is reported in section 5.11.

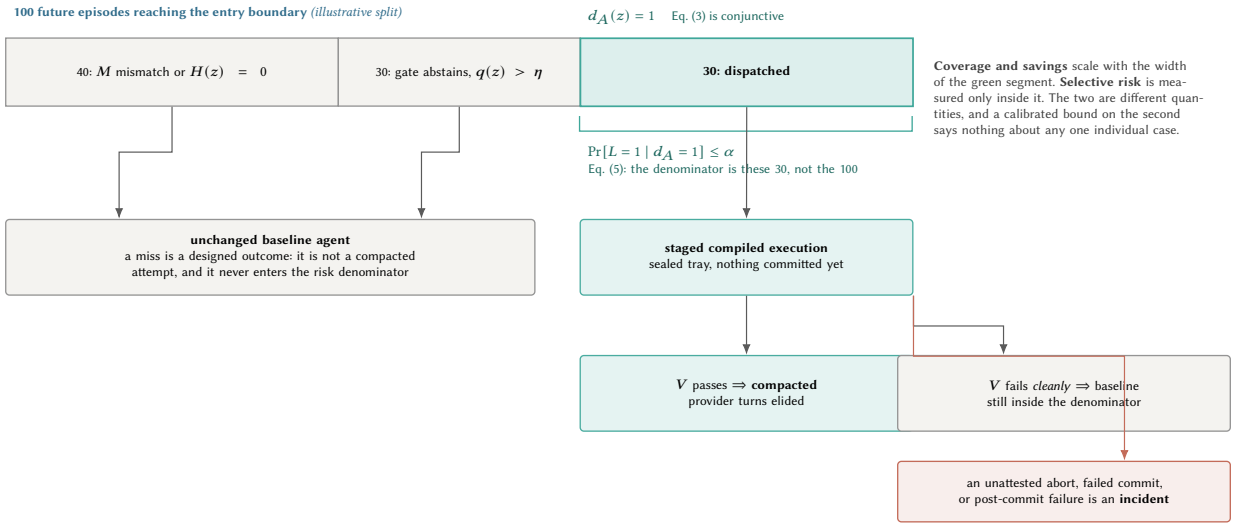


Figure 5: The population that (3) and (5) quantify over, and the slice the risk constraint conditions on. Savings are earned across the dispatched segment, while selective risk is measured *inside* it: an episode that misses on the manifest, the guard, or the gate returns the baseline and never enters the denominator, so it is not a compacted failure. A staged failure caught before the commit line also returns the baseline, but it does stay in the denominator. A failure whose effects cannot be proved clean raises an incident. The 40/30/30 split is illustrative and is drawn only to separate the two quantities; no coverage rate is claimed here.

a later stage may reject a candidate an earlier one passed, but no later stage can authorize one a prior stage blocked.

3.1 Typed provenance and canonical families

A capture adapter normalizes framework traces into the Episode IR. The OpenAI Agents SDK is a natural substrate because its agent loop records model calls, tools, agents, guardrails, and handoffs [37]; the compiler does not depend on the SDK. The artifact persists normalized episodes as canonical local JSONL with atomic snapshot replacement and strict validation. We intentionally removed an unused MLflow adapter: none of the experiments consumed its search or tracking surface, while retaining it added a second compatibility and privacy boundary. This simplifies reproduction but gives up server-side trace search and leaves framework neutrality demonstrated by the typed seam and dependency layering, not by two independently validated foreign-trace mappings. Manifests pin prompt, policy, guardrail, tools, model, SDK, tracer, entry contract, and effect-catalog identities. Episodes with different compatibility keys are partitioned before learning.

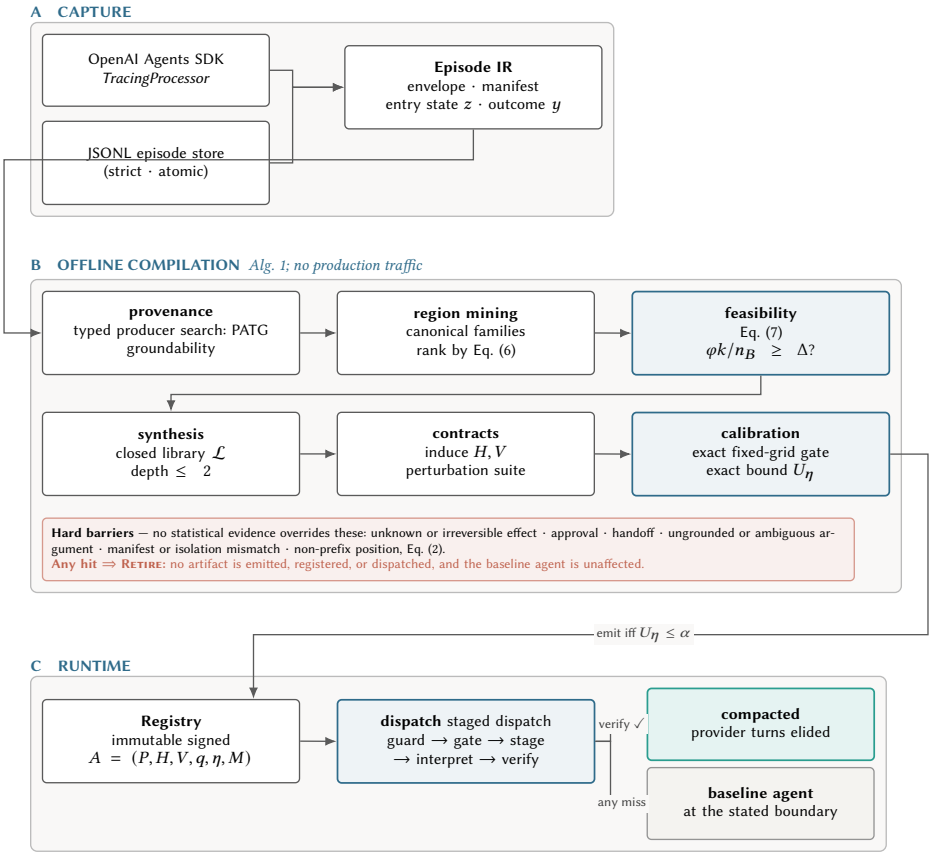


Figure 6: System architecture of GAC. (A) Capture normalizes framework traces into one typed Episode IR. (B) Offline compilation touches no production traffic and is a cascade of independent rejection points; the shaded *barrier* band lists conditions under which no statistical evidence can license an artifact. (C) Runtime resolves at the entry boundary. Of the terminal runtime edges, exactly one produces a compacted execution; every clean rejection returns the baseline and any failure whose reversibility cannot be attested raises an incident. "Baseline" is exact only where a staging owner holds the commit boundary: the outer runner can restore byte-identical model input, whereas the model-boundary adapter cannot un-emit a response the host has already committed to history (section 3.8.1).

Above that seam, an optimization-pass API composes GRC and trace-guided workflow specialization without coupling either algorithm to the SDK. The evidence portfolio is a later decision layer: it consumes paired measurements for named actions and cannot synthesize, approve, or execute an unmeasured macro. This separation keeps trace capture, program construction, empirical action selection, and runtime lifecycle as independently auditable boundaries.

For each call-argument slot, the program-argument trace graph (PATG) searches entry-state and prior-result paths for typed values and bounded transforms. Schema-declared literal-only slots are preserved as constants; exact matches, stable field paths, and transformations in $\mathcal{L}$ become candidate producer edges for every other slot. Ambiguous candidate sets above a configured cap and ungrounded values block a region. This is execution provenance in the database sense [10], specialized to tool arguments rather than tuple derivations. Algorithm 2 states the search. Its two rejection branches carry the design commitment: a slot with no witness is a genuine model decision and a slot with too many witnesses is ambiguous, and both block the region instead of resolving to the cheapest available explanation.

The miner enumerates model-boundary-to-result windows with at most $w_{\max}$ tool calls, qualifies effects and live-ins, and hashes the tool signatures, dependency topology, run shape, and live-in/out shape while abstracting literal values. Families require support across independent scenario groups rather than raw event frequency; the implementation also supports a minimum-distinct-days requirement, which the live study of section 5.4 does not exercise (it sets `min_days` = 1 over a single-snapshot corpus). With N episodes, T events, and a bounded call window $w_{\max}$, enumeration is practically bounded by the number of reachable result boundaries; the implementation's nested scan is worst-case $O(NT^2)$ when non-call spans are adversarial. We report this rather than the stronger $O(NT)$ claim sometimes suggested by the bounded-call intuition.

Surviving families are ranked by an MDL-inspired trade-off that pays for support-weighted savings and charges for heterogeneity,

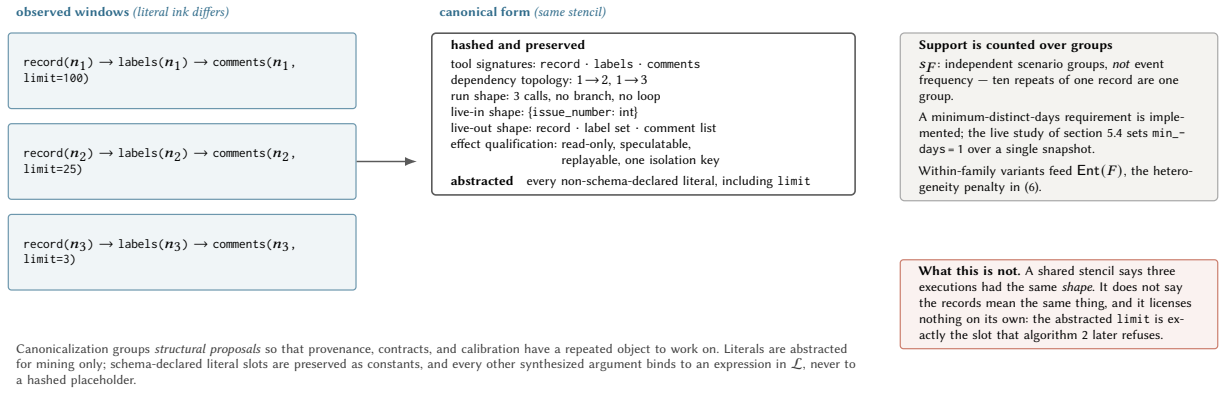


Figure 7: What a canonical family is, and what it is not. Hashing the tool signatures, dependency topology, run shape, and live-in/out shape while abstracting literals groups *structural proposals* so that provenance, contracts, and calibration have a repeated object to work on. Support is counted across independent scenario groups rather than by event frequency. A shared stencil is not evidence that the records mean the same thing, and it licenses nothing on its own: the abstracted `limit` is exactly the slot algorithm 2 goes on to refuse. The three observed windows are written with symbolic identifiers because only issue 4420 is a reported held-out record.

Algorithm 1 COMPILE — the compile-or-retire cascade. Every candidate is either registered or assigned an attributed RETIRE, so refusal accounting remains explicit. Stages marked (CALLER) are separate library entry points run in this order rather than steps inside one function.

Require: episodes $\mathcal{E}$; effect catalog C; entry schema Θ; transform library $\mathcal{L}$; ambiguity cap κ; budgets α, δ; grid Λ; feasibility target Δ
Ensure: artifact set $\mathcal{A}$, or RETIRE with attributed reasons

$\mathcal{A} \leftarrow \emptyset$
$\mathcal{E} \leftarrow \textsc{Qualify}(\mathcal{E}, C)$; $\{\mathcal{E}_M\} \leftarrow \textsc{PartitionBy}(\mathcal{E}, \text{manifest, principal, isolation})$ ▷ drop partial runs, unpinned manifests
for all partitions $\mathcal{E}_M$ **do**
  $(\mathcal{T}, \mathcal{D}, \mathcal{K}, \mathcal{S}) \leftarrow \textsc{GroupedSplit}(\mathcal{E}_M)$ (CALLER) ▷ train / dev / calibration / sealed test, split by *group*
  $G \leftarrow \textsc{BuildPatg}(\mathcal{T}, \Theta, C, \mathcal{L}, \kappa)$ ▷ algorithm 2: block ungrounded or ambiguous slots
  $\mathcal{F} \leftarrow \textsc{Canonicalize}(\textsc{MineRegions}(G, C, w_{\max}))$, keeping only cross-group support
  if $\textsc{Ceiling}(\mathcal{T}, \mathcal{F}) < \Delta$ **then** (CALLER)
    record RETIRE **(infeasible ceiling, Eq. 7)**; **continue** ▷ try the next compatible partition
  end if
  for all families $F \in \mathcal{F}$ ranked by Eq. (6) **do**
    $P \leftarrow \textsc{Synthesize}(F, \mathcal{L})$ at depth ≤ 2; **if** $P = \perp$ **then record** RETIRE **(synthesis); continue**
    $(H, V) \leftarrow \textsc{InduceContract}(F)$; **if** $\neg\textsc{ReplayAndChallenge}(P, V, \mathcal{D})$ **then record** RETIRE **(counterexample); continue**
    $q \leftarrow \textsc{FitScore}(\mathcal{D})$; **freeze** q ▷ dev groups only; never calibration
    $\eta \leftarrow \textsc{Calibrate}(P, H, V, q, M, \mathcal{K}, \Lambda, \alpha, \delta)$; **if** $\eta = \perp$ **then record** RETIRE **(risk); continue** ▷ algorithm 3
    $\mathcal{A} \leftarrow \mathcal{A} \cup \{\textsc{Package}(P, H, V, q, \eta, M)\}$ with evidence and lifecycle
  end for
end for
return $\textsc{SelectUndominated}(\mathcal{A})$ if nonempty, else RETIRE

Algorithm 2 BUILDPATG — typed argument provenance. A slot with no witness is a slot unresolved in the bounded library; a slot with too many witnesses is ambiguous. Both block the region rather than defaulting to the cheapest explanation.

Require: episodes $\mathcal{T}$; entry schema Θ; effect catalog C; transform library $\mathcal{L}$; ambiguity cap κ
Ensure: provenance graph G retaining $1, \ldots, \kappa$ candidate edges per nonliteral grounded slot; declared literals are marked separately

$G \leftarrow \emptyset$
for all episodes $E = (z, M, e_{1:T}, y) \in \mathcal{T}$ **do**
  $\Sigma \leftarrow \{(\text{“}z\text{”}, \pi, v) : (\pi, v) \in \textsc{Flatten}(z), \pi \in \Theta\}$ ▷ only allowlisted entry paths are eligible
  for all tool calls $c_j \in e_{1:T}$ in order **do**
    for all argument slots $u \in \text{args}(c_j)$ **do**
      if u is declared *literal-only* for c_j **then**
        mark (c_j, u) LITERAL; **continue** ▷ preserve the declared constant; infer no provenance
      end if
      $\mathcal{H} \leftarrow \textsc{BestPerSource}\{(\sigma, g) \in \Sigma \times \mathcal{L} : g(\text{value}(\sigma)) = u, \text{depth}(g) \le 2\}$
      if $\mathcal{H} = \emptyset$ **then**
        mark (c_j, u) MODEL-ORIGINATED; **block** region
      else if $|\mathcal{H}| > \kappa$ **then**
        mark (c_j, u) AMBIGUOUS; **block** region
      else
        $G \leftarrow G \cup \{\sigma \xrightarrow{g} (c_j, u) : (\sigma, g) \in \mathcal{H}\}$ ▷ synthesis chooses a stable source across groups
      end if
    end for
    $\Sigma \leftarrow \Sigma \cup \{(\text{var}(c_j), \pi, v) : (\pi, v) \in \textsc{Flatten}(o_j), \textsc{Eligible}(\pi, v)\}$ ▷ a result witnesses only *later* slots
  end for
  for all pairs (e_s, e_t) sharing a resource where either writes it **do**
    $G \leftarrow G \cup \{e_s \prec e_t\}$ ▷ order edge: no reordering is ever proposed across a conflict
  end for
end for
return G

effect exposure, and program size:

$$\text{score}(F) = \underbrace{s_F\, \bar{k}_F\, c_m}_{\text{support-weighted saving}} - \lambda_1 \text{Ent}(F) - \lambda_2 \rho_{\text{eff}}(F) - \lambda_3 |F|, \tag{6}$$

where s_F is the number of supporting train groups, $\bar{k}_F$ the mean removable model requests, c_m their unit cost, and $\text{Ent}(F)$ the Shannon entropy over canonical variants inside the family. The penalties matter: without λ_1 the ranking prefers a heterogeneous family whose single canonical form fits none of its members.

Two terms are coarser in the implementation than the notation suggests, and we state them as implemented. ρ_{eff} is intended as the declared-effect exposure of the region, but the shipped ranking approximates it by the fraction of tool names containing a namespace separator — a naming convention, not a reading of the effect catalog. $|F|$ is intended as the size of the program the family would produce, but ranking precedes synthesis, so the implementation substitutes the argument-slot count of the first observed window. Both affect ranking and hence the candidate tested, but neither enters the risk bound. The effect catalog is checked in qualification and in the runtime facade. Making them faithful would reorder candidates and therefore require re-running the offline study; we list it as future work rather than describe semantics the code does not enforce.

Before any synthesis runs, a feasibility estimator bounds what the compiler could achieve even with a perfect verifier and a gate that never abstains:

$$\Delta_{\max} = \frac{\varphi\, k}{n_B}, \qquad \text{necessary:} \quad \varphi\, \rho\, k \;\geq\; \Delta\, n_B, \tag{7}$$

with n_B baseline model requests per episode, φ the fraction of episodes holding at least one eligible region, k the requests removed per successful dispatch, and ρ the verifier pass rate. Equation (7) bounds requests removed by the prefix at fixed continuation workload. Changes to downstream reasoning or fallback can change total requests beyond that calculation. For the fixed-workload example in section 5.4: with $\varphi = 1$, $k = 3$ and $n_B = 4$ the ceiling is 0.750 and the measured reduction is 75.0%. Reaching this ceiling measures boundary removal on this workload; it does not establish usefulness on workflows with less recurrent structure. Reporting the ceiling first makes a decisive negative answer cheap: a workload whose ceiling is under the target can be declined without building a compiler for it.

3.2 Bounded synthesis and contracts

Program synthesis searches a 23-operator library with depth at most two. Operators cover path selection, indexing, string normalization, arithmetic, comparisons, bounded collection operations, and narrow loops. Candidate bindings are ranked by stability across groups; a group-wise refit rejects bindings that exploit row-level coincidence. The approach resembles programming-by-example [17] and bounded sketching [42], but the hypothesis space is deliberately small enough to inspect. It does not generate arbitrary Python.

Hard guards constrain entry types, required fields, categorical sets, numeric intervals, patterns, isolation keys, allowed effects, and manifest pins. Verifiers constrain live-out presence, type, cardinality, empirical hulls, provenance, effect multisets, and call counts. These are likely invariants [15], not proofs for all future inputs. Replay and perturbation are therefore available to challenge a candidate before calibration, but they are only as strong as the substrate supplied: without a sandbox the suite cannot run at all, and the artifact then records `perturbations_claimed: false` rather than implying a challenge occurred. A missing sandbox is therefore a recorded absence of challenge, not a silent pass. The headline live artifact of section 5.4 is in exactly that position, and we report it there as a limitation of that run rather than as a property of the method.

3.3 Worked example: why the compiler stops after two reads

Consider held-out issue 4420 in the expanded 30-record study. The entry state is only $z = \{\texttt{issue_number} = 4420\}$. Its observed trace calls `record(4420)`, passes the returned `record.issue_number` to `labels` and `comments`, and uses `limit=100` for the latter. The following applies the executed compiler rule to that record; it is not a separate experiment.

Table 1 walks the record from left to right: what the trace contains, and what the compiler is entitled to conclude from it.

This also shows why replay alone is insufficient. A hard-coded comment limit can replay the observed source calls yet alter the evidence seen by the final model. In the earlier three-read follow-up, tool replay passed but one answer lost a required URL. Provenance decides what can be constructed; contracts and the gate decide whether it may be used. The two checks are worth naming apart: *tool replay passed* is a statement about the source calls a program reproduces, while *continuation answer preserved* is a statement about the answer the model then writes. The earlier 18-record cohort satisfied the first and failed the second on issue 6602, and it is a separate study with a separate artifact — calibrated at $\alpha = .10$ — not an earlier run of the two-read artifact described here.

3.4 Guarded composite synthesis

The macro comparison revealed a structural asymmetry: the manual baseline may redesign the application interface, while the original compiler preserves every source-tool observation. GCS closes that specific gap without moving application code into the optimizer's trust boundary. After ordinary synthesis and admission, it packages the same program as $\widetilde{P} = (P, \Pi, \tau_c)$, where P is the verified internal program, Π is a declared projection over verified live-outs, and τ_c is the exact compatibility key of the continuation that consumes the projection. A tool must carry an explicit `batchable` capability before it can appear inside this interface. The outer composite is a declared *view* of verified internal receipts, not a new source operation, and fig. 10 draws it that way.

Some recurrent calls differ only in arguments that are irrelevant to the registered task. The effect catalog may therefore attach a signed, declarative task-semantic canonicalization to a dotted argument path. The implementation admits only five closed operations (integer clamping, whitespace stripping, case folding, stable set-like sorting, and finite aliases), and integer rules may declare an admissible input domain. This is not learned equivalence: values outside that domain raise and deoptimize. In the GitHub task, the consumer uses at most the first three comments and the snapshot tool already caps its result at three, so observed limits of at least three share the representative `limit=3`; `limit=1` is deliberately outside the contract. The intuition to keep is that this is a *declaration the compiler enforces*, supplied and signed by the application and bounded to a stated domain, rather than an equivalence the compiler inferred from traces — which is why a value outside the domain raises instead of being folded into the representative.

The projection language reuses the bounded binding DSL. It can select only existing verified live-outs and cannot call arbitrary code, resolve a dynamic tool, or access unverified state. Runtime execution retains all internal source calls, effects, and field-level

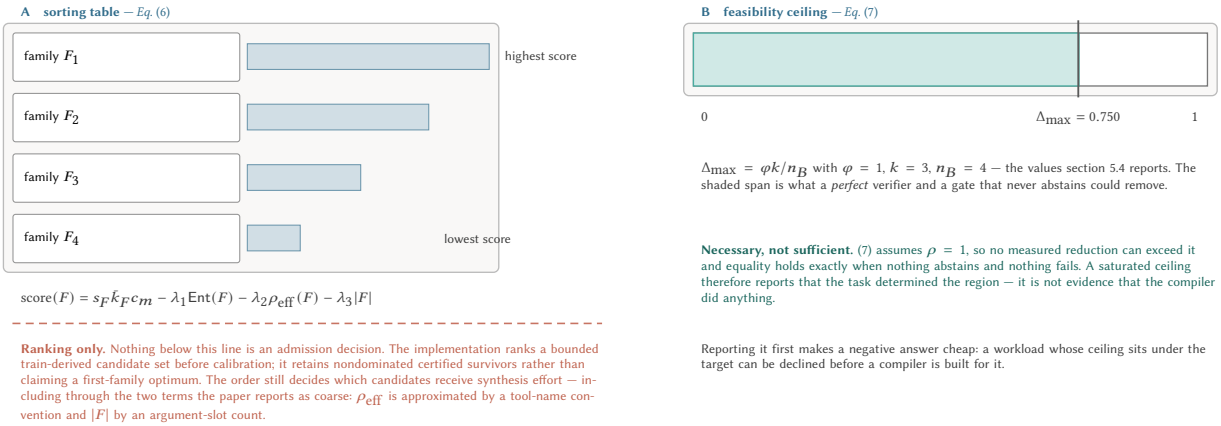


Figure 8: Two pre-synthesis questions, neither of which is admission. (A) Equation (6) orders candidates so that synthesis effort goes somewhere useful. It affects which fixed candidates are tried, but the risk bound does not use the ranking score and the compiler retains nondominated calibrated survivors rather than claiming a first-family optimum. (B) Equation (7) bounds what a perfect verifier and a never-abstaining gate could remove. It is necessary, not sufficient: a saturated ceiling reports that the task determined the region rather than that the compiler succeeded. The gauge is drawn at the values section 5.4 reports.

Table 1: The executed compiler rule applied to held-out issue 4420, with the formal object responsible for each decision. The left column is the observed record; the right column is the entitlement it produces. The decisive line is the third: one argument slot without a witness retires the *whole* region, and the compiler keeps the justified prefix instead of guessing the missing value or discarding the episode. Nothing in this table is a separate experiment.

Observed trace	Compiler decision, and what licenses it
Entry state z = {`issue_number = 4420`}; manifest M pins the workflow	**Propose** (E, M, z). All 132 discovery executions take the same route, so the normalized traces form one candidate family. Recurrence establishes a pattern, not permission to deploy it.
`record(4420)`	**Trace backward** (Eq. (1), algorithm 2). The argument is `z.issue_number` — a witness in the closed binding language.
`labels(record.issue_number)`	Same verdict through the stable result path `record.issue_number`.
`comments(record.issue_number, limit=100)`	**Refuse** the region. The observed `limit` varies across otherwise similar entries and is neither in z nor recoverable from `record` or `labels`. With no witness for that slot the full three-read region is rejected as `ungroundable_slot`; the compiler does not guess a modal value.
—	**Emit the maximal justified prefix** (P). `record(z.issue_number) -> labels(record.issue_number)` is groundable, read-only, and at the entry boundary, so it reaches the induced contracts H and V and the manifest pins M.
92 held-out calibration groups	**Calibrate** (q, η, U_η). All 92 are admitted with zero violations at $\alpha = .05$, $\delta = .10$ and $\lvert\Lambda\rvert = 11$, for simultaneous upper bound 0.0498 (Eq. (8)). This is a per-candidate threshold-grid certificate, not a proof of semantic correctness.
A future compatible record	**Dispatch** (d_A). A matching, low-score entry receives the two-read prefix in staging. The ordinary agent still selects the comment limit and renders the answer, so the decisions the evidence did not determine stay with the model.
Anything else	**Decline**. A manifest or guard mismatch, a failed verification, or a failed risk gate runs the unchanged agent.

provenance. Verification and projection both complete inside the staging boundary; a missing projected field returns the baseline before release. A pre-model dispatch additionally requires an exact continuation-manifest match. It then exposes one native composite observation to the provider, so the provider receives the sufficient record on its first request rather than spending a request selecting the composite.

GCS is consequently narrower than automatic API fusion. It does not parallelize the internal calls, reduce the number of source reads, generate a new remote service, or prove the application-supplied task-semantic contract. Each of those non-claims is visible in fig. 10 rather than only asserted here: the three source reads are drawn inside the staging box, in sequence, with their receipts retained. Its contribution is a serializable, fail-closed interface

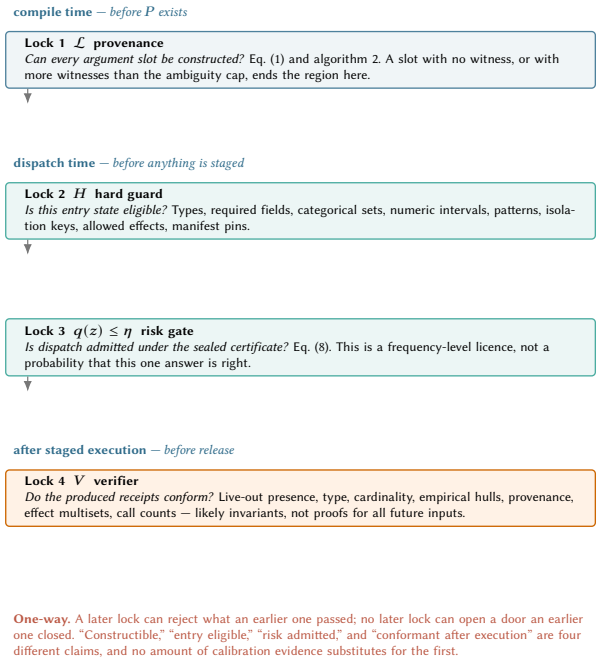


Figure 9: Four claims, tested at three times, in one direction. Provenance licenses *construction* before P exists; the hard guard licenses an *attempted* execution at the entry boundary; the calibrated gate licenses the *frequency* of that attempt under its assumptions; and the verifier checks the staged result before release. "Constructible," "entry eligible," "risk admitted," and "conformant after execution" are independent, and no later lock reopens a door an earlier one closed.

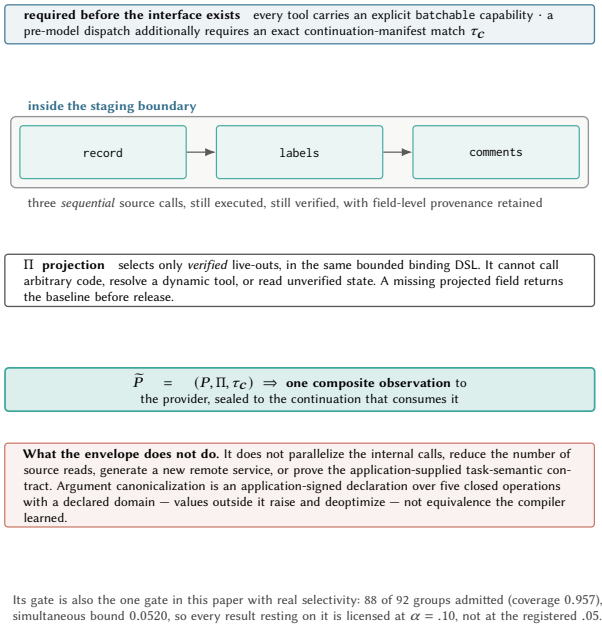


Figure 10: Guarded composite synthesis as a sealed envelope. The three source reads are still executed, still serial, and still verified with field-level provenance; Π exposes only verified live-outs, and τ_c seals the result to the continuation that consumes it. What changes is the interface the provider sees, not the work done behind it. Its gate is also the only one in this paper that rejects a calibration group at its selected threshold, which is why every result resting on it is licensed at $\alpha = .10$ rather than the registered .05.

transformation over a program the existing compiler can already verify, and every result that uses it in this paper is a 10%-risk result.

3.5 Fair manual and learned-optimizer interfaces

To separate runtime placement from automatic discovery, we implement a second pre-model runner that does not consume a compiler artifact or its calibrated gate. It accepts an explicit, hand-authored read program, source and continuation manifests, effect catalog, bounded projection, and output/provenance/call-count verifier. Manifest drift, an undeclared effect, a missing field, a repeated observation, or any verifier failure returns the unchanged agent before release. This is a guarded laboratory baseline, not an independently reviewed production program; its construction cost is not measured.

For learned prompt optimization we wrap the official GEPA `optimize_anything` interface behind a provider-agnostic adapter. The adapter requires disjoint train and validation identifiers, enforces hard proposal, metric-call, and candidate-length budgets, records a sanitized audit log, and leaves tracking disabled. Only one operational-strategy sentence is mutable; the factual output schema, safety instructions, tools, grader, and quality-dominant objective remain fixed. Prompt optimization and guarded program execution therefore change orthogonal variables and can be evaluated alone or together. Optimization requests, tokens, latency, and cost are accounted separately from held-out evaluation.

3.6 Exact risk-gated admission

The gate is a conservative admission test over a fixed menu of thresholds, not a confidence score that proves an individual output correct. Two data signals do two different jobs in it, and conflating them is the easiest way to misread the guarantee: the score is *fitted* on development groups that turned out unproductive in any way — wrong or abstained — and then frozen, while the bound is *counted* only from groups that were dispatched and turned out wrong.

The score $q(z)$ is a logistic model over entry-observable features: unseen category, distance proxy, missing fields, hull margin, temporal drift, provenance ambiguity, and branch entropy. It is trained on development-group *unproductive* outcomes (wrong or abstained), then frozen. Safety calibration uses only *violations* (wrong after dispatch). The fixed threshold grid is

$$\Lambda = \{.02, .05, .08, .11, .14, .17, .20, .25, .30, .40, .50\}.$$

For threshold η, let n_η calibration groups be admitted and k_η contain at least one violation, and write $\gamma = \delta/|\Lambda|$. The compiler

Algorithm 3 CALIBRATE — fixed-grid exact selective admission. The score q and the grid Λ are frozen *before* this procedure sees a calibration group, which is what makes the union bound legitimate.

Require: frozen candidate $A_0 = (P, H, V, q, M)$ without threshold; calibration groups $\mathcal{K}$ with fixed per-instance violations $L(A_0, x)$; grid Λ; budgets α, δ
Ensure: threshold η with simultaneous guarantee, or $\perp$ (RETIRE)
$\mathcal{A} \leftarrow \emptyset$ ▷ admissible (coverage, threshold) pairs
for all $\eta \in \Lambda$ **do**
  define $d^\eta(x)$ by eq. (3) with threshold η; $D^\eta(g) \leftarrow \max_{x \in g} d^\eta(x)$; $W^\eta(g) \leftarrow \max_{x \in g} d^\eta(x) L(A_0, x)$
  $K_\eta \leftarrow \{ g \in \mathcal{K} : D^\eta(g) = 1 \}$; $n_\eta \leftarrow |K_\eta|$; **if** $n_\eta = 0$ **then continue** ▷ vacuous bound $U_\eta = 1$
  $k_\eta \leftarrow |\{ g \in K_\eta : W^\eta(g) = 1 \}|$ ▷ *wrong after dispatch* only, never abstention
  $U_\eta \leftarrow 1$ if $k_\eta = n_\eta$, else $\mathrm{Beta}^{-1}\big(1 - \frac{\delta}{|\Lambda|}; k_\eta + 1, n_\eta - k_\eta\big)$; **if** $U_\eta \le \alpha$ **then** $\mathcal{A} \leftarrow \mathcal{A} \cup \{(n_\eta / |\mathcal{K}|, \eta)\}$ ▷ eq. (8)
end for
return $\perp$ if $\mathcal{A} = \emptyset$, else the η component of $\mathrm{lexmax}_{(\mathrm{cov}, \eta) \in \mathcal{A}}(\mathrm{cov}, \eta)$ ▷ largest empirical group coverage; larger threshold breaks ties

computes the one-sided Clopper–Pearson upper bound [13]

$$U_\eta = \begin{cases} 1, & n_\eta = 0 \text{ or } k_\eta = n_\eta, \\ \mathrm{Beta}^{-1}(1 - \gamma; k_\eta + 1, n_\eta - k_\eta), & 0 \le k_\eta < n_\eta. \end{cases} \tag{8}$$

For $k_\eta = 0 < n_\eta$, this reduces to $U_\eta = 1 - \gamma^{1/n_\eta}$. The largest-coverage admissible threshold with $U_\eta \le \alpha$ is selected; if none exists, the family retires. The explicit edge cases match the implementation and prevent an empty admitted set or an all-violation set from producing an undefined beta quantile. Algorithm 3 gives the selection loop. The ordering in it is the load-bearing part: the score and the grid are frozen before the procedure reads a single calibration group, which is what makes the union bound over Λ legitimate. Figure 11 runs that selection on two sealed gates.

PROPOSITION 1 (PER-CANDIDATE SELECTIVE-RISK ADMISSION). *Condition on train/development data and fix one candidate (P, H, V, q), group-level violation rule, and finite grid Λ before observing calibration. If calibration groups are i.i.d. from the future-group distribution, then with probability at least $1 - \delta$, every $\eta \in \Lambda$ satisfies $r_\eta \le U_\eta$. For positive population admission, define*

$$r_\eta = \Pr(W^\eta = 1 \mid D^\eta = 1);$$

otherwise define $r_\eta = 0$. Consequently the selected $\hat{\eta}$ satisfies $r_{\hat{\eta}} \le \alpha$ whenever $U_{\hat{\eta}} \le \alpha$.

PROOF. Condition on the train/development data, fix $\eta \in \Lambda$, and write $\gamma = \delta / |\Lambda|$. Let D_i^η and W_i^η be the group admission and violation indicators defined in section 2. Because the entire candidate and grid were frozen before calibration and groups are i.i.d., the pairs (D_i^η, W_i^η) are i.i.d. Conditional on $n_\eta = m > 0$ admitted groups, their violation count therefore satisfies $k_\eta \mid n_\eta = m \sim \mathrm{Binomial}(m, r_\eta)$, where $r_\eta = \Pr(W^\eta = 1 \mid D^\eta = 1)$. Inverting the one-sided exact binomial test [13] gives the upper bound $U_\eta = \mathrm{Beta}^{-1}(1 - \gamma; k_\eta + 1, m - k_\eta)$ of eq. (8), which satisfies $\Pr\{r_\eta > U_\eta \mid n_\eta = m\} \le \gamma$. When $m = 0$ the event is impossible under the convention that empty-admission thresholds receive the vacuous bound $U_\eta = 1$. Averaging over the random value of n_η therefore yields $\Pr\{r_\eta > U_\eta\} \le \gamma$ for each fixed threshold. Applying the union bound over the finite grid gives

$$\Pr\big\{\exists \eta \in \Lambda : r_\eta > U_\eta\big\} \le |\Lambda| \gamma = \delta.$$

On the complementary simultaneous event, every threshold whose upper bound is at most α satisfies the group-level target in eq. (5), including the threshold returned by algorithm 3. □

This proposition is deliberately *per fixed candidate*. The compiler may calibrate several train-fixed candidate families and retain survivors, but its current confidence budget is split across thresholds rather than candidate families, so (8) alone does not provide a compiler-wide guarantee for that outer search. A direct Bonferroni repair for m candidates would use $\gamma = \delta / (m|\Lambda|)$; at $m = 2$, $\alpha = .05$, $\delta = .10$, and zero violations, the requirement rises from 92 to 106 admitted groups. The retained experiments are therefore reported with per-candidate conditional certificates, not a multiplicity-corrected compiler-wide certificate. Figure 12 draws that boundary as a fence around the proposition rather than as a caveat beside it.

The fence has a gate, and the compiler already ships it: `GrcConfig`'s `freeze_one_candidate_before_calibration` flag ranks families on train data, synthesizes and challenges each on train and development windows only, and halts the search the instant any candidate reaches the calibration stage — admitted, retired, or lacking calibration groups — without ever evaluating a second candidate against calibration windows. That is precisely proposition 1's hypothesis, applied to the compiler's entire search rather than to a single candidate assumed fixed in advance.

COROLLARY 2 (COMPILER-WIDE GUARANTEE UNDER FROZEN SELECTION). *If algorithm 1 runs with* `freeze_one_candidate_before_calibration` *enabled, proposition 1's guarantee holds for the compiler's actual reported output with no adjustment to $\gamma = \delta / |\Lambda|$: no Bonferroni correction over candidates is needed, because the run leaves no candidate multiplicity to correct for.*

PROOF. By construction, family ranking is a function of train windows alone; synthesis and the challenge test are functions of train and development windows alone; and the frozen score model q is fit on development samples before a single calibration window is inspected. The search reaches the calibration stage for the top-ranked surviving candidate exactly once and halts immediately after that candidate's outcome — admission, retirement for want of an admissible threshold, or absence of calibration groups — with no other candidate's program, guard, verifier, or score model ever compared against a calibration window. Proposition 1's precondition, "fix one candidate…before observing calibration," is therefore satisfied by the run's entire candidate search, not by a hypothetical single-candidate sub-procedure, so its conclusion transfers unchanged. □

Corollary 2 is a corollary, not a new inequality: every bit of its statistical content is proposition 1's, restated for a search that never lets a second candidate see calibration data. It is also not hypothetical here. The cross-repository extension of section 4.7 already runs with freezing enabled, and every sealed repository report there shows exactly one candidate reaching calibration, so corollary 2 applies to those admissions directly. Conditional on train and development data, the number m of candidates that reach calibration is fixed before any calibration label is read. Issue-type routing did not freeze, but its second candidate retired at synthesis (an ungroundable slot) and never saw calibration, so $m = 1$ and the

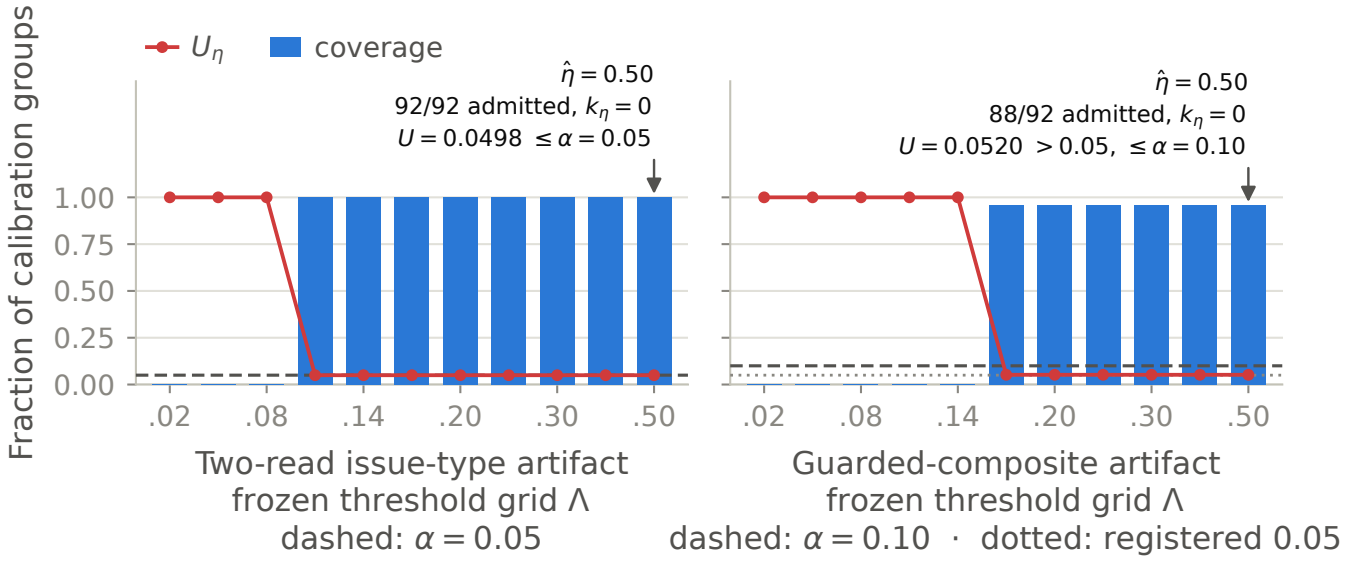


Figure 11: Fixed-grid selection on two sealed gates, from frozen grid to counts (n_η, k_η) to the bound U_η to the admitted threshold. Coverage and U_η are both fractions, so they share one axis. Left: the two-read issue-type artifact. It is degenerate — it admits every calibration group as soon as it admits any — and clears the registered budget. Right: the guarded-composite artifact, the only gate here that rejects groups at its selected threshold; the same selectivity puts n below the 92 a zero-violation 5% bound needs, so its bound is 0.0520 and it is admitted only at $\alpha = .10$. Zero observed violations never produces a zero bound: $\gamma = \delta/|\Lambda|$ pays for the union bound over the grid, and reading the counts before choosing $\hat\eta$ is legitimate only because the candidate, the score, and the grid were frozen first.

same argument applies. PR-outcome and backlog-attention routing each calibrated $m = 2$ candidates on the same 92 groups and kept the higher-removal survivor by dominance: their certificates are per-candidate at $\alpha = 0.05$, and under $\gamma = \delta/(m|\Lambda|)$ the retained zero-violation tables give $U = 0.0569 > \alpha$, a compiler-wide certificate only at $\alpha \ge 0.057$ or after 106 groups. A fresh 106-group calibration under the pre-registered protocol `multiplicity-repair-protocol.md` is the repair; it failed its own go/no-go check (after excluding every record used in a retained cohort, the pinned snapshot holds 33 unused open pull requests and 8 unused owned backlog issues against the 36 per class the design requires), so it was not run and the per-candidate wording stays.

A provider-free simulation (`frozen_candidate_coverage_simulation.py`) quantifies exactly what freezing buys and what an uncorrected search would spend. Its first part is exact combinatorics, not Monte Carlo: for $n = 92$ calibration groups it locates the population violation rate that maximizes a single candidate's Clopper–Pearson miscoverage probability ($r^\star \approx 0.637$, miscoverage ≈ 0.00909, matching $\gamma = \delta/|\Lambda|$ to four digits, which is the guarantee's own defining property rather than a new fact) and then asks what happens if m such candidates are each calibrated independently at that same per-candidate budget with no correction. Table 2 shows the consequence: the union probability that *some* reported certificate is wrong grows with m, crossing the registered $\delta = .10$ once m exceeds the eleven-point grid it shares the budget with, while the Bonferroni correction already stated above holds every tested m under 1% at the published cost in admitted groups. The same script independently recomputes the $m = 1$ and $m = 2$ group requirements (92 and 106) from this combinatorial argument, matching the figures already reported above rather than merely restating them.

Its second part is a seeded Monte Carlo stress test ($M = 200{,}000$ replicates, seed `20260822`) of exactly the independence caveat below: at zero within-block correlation realized miscoverage matches the exact baseline, but drawing the 92 groups in two 46-group blocks that each carry a 0.30 elevated violation rate with probability 0.25 pushes realized miscoverage on the population's marginal rate to 56% against a 0.9% nominal budget, while milder clustering at the same shift probability but a smaller shift leaves it unmeasurable over that many replicates. The magnitude is a property of the stress configuration chosen for contrast, not a measurement of the live studies' true correlation structure, which remains unknown; what it establishes is that the independence assumption is load-bearing by a wide margin once records are drawn correlated rather than merely once, not only in principle.

The guarantee concerns disagreement with the induced verifier V, not semantic task error. It also requires independent group indicators, no distribution shift, and a candidate fixed relative to calibration; clustered data or adaptive candidate/artifact selection needs a different risk allocation. The independence requirement is where our own live configuration is weakest and it is worth naming here rather than only in section 9: the live studies set `min_days = 1` and `min_principals = 1` over a single repository snapshot, so a "group" is one record from one project at one instant. Records drawn that way share a repository, a schema, an author community, and a period, and nothing in the calibration establishes that

Table 2: Exact miscoverage under an uncorrected versus Bonferroni-corrected m-candidate search, at the population rate $r^\star \approx 0.637$ that maximizes a single candidate's Clopper–Pearson miscoverage at $n = 92$. "Groups required" is the zero-violation admission count the correction demands at that m.

Candidates m	Uncorrected union miscoverage	Bonferroni-corrected miscoverage	Groups required
1	0.9%	0.91%	92
2	1.8%	0.54%	106
3	2.7%	0.81%	114
5	4.5%	0.69%	124
8	7.0%	0.55%	133
11	9.6%	0.75%	139
16	**13.6%**	0.52%	146

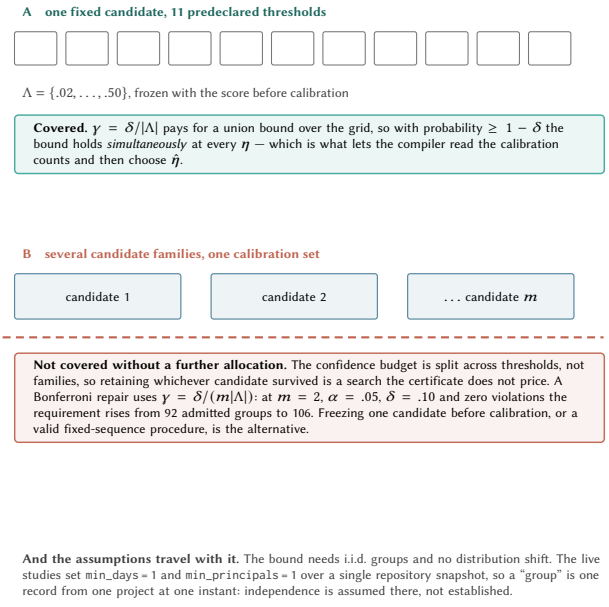


Figure 12: What the proposition licenses, and what it does not. (A) One fixed candidate against the predeclared grid is covered: $\gamma = \delta/|\Lambda|$ buys a bound that holds simultaneously at every η, which is what permits reading the calibration counts before choosing $\hat{\eta}$. (B) Several candidates sharing one calibration set is a search the certificate does not price, and the assumptions travel with the fence: the live configuration makes group independence an assumption the snapshot cannot verify.

their violation indicators are independent. The bound is therefore conditional on a sampling assumption the snapshot cannot verify, and the clustered-group simulation above quantifies rather than only asserts how far a bound can drift once that assumption fails. For one fixed candidate, with $\alpha = .05$, $\delta = .10$, $|\Lambda| = 11$, and zero violations, 92 admitted groups are required. A single pre-committed threshold would require 45, so the NESTFUL refusal is specific to the declared grid and risk configuration, not an intrinsic property of that benchmark. These boundaries are consistent with learn-then-test risk control [3, 7] and are revisited in section 9.

Table 3: Selective-risk configuration of every admitted artifact, generated from the sealed gates rather than restated in prose. These are per-candidate threshold-grid certificates, not candidate-search-adjusted compiler-wide guarantees. $\alpha = .05$ is the paper's registered target; three artifacts were calibrated at $\alpha = .10$ and do *not* meet it. All seven use $\delta = .10$ and the same 11-threshold grid, and all record zero calibration violations. Results resting on a row marked no are licensed only at the 10% selective-risk level.

Admitted artifact	Pairs	α	Admitted	U_η	$U_\eta \le .05$
Prescribed-prefix ablation	18	.05	92/92	0.0498	yes
Natural-order, three-read	18	.10	45/45	0.0992	**no**
Expanded replication (issue type)	30	.05	92/92	0.0498	yes
PR-outcome audit	30	.05	92/92	0.0498	yes
Backlog-attention routing	30	.05	92/92	0.0498	yes
Guarded composite synthesis	12	.10	88/92	0.0520	**no**
Comparator deployment (same GCS artifact)	6	.10	88/92	0.0520	**no**

3.6.1 Which artifact was admitted at which risk level. α is a configured input, not a constant of the method, and this paper does not use one value throughout. Table 3 therefore reports the risk configuration of each admitted artifact next to the study it licenses. The three primary workflow families and the prescribed-prefix ablation record per-candidate calibration at the registered $\alpha = .05$ over 92 zero-violation groups. The earlier three-read natural-order artifact (section 5.4) and the guarded-composite artifact used by sections 5.8 and 5.9 were calibrated at $\alpha = .10$, with simultaneous upper bounds 0.0992 and 0.0520. Neither would have been admitted at .05, so every GCS result in this paper is a 10%-risk result and we mark it as such at each point of use rather than once in an appendix.

The GCS row is the informative one, because it fails the tighter bound for a reason worth stating. It is the only gate in the paper that rejects any calibration group at its selected threshold: it admits 88 of 92 (coverage 0.957) where every other artifact admits all of them. Rejecting four groups reduces n below the 92 that a zero-violation 5% bound requires, so the same discrimination that makes this gate less degenerate than the others is what puts its bound at 0.052. Coverage and risk trade off against each other exactly as the

finite-sample calculation says they must, and at these sample sizes the trade is visible at the first sign of any selectivity at all. The right panel of fig. 11 is that row drawn against the left panel: identical grid, identical δ, zero violations in both, and the only difference is that four groups fall outside the admitted set.

3.7 Risk-bounded transformation portfolio

Compiler admission and application-level choice are distinct. A portfolio layer compares only actions measured on the same independent groups. It forms a declared weighted paired utility over cost, latency, tokens, and tool calls, then applies multiplicity-adjusted exact bounds to both task failure and non-positive utility. Among compatible actions passing both bounds and minimum support, it recommends the highest mean utility; otherwise it returns the baseline. Missing evidence is never imputed, manifest drift invalidates the decision, and macros remain review-required. This layer selects among measured actions; it does not synthesize application code or prove optimality over unmeasured alternatives.

3.8 Runtime and framework integration

At runtime, resolution is a bounded lookup: the registry indexes artifacts by compatibility and partition key, then filters the small candidate list at that key by lifecycle, kind, and signature. Cost is therefore constant in registry size but linear in the number of artifacts sharing one key, which the packaging step keeps small rather than the data structure guaranteeing it. The dispatcher checks lifecycle, manifest, hard guard, calibrated gate, mode, budget, and quota snapshot; executes through a permission facade into a staging area; verifies the result; and commits only on success. Shadow mode records would-dispatch behavior but does not change the agent. Live resolution accepts only active artifacts. The current version compiles reads only; irreversible operations remain with the baseline agent.

Algorithm 4 gives that boundary in full, and reading it against fig. 6 makes the asymmetry explicit: the cheap deterministic checks reject first, the calibrated gate runs only on what survives them, and of the terminal edges exactly one compacts. Every clean rejection returns the unmodified agent; a failed abort, failed commit, or post-commit failure raises an incident rather than feigning a rollback.

For the OpenAI Agents SDK, the library provides trace capture plus a custom model-boundary adapter for local function tools. Streaming, hosted tools, MCP tools, handoffs, loops, and assertions bypass rather than degrade. The SDK provides the loop and lifecycle, but it is not itself an offline optimizer [37].

3.8.1 Where "unmodified baseline" is exact, and where it is not. The two integration paths do not offer the same guarantee, and the difference is structural rather than an implementation gap (fig. 13). The outer runner owns the entry snapshot and the commit boundary, so a rejected attempt restores byte-identical model-visible input: for that path, "the baseline runs unchanged" is exact at every failure point. The model-boundary adapter cannot make the same promise. Once it has returned a synthesized `ModelResponse`, the host runner may already have committed that item to session history, and the `Model` interface provides no way to withdraw it;

Algorithm 4 DISPATCH — boundary-time admission. Cheap deterministic checks reject first and the calibrated gate runs only on what survives them. Exactly one exit path compacts; clean rejections return the unmodified agent, while an unattested or post-commit failure raises an incident.

Require: entry state z; runtime context M'; registry $\mathcal{R}$; effect catalog C; mode $m \in \{\text{off}, \text{shadow}, \text{live}\}$
Ensure: observations for the region, FALLBACK to baseline agent B, or INCIDENT

if $m = \text{off}$ **then**
  return FALLBACK ▷ conformance: model-visible input must be byte-identical
end if
$A \leftarrow \mathcal{R}.\text{RESOLVE}(M'.\text{compatibility}, M'.\text{partition})$
if $A = \bot \lor A.\text{lifecycle} \neq \text{ACTIVE} \lor \neg\text{VERIFYSIGNATURE}(A)$ **then**
  return FALLBACK
end if
if $H_A(z, M') \neq 1$ **then**
  return FALLBACK ▷ manifest pins, isolation keys, typed hulls, effect set
end if
if $\text{tools}(A.P) \cap \text{ALREADYOBSERVED} \neq \emptyset$ **then**
  return FALLBACK ▷ the region already ran; its live-ins are not the fitted ones
end if
if $m = \text{live} \land \exists f \in \text{tools}(A.P) : C(f).\textit{quota_attested} \land \text{SNAPSHOT} = \bot$ **then**
  return FALLBACK ▷ no staging owner, so a discarded attempt is not attestable
end if
if $q_A(z) > \eta_A$ **then**
  return FALLBACK ▷ calibrated abstention on an uncertain input
end if
if $m = \text{shadow}$ **then**
  log would-dispatch; **return** FALLBACK ▷ observe coverage without changing behaviour
end if
$S \leftarrow \text{STAGE.BEGIN}(); \quad r \leftarrow \text{INTERPRET}(A.P, z, \text{FACADE}(C))$ ▷ the facade re-checks effects at execution time
if r raised PRECOMMITERROR **then**
  return FALLBACK if $S.\text{ABORTANDATTEST}()$, else INCIDENT
else if r raised POSTCOMMITERROR **then**
  return INCIDENT ▷ an external commitment happened; do not feign rollback
end if
if $V_A(r) \neq 1$ **then**
  return FALLBACK if $S.\text{ABORTANDATTEST}()$, else INCIDENT ▷ live-outs, effect multiset, and call count are checked before use
end if
return r.observations if $S.\text{COMMIT}(r.\text{effects})$ succeeds, else INCIDENT

a verifier failure after that point delegates to the wrapped model with history the baseline would never have contained. Every check that can reject *before* emission — lifecycle, signature, manifest pins, hard guard, already-observed tools, quota attestation, and the calibrated gate — is therefore exact for both paths, and only post-emission verifier or binding failures are weaker for the adapter. Claims of universal clean fallback in this paper should be read as scoped to the staging-owning runner; the adapter is guarded substitution with explicitly weaker post-emission semantics. A general solution requires artifacts to target explicit control points with continuation tokens rather than implicit model boundaries, which we leave to future work (section 8.3).

The operational invariant of the whole runtime fits in one sentence: one path compacts, clean misses preserve the baseline at the stated boundary, and post-commit contamination is reported as an incident rather than relabelled as fallback.

4 EXPERIMENTAL METHODOLOGY

4.1 Evidence tiers and hypotheses

We use three non-overlapping evidence tiers, ordered by what each can license.

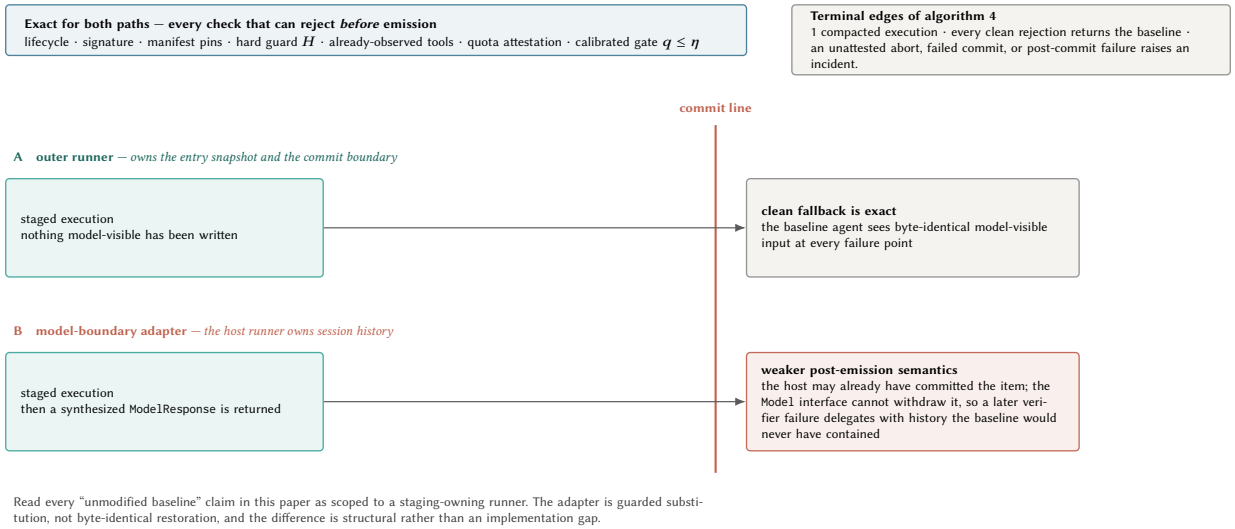


Figure 13: Where "the baseline runs unchanged" is exact. Both integration paths share every check that can reject *before* emission, so those rejections are exact for both. They differ only to the right of the commit line: the outer runner owns the entry snapshot and can restore byte-identical model-visible input, while the model-boundary adapter cannot withdraw a `ModelResponse` the host may already have committed to session history. The difference is structural rather than an implementation gap, and it is why one terminal edge of algorithm 4 raises an incident instead of reporting a fallback.

Tier 1 — external trace-compiler evidence. A sealed manifest pins NESTFUL and API-Bank, the two audited public corpora that retain the observed intermediate values needed to reconstruct and replay post-trace programs. They test provenance, synthesis, and admission rather than end-to-end agent quality. Eight additional benchmark adapters remain in the artifact's supplementary interoperability ledger, but not in the main evaluation: their checkers, task descriptions, simulated environments, hosted services, or missing outputs do not support a compiler-compatible head-to-head result.

Tier 2 — real records, live provider. Three primary GitHub workflow families use actual public records, deterministic local reads over one pinned snapshot, and live OpenAI provider calls. Each family has a distinct three-tool vocabulary and exact three-class output contract: issue type, pull-request outcome, or backlog attention. Every family uses 132 balanced discovery records and 30 disjoint balanced held-out records. The unchanged, compiled, and hand-written conditions receive the same records, model, factual contract, and condition placement. The latter two families compile a verified three-read pre-model composite; the original issue family retains its conservative two-read prefix so its published negative comparison remains intact. These experiments establish transfer across workflow families on a real snapshot, not across repositories, time, or live GitHub service behavior.

Earlier fixed-prefix, depth, portfolio, GCS, and bounded-GEPA studies remain as ablations and negative controls. They preserve the historical denominator, but they are not counted as additional workflow families or pooled into the primary 90-record result.

Tier 3 — controlled stress suite. Six workflow *shapes* are run end to end through the real SDK runtime with live provider calls against deterministic local services. The business records here are fictional, so this tier cannot support a claim about real-world data; what it can do is cover control-flow structures the other two tiers do not contain at all — observation-dependent branches, pagination, a mandatory irreversible write, a handoff barrier, an undeclared-effect tool surface, and route specialization — and expose the behaviour of the runtime when it must *refuse*. We report it because a suite in which every case succeeds would be evidence of a weak test set, not a strong compiler. Three of its eight conditions are negative controls whose only correct outcome is no compaction.

4.2 Benchmark at a glance: tasks, traces, and contracts

The benchmark is intentionally not a single accuracy leaderboard. It asks whether a compiler can reconstruct a safe deterministic prefix from retained traces and, if so, whether replacing the corresponding model turns preserves a task contract. Table 4 maps concrete inputs, traces, and contracts while separating live source-grounded workflow studies from post-trace compiler substrates.

4.3 Prospective public-record extension: HMDA

The extension harness adds a privacy-modified public Home Mortgage Disclosure Act (HMDA) loan-application-record (LAR) substrate to the same typed trace and effect contracts. Each case binds a year, filer identifier, row digest, non-sensitive field families, year-specific schema and code definitions, and source identifiers. Protected demographic attributes are excluded from the task and no output is interpreted as a lending, fairness, compliance, or legal decision. The primary oracle requires exact agreement with the frozen public row and its year-specific definitions; "NA", exempt,

Table 4: Benchmark at a glance. "Trace" is what the compiler observes after an agent execution; "contract" is the outcome checked on a held-out input. GitHub inputs are real public records from one pinned snapshot. The four external substrates use retained or obtained executable traces with no model in the loop, and therefore cannot support an end-to-end agent-quality claim; NESTFUL and API-Bank retain their results upstream, while BFCL and AppWorld obtain them by executing official gold artifacts on their own pinned backends.

Benchmark	Representative input → required output	Observed recurrent trace	What the result licenses
Issue-type routing	Issue number → exact identity, label-derived class (bug/enhancement/question), and a verbatim source excerpt	`record(issue_number) -> labels(record.issue_number) -> comments(record.issue_number, limit)`; the last argument varies	A live comparison of unchanged agent, partial compiler, and manual bundle. The compiler may emit only `record -> labels`; the model retains the comments decision.
PR-outcome audit	Pull-request number → exact title, state, outcome (open/merged/closed_unmerged), and discussion evidence	`pr_get_record -> pr_get_merge_status -> pr_get_discussion`	A three-read pre-model composite tested on a disjoint, balanced 30-record cohort.
Backlog-attention routing	Open issue → route (owned/discussed_-unowned/awaiting_first_response), owner, and comment excerpt	`backlog_get_record -> backlog_get_ownership -> backlog_get_discussion`	A distinct tool vocabulary and contract, preventing transfer from being a renamed issue-type task.
Time-forward PR extension	Pull request from a later snapshot period → exact number, title, and three-way outcome	`record -> merge_status` on each frozen repository	Cross-repository, time-forward evidence on a deliberately narrower two-read task; a repository without an admitted artifact remains a recorded retirement.
NESTFUL	Nested API episode: an earlier result supplies an argument to a later call	Typed producer references across executable multi-call sequences	Provenance, synthesis, held-out replay, and support-gate evidence—not LLM planning accuracy.
API-Bank	Description-led runnable API dialogue with one to five calls	Recorded cross-call values over 49 APIs, including write-bearing tools	Effect barriers and support-gate refusal: write-bearing windows are blocked, and remaining recurrent families lack sufficient support.
BFCL v4 multi-turn	Official gold plan executed on the pinned stateful backend → observed results for every reference call	1,142 calls over 81 methods and eight scenario classes, with a byte-exact provider-free replay oracle	A pre-registered third refusal on a corpus with a different tool surface and a more generous entry contract; not planning accuracy.
AppWorld	Official gold solution executed on the pinned resettable backend → observed results, plus the task's own entry snapshot	7,070 calls over 68 typed APIs in 136 tasks, byte-exactly reproducible	The first external substrate whose group count makes (8) satisfiable: one admitted artifact, four marginal retirements, and a held-out dispatch-eligibility measurement.

and unavailable states remain explicit values rather than imputed facts.

The provider-free validator reconstructs an independently generated gold record for all 420 HMDA groups (420/420 exact-gold passes). Of these, 416 groups (99.05%) contain a variable path suitable for a future trace study; the remaining four are valid fixed paths, not failures. The same preflight retains 420/420 vulnerability groups for context, while the SEC pool is unavailable because its compliant source contact is not configured. No OpenAI call, baseline-vs-GRC comparison, macro approval, token result, or latency result is licensed by this checkpoint.

The directional hypotheses below were fixed internally before the sealed test was scored, but the study was not externally preregistered: (H1) provenance places the expected producer in the candidate set for more than 90% of executable nested dependency slots — a recall statement, reported alongside unique-resolution and precision rates so it cannot be read as exact reconstruction; (H2) the compiled condition reduces provider requests and total tokens; (H3) no observed quality loss occurs on the sealed task; (H4) families below the exact-gate sample requirement retire; and (H5) a condition that must refuse reproduces the baseline model-call count and quality. H5 is deliberately *not* stated in dollars: section 5.5 shows refusing conditions costing more than baseline, for prompt-cache reasons unrelated to whether the refusal was correct. H3 is an observed-sample hypothesis, not a population equivalence claim. The portfolio pilot was designed after the replication was known but before its fresh cohort was selected or executed.

Table 5: Provider-free public-record extension preflight. Exact gold is an independently implemented reconstruction, not a model-quality score. Variable paths identify cases that could support a future trace comparison.

Domain	Groups	Exact gold	Variable paths	Disposition
Vulnerability	420	420/420	48/420	Provider-free preflight
HMDA	420	420/420	416/420	Provider-free preflight; privacy-modified public LAR
SEC filing facts	0	–	–	Source-gated; no provider run

Its prospective hypothesis (H6) is therefore narrower: the frozen selector's chosen action will pass all observed fresh task contracts and reduce its weighted resource objective relative to baseline. H6 was not externally preregistered and cannot establish that selection beats an always-macro policy. GCS was designed after both macro results were known. Its 12-pair study is therefore an explicitly post-study, exploratory test rather than an additional preregistered hypothesis. The issue selection is provider-outcome-free and disjoint from all 424 earlier issue IDs, but it was recorded by

the same run that executed the provider calls rather than sealed in an externally timestamped protocol. The learned/manual comparator was likewise designed after the GCS result and is explicitly exploratory. Its test identifiers were frozen before prompt optimization, but the study was not externally preregistered and its six cases are not a powered non-inferiority test.

4.4 External benchmark: NESTFUL

NESTFUL contains more than 1,800 executable nested API sequences and was designed to test whether later calls correctly reuse earlier outputs [6]. We pin upstream commit `fc2c4123e735` (the full identifier is in the source manifest) and verify SHA-256 digests before use. Of 1,861 records, 1,416 use the audited `basic_functions` module; 1,415 execute successfully. One upstream record raises a string/integer type error and remains in the failure log.

We convert each successful execution into the same typed Episode IR, reconstruct expected producer references, mine families, split within families by group, synthesize from train/dev windows, and replay on held-out windows. This experiment does not call an LLM and does not measure NESTFUL planning accuracy. It tests the compiler after a valid trace exists. The exact gate is then applied to observed family support.

4.5 External benchmarks: BFCL v4 and AppWorld

Two further public corpora ship an executable backend but no recorded tool results, so both are converted the same way: execute the official gold artifact on the pinned backend, retain each call's observed result, and let the task's own snapshot supply the entry state. Neither runs a model, so neither measures function-calling accuracy.

BFCL v4's `multi_turn_base` tasks each publish an `initial_config` snapshot and the API classes they involve. Executing the 200 official gold plans through the upstream helper yields 1,142 calls over 81 methods. Five preconditions fail the run closed rather than publishing a number: an undeclared method, any upstream execution-error sentinel, a read-like declaration observed mutating state, a compilable declaration observed advancing the seeded scenario RNG, and any inexact re-execution. All held. The predeclared outcome, decision rule, and claim boundary were committed before the compiler saw the corpus.

AppWorld supplies nine simulated apps, 457 typed APIs, 100 fictional people, and a resettable per-task database [44]. Its public `minimal` data mode carries gold solution programs for the `train` and `dev` splits only, so those 147 tasks are the substrate; `test_normal` and `test_challenge` withhold ground truth and cannot serve a post-trace question. Calls are captured at the single dispatch point every `apis.<app>.<api>(...)` invocation passes through, so the recorded identity is a semantic app/API pair with typed keyword arguments and a decoded JSON result rather than a URL and a form body.

AppWorld is included for one reason, stated before the run. The three earlier substrates reached largest family supports of 26, 8, and 15 against the 92 zero-violation groups (8) requires, so each retirement was settled by corpus size before any compilation happened, and a reader is entitled to read all three as foregone conclusions rather than as tests of the gate. AppWorld is the first external corpus where the requirement is reachable, and therefore the first where the gate can be wrong. The pre-registration accordingly records the paper's first non-null external prediction.

It also has exactly one contestable effect declaration, and that declaration decides how deep a region can go, so both readings are declared in advance. Each app's `login` is a `POST` that mints a bearer credential; it was observed to leave the database unchanged on all 184 of its calls and its token is byte-identical under re-execution, because AppWorld freezes each task's clock. *Arm A* declares it `WRITE_REVERSIBLE`, treating credential minting as a state claim the fixture need not record. *Arm B* declares it `READ_EXTERNAL` with `speculatable` and `replayable`, licensed by the audit. Exactly those five entries change. Arm A is primary because it is stricter; Arm B is reported because Arm A's winning family is argument-free and therefore does not exercise provenance at all, while Arm B's third call does.

Groups are tasks rather than scenarios, which is the weaker of the two available choices and is declared as such: AppWorld's three variants of a scenario share a supervisor and a database, so their violation indicators are not independent, while scenario-level grouping gives 49 groups and could not reach 92 under any outcome. The bound is therefore conditional on task-level independence, the same conditional section 9 already records for the live studies; this substrate inherits it rather than repairing it. The calibration share is pinned at exactly the 92 groups the bound needs, drawn by seeded shuffle over all clean tasks, because a larger set would make the bound easier and a smaller one would make it unreachable.

Finally, admission is compiled from gold solutions, which are not agent trajectories: they contain no planning turns, no documentation lookups, and no recovery steps. AppWorld also releases the official baseline agents' experiment outputs — 28 runs over four agent architectures and four models — each retaining the ordered API calls the agent actually issued. Those released runs are used for one measurement only: whether the admitted region is *structurally eligible* at the entry boundary, meaning its exact call sequence occurs at normalized position 0. That is an upper bound on the dispatch rate ϕ and not ϕ itself, because the manifest check and the calibrated gate are not evaluated against runs that never used the artifact's pinned manifest; and it cannot bound savings at all, because the released logs retain API calls rather than model boundaries.

4.6 Three real-record workflow families

The primary transfer study reuses one revision-pinned 7,540-record public GitHub snapshot while changing the decision, tools, and exact grader. Pull requests are classified from record, merge-status, and discussion reads. Open non-PR issues are classified from record, ownership, and discussion reads. These tools are deliberately separate from the issue-type routing tools, preventing an apparent transfer result obtained by renaming one interface. Selection is provider-outcome-free, balanced by class, deduplicated by record identifier, and disjoint across discovery and test cohorts.

An initial pull-request pilot exposed two defects and is archived rather than overwritten: nullable merge timestamps were projected unsafely, and empirical numeric hulls treated opaque identifiers as quantities, rejecting unseen but schema-valid values outside the training extrema. The final compiler retains type, provenance, effect, and risk checks but uses an unconstrained value hull for fields named as identifiers, keys, or numbers; high- cardinality free text likewise avoids an empirical literal regex. After this change, the pull-request final uses the original 132 discovery traces and a freshly sealed 30-record test cohort. The backlog cohort was frozen after the same implementation was fixed. Compiler rejection triggers the unchanged agent, so a failed guard cannot become a missing answer. Checkpoints are written before paid evaluation and after every condition.

4.7 Cross-repository, time-forward PR-outcome-core extension

To test whether the guarded pre-model runtime survives beyond one retained repository, we add a narrower cross-repository study over five frozen public GitHub snapshot sources from Hugging Face mirrors: `huggingface/datasets`, `pandas-dev/pandas`, `psf/requests`, `streamlit/streamlit`, and `pytorch/pytorch`. The task asks for the exact pull-request record number and title, then classifies one of three exact outcomes: `open`, `merged`, or `closed_unmerged`. It uses two read-only local tools: record and merge-status. This removes comment-availability drift and keeps the exact grader repository-independent, but it is intentionally narrower than the three-tool workflow-family tasks.

Each repository first passes a provider-free preflight that deduplicates records, excludes earlier paper cohorts, seals 116 older discovery pull requests, and selects a balanced 30-record held-out cohort whose timestamps are strictly newer. The complete preflight supports 150 held-out pairs across the five repositories. Discovery then runs with the same provider, SDK, and model configuration as the earlier GitHub studies. Compilation freezes one candidate before calibration — so any admitted artifact here carries corollary 2's compiler-wide guarantee, not only the per-candidate one — uses 16 train, 8 development, and 92 calibration traces, and compares unchanged, compiled, and a fixed two-read template pre-model baseline on held-out records. Repositories with no admitted artifact are retained as fail-closed negatives rather than dropped.

Because that first five-repository cohort is class-skewed on some mirrors, we also run a balanced rerun on the three repositories with enough older records to satisfy the exact gate without changing the task: `pandas-dev/pandas`, `psf/requests`, and `pytorch/pytorch`. A provider-free round-robin selector seals 120 discovery pull requests and a disjoint 60-record held-out cohort per repository, each evenly split across `open`, `merged`, and `closed_unmerged` under the same strict time-forward rule. The model, frozen-candidate protocol, compiler, and three-arm held-out evaluation are unchanged. We treat this rerun as a protocol-sensitivity study: if it recovers open-state compaction or admits the repository that retired earlier, the earlier negative must be read as a cohort artifact rather than as an intrinsic limit of the two-read guarded runtime.

4.8 Controlled fixed-prefix live study

We pin revision `e344be7b84d1` (full identifier in the source manifest) of the Apache-2.0 `helmo/github-issues` dataset [20], verify the 12.7-MB Parquet digest, exclude pull requests, require at least 80 body characters, and deduplicate issue numbers. The scenario asks an agent to classify a `huggingface/datasets` issue as bug, enhancement, or question and return a short evidence-grounded summary. Three local function tools read the exact issue record, labels, and first three comments from the frozen public snapshot. The records and workload are real; the local tool service is deterministic so that API and website drift cannot contaminate paired comparisons.

Every model turn uses the OpenAI Agents SDK 0.19.2, OpenAI Python 2.52.0, and `gpt-5.6-luna` at low reasoning effort. The script loaded `OPENAI_API_KEY` for provider calls and `HF_TOKEN` for dataset retrieval; it serialized only boolean usage flags, never credentials. Cost is an estimate from the model-price table frozen with the experiment and the official pricing source [38].

The discovery cohort contains 132 issues, and all 132 satisfy the exact tool contract, so all 132 are compiler-eligible. The split consumes the first 116 under a fixed rule — train (16), development (8), calibration (92), sizes chosen before any outcome was scored, because $n = 92$ is fixed a priori by $(\alpha, \delta, |\Lambda|)$ — and leaves the remaining 16 unused. Those 16 are surplus to the calibration requirement, not filtered: they are neither excluded for a quality reason nor drawn upon, and we state the residue rather than report a denominator of 116 as though eligibility had produced it. A stronger design would either consume them or size the cohort to the split. The compiler, unmodified for the task, emits the prefix

$$\texttt{record}(z.\texttt{issue_number}) \downarrow \texttt{labels}(\mathit{record}.\texttt{issue_number}) \downarrow \texttt{comments}(\mathit{record}.\texttt{issue_number}, 3)$$

It blocks all 116 suffix candidates under the position invariant. The 18-item sealed test contains six issues per category, is disjoint from discovery, and excludes all 162 issues touched by the earlier pilot. Baseline and compiled conditions run the same task and tools. Six cases are repeated once for determinism analysis.

The baseline requires four provider responses around three tool calls. The compiled condition executes the same necessary tools natively and asks the model once to produce the final structured response. Thus the intervention tests decision elision, not tool elision or parallel scheduling.

4.9 Natural-order, source-grounded live studies

The first follow-up keeps the pinned dataset, model, SDK, and three read tools but changes the task contract. Its prompt requests exact issue number, title, state, label-derived category, evidence label, and a verbatim excerpt from one of the first three comments; it contains no tool function name and no execution order. The grader checks the structured facts against the snapshot and normalized excerpt containment against the returned source comments. Tool order is reported separately and contributes nothing to task quality. Executable regressions verify that reordering calls leaves the score unchanged and that a fabricated comment excerpt fails.

We exclude all 312 records touched by the fixed-prefix final run or pilot. A stable hash selects 80 fresh discovery issues and 18 disjoint test issues (5 bug, 2 enhancement, 11 other; no class quota). All 80 discovery agents independently choose `record→labels→comments`, and all pass the corrected source-grounded contract. The executed artifact was built under the online oracle, which falsely rejected one short but valid excerpt: 79 records were then eligible, a stable hash assigned 20 train, 10 development, and 45 calibration records, and five were unused. Under the corrected oracle, issue 1741 would rank sixth and replace issue 3511 in that split. We neither retrain the executed artifact post hoc nor assume the two artifacts are equivalent; the result manifest records the consequence. No filter uses tool order. The review-driven protocol was checked into executable tests before provider execution but was not externally preregistered.

The paired test adds a hand-written `issue_get_bundle` tool that returns the same record, labels, and three comments in one read. Each issue runs the unchanged agent, learned compiler, and macro exactly once. The six permutations of condition order are cycled three times across the 18 records, so every condition occupies every ordinal position equally often. All conditions use the same model and factual output schema. The full run made actual provider calls and records an estimated total cost of $0.0920; credentials are represented only by boolean usage flags.

The expanded replication uses the same provider and snapshot but a stricter typed output: exact issue number, title, state, total comment count, label-derived category and evidence label, plus a 20–240 character verbatim excerpt from the returned title, body, or comment payload. A provider-free preflight sealed 132 discovery records and 30 held-out records, balanced as ten bugs, ten enhancements, and ten questions. The paid run used those exact records and cycles all six baseline/compiler/macro orders five times. Ten test records are repeated once per condition for determinism.

Of the 132 discovery outputs, 130 pass the exact-source task and are eligible under the executed safety rule. All 132 read the record, then labels, then comments; however, the natural caller chooses different integer comment limits. Because no trace-grounded expression can reproduce that argument, the three-read candidate fails with `ungroundable_slot`. The compiler correctly falls back to the longest groundable prefix, `record→labels`, and the ordinary agent performs the comments read and final rendering. Sixteen training, eight development, and 92 calibration records are selected from the 130 eligible traces; the other 14 are unused. The complete run retains 252 agent outputs, 848 provider responses, no infrastructure failure, and an estimated total cost of $0.1913. An online literal-argument grader was corrected provider-free: all original outputs and measurements remain bound to the paid discovery checkpoint, while 212 prior quality objects are preserved under `online_quality`.

4.10 Prospective portfolio pilot

The portfolio calibration consumes only the 30 independent primary issue groups from the expanded replication. Compiler and macro observations are paired with the unchanged arm; ten repeated executions are averaged within their issue and do not increase support. We fix $U_{\min} = 0$, $n_{\min} = 30$, $\alpha_q = \alpha_u = .15$, and 95% familywise confidence split over two actions and two exact bounds. This 15% pilot limit is intentionally looser than the compiler's registered 5% contract bound and must not be conflated with it; nor should it be conflated with the 10% level at which the GCS artifact was calibrated (table 3). Three risk levels appear in this paper and each licenses a different set of results.

Before any prospective call, the script stores the decision and hashes the calibration file. A stable hash then selects 12 previously unused public issues: four bugs, four enhancements, and four "other" records. All eligible question records had already been consumed by earlier sealed protocols, so reusing them would have violated the prospective boundary. Each issue runs baseline and the selected reviewed action once; order is counterbalanced six each way. The non-selected action is not executed on this cohort. The provider-backed script requires an explicit macro-review flag, records 24 unique native traces, and serializes no credential. This evaluates a pre-data action choice on a fresh cohort, but only within one workflow family and one model configuration.

4.11 Bounded learned and pre-model comparator study

We first reconstruct both GCS and the manual program on all 12 selected split records without a provider call. Their projected evidence must match byte-for-byte before the paid study may proceed. Four records train prompt proposals, two evaluate them, and six remain sealed for held-out evaluation; all are disjoint from 443 issue identifiers used by earlier experiments. Strict exclusion leaves bug, enhancement, and other records but no question record, so the held-out cohort balances only the three available categories (two each).

Official GEPA 0.1.4 may make at most 16 task-metric calls and three proposals. Each task evaluation is a real Agents SDK execution, and each reflection is a real Responses API call. The score

$$1000I_{\text{exact}} + 100q_{\text{facts}} - 5N_{\text{req}} - N_{\text{tool}} - N_{\text{tok}}/10{,}000$$

makes exact factuality dominant over the available efficiency terms. After optimization, all five conditions run once per held-out issue under a balanced condition-order schedule. Because GEPA retains its seed, the nominal GCS+GEPA arm is an order-balanced replication of GCS, not evidence of a distinct combined optimizer.

4.12 Controlled stress suite: workflow shapes

Six SDK-backed stress scenarios cover a linear prefix, permission-scoped retrieval, a handoff barrier, undeclared MCP effects, an observation-dependent branch ending in an irreversible write, and route specialization. Three negative controls require exact fallback. Their purpose is runtime boundary coverage, not domain evidence; complete scenario definitions and measurements are retained in the repository.

4.13 Metrics and statistical analysis

The primary efficiency outcome is provider requests. Secondary outcomes are input, output, and total tokens; provider and wall latency; estimated cost; and tool calls. In the fixed-prefix ablation, quality requires the correct category, evidence label, issue number, valid summary shape, and exact tool contract. In the natural-order studies, quality instead requires exact source-grounded fields and label decisions and is independent of tool order. The expanded

task contract additionally requires one safe, grounded call to each necessary read tool while allowing any order and any integer comment-body limit. Paired differences use the same issues. We report 10,000-sample paired bootstrap confidence intervals [14] and two-sided Wilcoxon signed-rank tests [50]; binary quality uses exact McNemar tests [34]. Signed-rank p-values use the exact distribution where no ties occur and a tie-corrected normal approximation otherwise. One row is deliberately *not* given a p-value: every pair changes provider requests by exactly -3, so the difference is deterministic and a rank test on it reports an artifact of degeneracy rather than evidence. Secondary p-values are descriptive and unadjusted. Determinism is reported as decision, tool-trace, and byte-exact answer agreement. Peak memory and runtime are measured for the provider-free NESTFUL compiler stage. For portfolio admission, quality failures and non-positive group utilities are separate binary endpoints. Their one-sided exact bounds split 5% error across two actions and two endpoints; mean utility ranks only actions that pass both gates. The 12-record prospective cohort is an outcome evaluation, not additional calibration evidence. Its paired bootstrap intervals and signed-rank tests are descriptive and unadjusted. The exploratory GCS comparison uses the same paired procedures. Its primary contrast is GCS minus the measured provider-visible macro; it reports exposed tool interfaces and internal source reads separately so interface fusion cannot masquerade as eliminated work. The bounded comparator uses the same exact quality and paired resource procedures. Host monotonic wall time is primary. Provider-span latency is retained but excluded from a comparison if the summed spans exceed the surrounding host wall time by more than 250 ms. Optimization requests, tokens, latency, and cost are never mixed with held-out evaluation metrics.

5 RESULTS

The evidence separates three questions. First, can a trace be reconstructed and synthesized? Second, is there enough independent evidence to admit the result? Third, if admitted, does it improve the end-to-end workflow against a strong manual alternative? NESTFUL, API-Bank, and executed BFCL answer the second question with refusal, while AppWorld supplies the first reachable external admission and then exposes a distinct dispatchability constraint (sections 5.1 and 5.2). The three GitHub workflow families answer the third within one repository, and the later cross-repository extension tests the same question on a simpler time-forward task (sections 5.3, 5.3.1 and 5.7). The remaining sections retain the earlier depth failure, failed suffix pilot, harder workflow shapes, and prospective portfolio study because each constrains a different claim rather than adding another headline.

Table 6 adjudicates every hypothesis before the detail, including the two that split and the two questions that were asked after the results were known and can therefore only be exploratory. Two of the six directional hypotheses are supported on one reading and not on another; recording the split is the point, because either half alone would misdescribe the result.

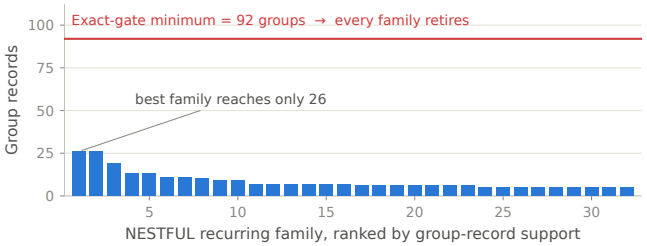


Figure 14: All NESTFUL families fall below the 92-group exact-gate requirement; the GitHub study was deliberately sized to reach it.

5.1 RQ1 and RQ3: provenance succeeds more often than certification

Across 1,415 executable episodes, GAC places the expected producer in the candidate set for 5,531 of 5,746 dependency slots (96.3%). That figure is *candidate recall*, not dependency reconstruction, and it is close to definitional: candidates are generated by value matching and the gold producer emitted the matched value, so it joins the candidate set whenever any admissible path exists at all. Consistently, there is not one slot in which a non-empty candidate set omitted the true producer — the 215 misses are exactly the 215 slots with no candidate. Recall here therefore measures search *reachability* and is insensitive to ranking quality; it cannot fall below the groundability rate however poor the ranking. The load-bearing numbers are the other two: only 4,636 slots (80.7%) are resolved to a unique producer, 895 (15.6%) contain the expected producer among several candidates, and 215 have no candidate at all. Counting all emitted candidate edges gives 84.3% precision (5,531 of 6,564). H1 was pre-specified internally as recovery above 90%; it is supported on recall and *not* supported on unique resolution, and we report both rather than the more favourable one. It finds complete groundable windows in 1,207 episodes (85.3%). The remaining full-window failures are overwhelmingly ambiguous value matches (207), with one ungrounded slot. That count is per *episode* under first-hit attribution, and it is not the same denominator as the 215 no-candidate *slots* in table 7: an episode is attributed to the first reason that blocks it, so an episode containing both an ambiguous slot and an ungrounded one is counted as ambiguous, and slots outside every enumerated window are counted in neither. The two numbers are therefore consistent but not comparable, and we report both rather than the one that reads better. Recurrence is fragmented: 714 compiler families exist, only 32 have support at least five, and maximum support is 26.

Of 32 attempted recurrent families, 12 synthesize. Their 36 held-out windows yield 24 passes, 12 abstentions, and zero wrong executions. The dominant synthesis failures are unsupported loop predicates and slots that fail grouped refitting. This zero-wrong result does not certify the programs: the per-candidate exact gate requires 92 zero-violation groups, so all families retire. H4 is supported, and the negative result distinguishes a guarded compiler from a recurrence-only macro miner.

Two properties of this benchmark path bound how far it generalizes. Its per-family split is lexicographic by episode identifier

Table 6: Every hypothesis and its outcome. "Pre-specified" means fixed internally before the sealed test was scored; no part of this study was externally preregistered. The exploratory comparator and determinism question were formulated after the results they examine were known, so they carry no hypothesis and license no confirmatory claim.

	Question	Pre-specified	Outcome	Reported in
H1	RQ1	yes	Split: supported on recall (96.3% of dependency slots), not supported on unique resolution (80.7%) against a 90% target	section 5.1
H2	RQ2, RQ6	yes	Supported: requests −50.0 to −75.0% and total tokens −39.5 to −81.4% across three families	section 5.3
H3	RQ2	yes	Split: supported for the expanded two-read artifact (30/30 exact), not supported for the earlier three-read artifact (17/18)	section 5.4
H4	RQ3	yes	Supported: every synthesized family retires — 12 on NESTFUL and both on API-Bank — for insufficient calibration support	section 5.1
H5	RQ4	yes	Supported on model calls and quality for all three negative controls; deliberately not stated in cost, which rises up to 55.4%	section 5.5
H6	RQ5	before the fresh cohort	Supported on 12 fresh pairs (12/12 exact, requests −50.0%); cannot establish that selection beats an always-macro policy	section 5.10
—	RQ2 comparator	no (exploratory)	GCS beats the measured provider-visible macro on one family, and reaches parity with a fairly placed manual program	sections 5.7 and 5.8
—	RQ4	no (post hoc)	Not supported: byte-exact answer agreement is 1/6 for the baseline and 0/6 compiled	section 5.11

Table 7: External NESTFUL compiler results. "Wrong" means a synthesized program executed but disagreed with the recorded held-out outputs; abstentions are explicit.

Measurement	Result
Executable basic-function episodes	1415
Expected producer in candidate set (recall)	5531/5746 (96.3%)
of which uniquely resolved	4636 (80.7%)
of which ambiguous (truth among many)	895 (15.6%)
Slots with no candidate	215
Candidate-edge precision	5531/6564 (84.3%)
Complete groundable windows	1207/1415 (85.3%)
Recurring families with support ≥ 5	32
Synthesized families	12/32
Held-out replay: pass / abstain / wrong	24 / 12 / 0
Maximum family support / gate minimum	26 / 92
Certifiable families	0
Peak memory / elapsed time	23.3 MiB / 5.98 s

rather than randomized or chronological, so it tests held-out *windows* but not robustness to a shifted distribution; and the development partition it allocates is reported but not consumed, because synthesis reads the training windows and replay reads the test windows directly. This path also builds provenance, mining, synthesis, and replay itself rather than invoking the full compiler, so it is an executable structural benchmark, not end-to-end validation of the deployed gate.

The provider-free NESTFUL analysis completes in about six seconds, including roughly 1.4 for provenance construction, with 23.3 MiB peak traced memory. These numbers characterize this pinned dataset and Python process, not distributed production throughput.

5.2 The reachable case: one external admission, and four marginal refusals

BFCL executed exactly as pre-registered. All 200 gold plans and 1,142 calls ran with no upstream error sentinel, an independent pass reproduced 1,142/1,142 results byte for byte, declared write barriers suppressed 2,802 candidate spans, and 146 candidate windows in 86 tasks formed 77 families. Nine reached the support floor, four synthesized, and held-out replay returned 3 passes, 3 abstentions, and zero wrong executions. The gate retired every family at $n_{\max} = 15$ against 92. Its empirical effect audit also produced the finding that justifies signing catalogs by hand: over 1,060 observed mutating calls, no read-like declaration was ever observed mutating and no compilable declaration ever advanced the RNG, but `TravelAPI.get_flight_cost` — a nominal getter by name and docstring — mutated its instance's internal lookup on 36 of 36 calls. A name- or docstring-derived effect label would have admitted a write into a compiled region.

AppWorld is the case the other three could not test. Of 147 public gold solutions, 136 execute cleanly; 11 `dev` solutions raise against the shipped database version and are reported as upstream compatibility outcomes rather than dropped, in the same way as API-Bank's partial re-execution. The independent second pass reproduced 7,070/7,070 observed results byte for byte, minted access tokens included. The corpus spans 46 scenarios and 79 distinct fictional supervisors, with 5/36/244 calls per task at minimum, median, and maximum.

The empirical effect audit is cleaner here than on BFCL. Of 7,070 calls, 1,320 mutate the application database; no read-like declaration was observed mutating; and — unlike BFCL — no API mutates on some calls but not others. Every `GET` was clean on every call and every `DELETE` and `PATCH` mutated on every call, so the effect boundary is crisp. The five `*.login` endpoints are the only non-`GET` APIs that never mutate, which is precisely why they are the pre-registered contestable declaration.

Table 8: The four external compiler substrates at the stages of algorithm 1. Every substrate clears provenance, synthesis, and held-out replay with zero wrong executions. Three retire because $n_{\max}$ falls short of the 92 zero-violation groups (8) requires — a shortfall settled by corpus size before the compiler ran. AppWorld reaches 136 and admits.

Substrate	Traces	Slots recovered	Episodes with a window	Families at support floor	Families synthesized	Held-out replay pass / abst. / **wrong**	$n_{\max}$	Artifacts admitted
NESTFUL	1,415	96.3%	1,207	32	12	24 / 12 / **0**	26	**0**
API-Bank	212	63.5%	37	8	2	0 / 2 / **0**	8	**0**
BFCL v4	200	68.0%	86	9	4	3 / 3 / **0**	15	**0**
AppWorld	136	63.2%	136	33	1	34 / 0 / **0**	**136**	**1**

Table 8 shows the consequence. AppWorld reaches $n_{\max} = 136$, so (8) is satisfiable, and the deployable pipeline admits one artifact in *both* arms — the same artifact in each: the two-step program `supervisor.show_profile` → `supervisor.show_account_passwords`, at 92 of 92 calibration groups accepted, zero violations, $\hat{\eta} = 0.50$, and $U_{\hat{\eta}} = 0.0498 \leq \alpha = 0.05$, with 11/11 exact held-out replay. This is the first admitted artifact on any external corpus in this paper.

Three properties bound that admission, and all three cut against it. First, the gate is degenerate: coverage moves from 0 at $\eta = .05$ to 1.00 at $\eta = .08$ and stays there, so it admits every calibration group as soon as it admits any. AppWorld therefore *reproduces* the gate non-discrimination reported in section 5.12 on a public corpus rather than repairing it. Second, the admitted program is argument-free: both calls take no arguments, so the admitted artifact does not exercise provenance, and what provenance decided was its *length*. Third, no saving is shown, because no model ran.

Where provenance and the barriers do bind is the permissive arm's refusals (table 9). Six candidates reached the pipeline: one was admitted, one was dominated by a longer region, and four retired at calibration. None retired for lack of support. Two retired on 89 and 90 eligible calibration groups against 92 — short by three groups and two groups, and by 0.001 of bound. The other two are app-specific, so only the calibration tasks touching that app are eligible, and 46 and 26 groups cannot reach a 5% bound at any threshold on the frozen grid. The admitted candidate is the same mechanism a third time: its mined window is three calls including `phone.login`, but its synthesized program is two steps, because the login's password slot is not groundable in every supporting group.

5.2.1 Admissible is not dispatchable. The admitted artifact was compiled from gold solutions. Measured instead against the 28 released official baseline runs, it is structurally eligible on 3,102 of 8,190 held-out trajectories (37.9%) — and that pooled number averages over a sharply bimodal population (table 10). Full-code agents present the region on 2,339 of 2,340 trajectories and iterative parallel function-calling agents on 762 of 1,170, while ReAct and plan-and-execute agents present it on 1 of 4,680. The mechanism is legible in the first call: 3,752 of the released trajectories begin with an API-description lookup, because those architectures read documentation before acting, and the compiled prefix is then no longer at position 0, so (2) refuses. A further 61 trajectories contain the program somewhere other than position 0, which the same invariant also refuses.

Two secondary observations sharpen the reading. The rate is flat across task difficulty (0.377 on `test_normal` against 0.380 on `test_challenge`), so this tracks the agent's calling discipline rather than the workload. And the maximal compilable depth on the eligible families is 2, so the admitted two-step program sits at the ceiling the effect catalog permits rather than short of it; on the ineligible families the measured depth is a floor rather than an estimate, because the API-description call is undeclared in the signed catalog and unknown effects are not reads.

The consequence is a deployment precondition that no admission condition in section 3 expresses. A miss costs a forgone saving rather than an error, because the guard abstains and the unchanged agent runs. But a compiled prefix is only worth compiling for an architecture that actually issues one, and nothing in proposition 1, the hard guard, or the calibrated gate tests that. It also explains the paper's own live results rather than sitting beside them: the GitHub agents share the fixed-prefix discipline of the full-code family, which is the regime in which a prefix artifact can pay at all.

5.3 RQ6: transfer across three workflow families

The result transfers across all three decisions and tool vocabularies. Compiled programs reach 90/90 exact outcomes versus 89/90 for unchanged agents. Aggregating raw paired counts, compilation reduces provider requests 66.6%, visible tool interfaces 44.2%, tokens 63.1%, observed wall latency 64.2%, and estimated cost 58.7%. Per-family ranges are more informative than the aggregate and are plotted in fig. 15: requests fall 50.0–75.0%, tokens 39.5–81.4%, latency 51.7–73.0%, and cost 32.0–75.3%. The single baseline miss in backlog routing does not establish a quality improvement (exact McNemar $p = 1$). All three artifacts record the paper's tightest per-candidate calibration: registered $\alpha = .05$ over 92 zero-violation groups (table 3). This does not upgrade them to a candidate-search-adjusted compiler-wide guarantee.

Because the three arms are paired on the same records, the preservation statement can be pooled: across all 90 held-out pairs there is no compiled-only failure, so the one-sided 95% exact upper bound on the compiled-only discordance rate is 3.3%, against 9.5% from any single 30-pair family. This is the strongest preservation bound the paper's data supports. It is nonetheless a bound over three *different* admitted artifacts on one snapshot, so it constrains the observed sample rather than certifying any one artifact for a new workload, and it is still an order of magnitude away from a non-inferiority margin a deployment would want.

Table 9: Every candidate the deployable pipeline reached in the permissive arm. The refusals are not support failures: two miss on 89 and 90 eligible calibration groups against the 92 required, at $U = 0.051$ against $\alpha = 0.05$. Two rows mine a three-call window but synthesize two steps, because the login password slot is not groundable in every supporting group — the maximal-justified-prefix retirement of fig. 3, on a public corpus.

Mined candidate window	Synthesized	Support	Elig. cal.	U	Outcome
`profile → passwords`	2 steps	22	92	**0.0498**	admitted
`profile → passwords → phone.login`	2 steps	22	92	**0.0498**	admitted
`profile → passwords`	2 steps	22	92	0.0498	dominated
`profile → passwords → spotify.login`	2 steps	22	89	0.051	retire
`profile → passwords → simple_note.login`	2 steps	19	90	0.051	retire
`profile → passwords → spotify.login`	3 steps	11	46	1.000	retire
`profile → passwords → phone.login`	3 steps	8	26	1.000	retire

Table 10: Structural dispatch eligibility of the admitted AppWorld artifact on the 28 released official baseline runs, by agent architecture. Eligibility is the admitted program's exact call sequence at normalized position 0, so it is an upper bound on ϕ and not ϕ: the manifest check and the calibrated gate are not evaluated, and the released logs retain API calls rather than model boundaries. A miss is a clean abstention to the unchanged agent.

Released baseline architecture	Runs	Trajectories	Eligible at position 0	Rate
Full code with reflection	8	2,340	2,339	1.000
Iterative parallel function calling	4	1,170	762	0.651
Plan and execute	8	2,340	1	0.000
ReAct	8	2,340	0	0.000
Pooled	28	8,190	3,102	0.379

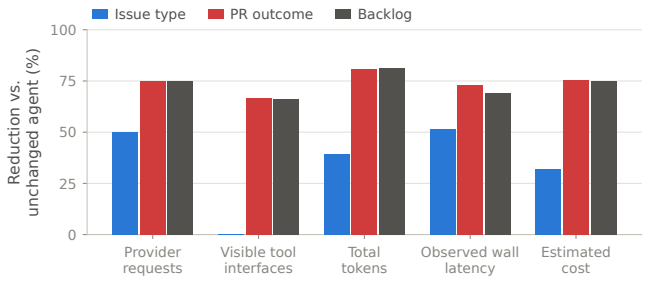


Figure 15: Per-family reduction against the unchanged agent. The issue-type family is the low end of every metric because its groundability proof admits only a two-read prefix and leaves the third read — and the interface — with the agent.

The strongest comparator prevents an exaggerated conclusion. Hand-written conditions also reach 90/90. On the two new families, fairly placed pre-model manual programs have the same single-request and single-interface structure as compiled programs and comparable resource use. The measured value of compilation is therefore automatic trace-grounded discovery, rejection, and lifecycle management; this experiment does not show runtime dominance over a correct manual implementation. All records come from one snapshot of one repository, so RQ6 is supported for workflow-family transfer only, not cross-repository or time-forward generalization.

Context-compression ablation. To distinguish replacing model-mediated decisions from merely shortening the remaining context, we reran the PR-outcome and backlog-attention cohorts with version-pinned Headroom 0.5.18. The paired design adds Headroom-only and GAC plus Headroom to the unchanged and compiled conditions (and retains the manual pre-model reference as a fifth arm). At the deliberately narrow, model-visible JSON boundary, Headroom attempted 120 eligible payloads per family (90 tool results and 30 compiled evidence objects), applied zero transformations, and saved zero tokens. All four new paired quality contrasts remain 30/30 versus 30/30 exact contracts (McNemar $p = 1$). The observed total-token differences are therefore ordinary generation variation rather than compression: Headroom-only is −0.15% on PR outcome and +0.30% on backlog attention, while GAC plus Headroom is +0.28% and +0.25% relative to GAC alone. This is a negative result for this short JSON-payload boundary, not evidence that Headroom cannot help on longer or differently shaped contexts; it neither changes the compiler nor enters the primary 90-record aggregate.

5.3.1 Cross-repository, time-forward extension. A separate frozen-source extension asks a narrower question: can the guarded compiler survive beyond one repository when the task is exact and two-read? The provider-free preflight seals five repositories and a 150-case held-out capacity. The executed discovery pass reaches 580/580 exact traces (116/116 per repository). Four repositories admit artifacts and complete 120 held-out paired records: `huggingface/datasets`, `pandas-dev/pandas`, `psf/requests`, and `streamlit/streamlit`. Baseline, compiled, and fixed-template conditions all pass 120/120 exact contracts on those completed repositories.

Relative to baseline, compiled execution reduces provider requests 44.4%, total tokens 52.4%, observed wall latency 49.4%, and estimated cost 48.6% across the 120 held-out pairs. The fixed template is more efficient still, reducing requests 66.7%, total tokens 78.6%, observed wall latency 68.1%, and estimated cost 73.3%. That

Table 11: Primary real-record, live-provider results. Entries are reductions from the unchanged baseline; exact counts are paired held-out task contracts. "Interfaces" counts provider-visible tool calls, not the three verified reads inside a composite.

Family	Exact $B \to C$	Requests	Interfaces	Tokens	Latency	Cost
Issue-type routing	30/30⟶30/30	50.0%	0.0%	39.5%	51.7%	32.0%
PR-outcome audit	30/30⟶30/30	75.0%	66.7%	80.8%	73.0%	75.3%
Backlog-attention routing	29/30⟶30/30	74.8%	66.3%	81.4%	68.9%	75.1%
Weighted total	89/90⟶90/90	66.6%	44.2%	63.1%	64.2%	58.7%

matters because it sharpens the real claim: on one conservative frozen cohort the learned artifact is valuable for automatic discovery, guarded admission, and retirement, not because it dominates a fixed ungated template on this simplified task.

This conservative cohort is informative because it retains a principled negative. Across the four completed repositories, the learned artifact compacts all 40 `merged` and all 40 `closed_unmerged` held-out pull requests, falls back on all 40 `open` ones, and `pytorch/pytorch` retires at compile time under the frozen-candidate exact gate.

A second rerun asks whether that open-only coverage pattern is intrinsic. On the three repositories with enough class support for a balanced strict time-forward split (`pandas-dev/pandas`, `psf/requests`, and `pytorch/pytorch`), a provider-free round-robin preflight seals 120 discovery pull requests and 60 held-out pull requests per repository, evenly split across `open`, `merged`, and `closed_unmerged`. The executed discovery reaches 360/360 exact traces. All three repositories admit the same two-read artifact, and baseline, compiled, and fixed-template conditions all pass 180/180 exact contracts on the held-out cohort. Relative to baseline, compiled execution reduces provider requests 66.7%, total tokens 78.4%, observed wall latency 60.7%, and estimated cost 72.7%. The fixed template is essentially tied on this simplified task: it matches 180/180 exact contracts and the same 66.7% request reduction, with slightly lower observed wall latency (63.0%) and slightly smaller token and cost savings (78.4% and 72.3%) than the learned artifact.

Crucially, the learned artifact now compacts all 60 `open`, 60 `merged`, and 60 `closed_unmerged` held-out pull requests, and the verifier's `pr.state` hull contains both `open` and `closed` on all three repositories. The earlier open-only fallback pattern and `pytorch/pytorch` retirement are therefore properties of the original frozen cohort design, not intrinsic limits of the two-read guarded runtime. The balanced rerun strengthens the paper's cross-repository evidence, but it does not widen the task: both extensions remain narrower than the three-tool workflow-family studies.

Independent refusal on API-Bank. API-Bank contains 389 observed calls over 49 tools. Conservative effect barriers leave 48 candidate windows in 37 tasks and 19 families; maximum support is eight. Two families synthesize, but their two held-out windows produce zero passes, two abstentions, and zero wrong executions. The configured exact gate requires 92 independent zero-violation groups and retires every family. Re-executing the pinned APIs reproduces 338 of 389 recorded calls and 162 of 212 complete tasks; fixture drift and missing dependencies are reported separately. Thus a second trace-complete corpus supports the same narrower result as NESTFUL: recurrence and replayability do not supply admission evidence.

5.4 RQ2: real-provider efficiency and observed quality

5.4.1 Natural tool ordering, compilation depth, and a live macro. In the expanded replication, all 132 discovery traces read the record, then labels, then comments without the prompt naming tools or order; 130 pass the exact-source task. The compiler does *not* blindly turn recurrence into a macro. It rejects the full three-read candidate because the comments limit has no consistent trace-grounded expression, then emits the groundable two-read prefix. The gate admits 92/92 calibration groups with zero registered violations at $\alpha = .05$, $\delta = .10$, and 11 thresholds, for simultaneous upper bound 0.0498. Thresholds below 0.11 admit none and those at or above 0.11 admit all, so this remains a sample-size gate rather than a demonstrated risk–coverage discriminator.

Relative to the unchanged agent, the partial compiler reduces requests 50.0%, total tokens 39.5%, observed wall latency 51.7%, and estimated cost 32.0%, while leaving three necessary reads. The macro also halves requests, but reduces tokens 58.2%, cost 37.5%, and tool calls 66.7%. It therefore beats the learned artifact on tools, tokens, and dollars; the compiler has lower observed mean wall time (2.97 versus 3.25 seconds), but its paired 95% interval against the macro crosses zero. Every primary arm passes 30/30 exact factual and full task contracts. With zero compiler-only failures, the one-sided 95% exact upper bound on the sample's underlying discordance rate is still 9.5%; this is observed preservation on one domain, not equivalence or non-inferiority.

Ten records are repeated once per arm. Category decisions agree in 10/10 for all arms, but byte-exact answers agree in 7/10 baseline, 6/10 compiled, and 4/10 macro pairs; tool traces agree in 9/10, 9/10, and 10/10. Compaction therefore does not improve free-text determinism here. The provider-free semantic regrade changes no output, token, timing, or cost measurement: it removes a planted literal comment-limit requirement that contradicted the "as needed" prompt, preserves all 212 online quality objects, and binds the final result to the paid discovery checkpoint by SHA-256.

Table 12: Expanded natural-order Tier-2 replication on 30 fresh public issue records with live provider calls and all six condition orders balanced five times. Means use primary pairs only. Exact pass checks source-grounded fields; every arm also passes the safe-read task contract 30/30.

Condition	Requests	Tools	Tokens	Wall (s)	Cost	Exact pass
Unchanged agent	4.0	3.0	4259.4	6.16	$0.000872	30/30
GAC (two-read prefix)	2.0	3.0	2576.5	2.97	$0.000593	30/30
Hand-written macro	2.0	1.0	1780.8	3.25	$0.000545	30/30

The earlier 18-record follow-up tested the more aggressive three-read artifact and remains important negative evidence. Its artifact was calibrated at $\alpha = .10$ over 45 groups (upper bound 0.0992), so it too sits outside the registered 5% level (table 3); the factual miss below is therefore a miss by an artifact admitted under a weaker guarantee than the primary families receive.

Relative to its unchanged agent, that artifact reduces requests 75.0%, total tokens 66.0%, observed wall latency 76.4%, and estimated cost 48.7%; its macro reduces the same quantities by 50.0%, 60.5%, 17.7%, and 41.7% (table 13). The unchanged agent and macro pass 18/18 factual contracts, but GAC passes 17/18. On issue 6602 the source contains a Markdown link while the compiled continuation drops the URL and returns only its anchor text. The single discordance gives two-sided exact McNemar $p = 1$ and a one-sided 95% exact upper bound of 23.8%. H3 is therefore supported as an observed-sample result for the expanded two-read artifact and not supported for the earlier three-read artifact; preservation is not invariant to compilation depth.

The miss fixes the gate's scope. Zero calibration violations concern the compiled *tool program*: grounded arguments, tool outputs, call counts, and declared effects. The verifier does not certify the downstream model continuation. Thus 45/45 clean program replays and a held-out factual-output miss are compatible. A deployment claim requires a second admission layer over continuation outcomes or a deterministic checked renderer.

We implemented that boundary as a framework-neutral `ContinuationGuard`. It checks the candidate answer, then tries an optional deterministic renderer and baseline in that order, revalidating either output before release; invalid recovery returns no output. A provider-free replay accepts 17 retained compiled answers, detects issue 6602, and checked-renders it from source observations, yielding 18/18. This post-hoc mechanism check establishes detection and rendering on retained records, not live latency, cost, non-inferiority, or cross-domain safety.

That earlier study's online grader also imposed an undocumented excerpt minimum and misread a literal comment "none" as a sentinel. Its separate provider-free revision corrected four rows, retained online labels and the prior digest, and changed no provider metric. It did not retrain the executed artifact; after correction issue 6602 remains the only compiled-only failure.

5.4.2 Prescribed-prefix ablation. The compiled GitHub artifact has 16/16 training-group support, passes all eight development replays, and admits 92/92 calibration groups with zero observed violations. At $\alpha = .05$, $\delta = .10$, and 11 thresholds, the simultaneous upper bound is 0.0498. The artifact was promoted only under a paper-protocol lab flag; no production canary is claimed.

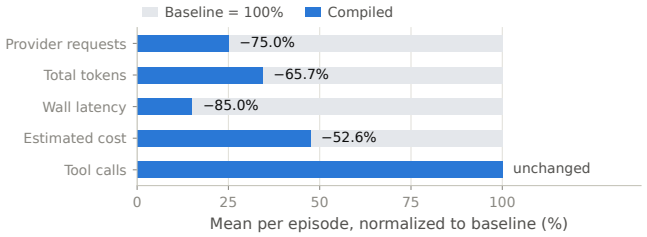


Figure 16: Prescribed-prefix ablation, 18 held-out pairs, each mean normalized to its own unchanged baseline. Tool calls are the control: the compiled program performs the same three reads, so what falls is the model-mediated control flow around them, not the work.

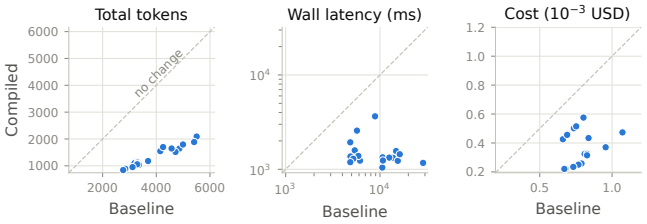


Figure 17: The same 18 pairs, unaggregated. Every issue is one point, baseline against compiled, so the reduction is visible as a distribution rather than a mean: no pair crosses the no-change diagonal on any of the three endpoints. Latency is plotted on log axes because the baseline spans an order of magnitude.

Provider requests fall from exactly 4 to 1 per issue (75.0%; paired 95% bootstrap difference $[-3, -3]$). Mean total tokens fall 65.7%, observed wall latency 85.0%, and estimated cost 52.6%; tool calls remain three in both arms (fig. 16). The effect is not carried by outliers: every one of the 18 pairs falls below the no-change diagonal on tokens, latency, and cost (fig. 17). All 18 pairs pass the registered contract, but zero discordances still permit a 15.3% one-sided 95% upper bound on degradation, so this is not an equivalence result. Discovery consumed 528 provider requests and 533,293 tokens; measured break-even ranges from 176 future episodes when

Table 13: The earlier free-order 18-record follow-up, in which GAC compiled the more aggressive three-read region. Means are per issue over 18 held-out pairs with all six condition orders balanced three times. This is the artifact calibrated at $\alpha = .10$, not one of the primary families, and it is the arm that records the single factual miss discussed below.

Condition	Requests	Tools	Tokens	Wall (s)	Cost	Factual pass
Unchanged agent	4.0	3.0	4065.4	5.69	$0.000787	18/18
GAC	1.0	3.0	1381.0	1.34	$0.000404	17/18
Hand-written macro	2.0	1.0	1604.9	4.68	$0.000459	18/18

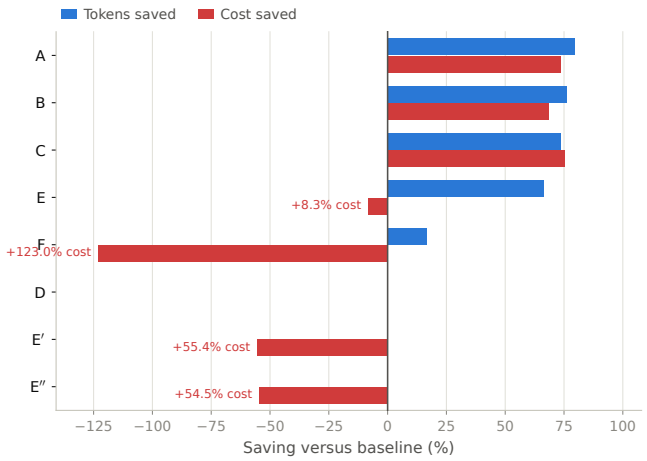


Figure 18: Controlled stress suite. Three of the eight conditions are negative controls whose only correct outcome is no compaction, and two of the compacting rows cost *more* than their baselines once prompt-cache reuse is fragmented.

amortizing requests to 292 when including confirmatory-arm cost, before engineering, review, monitoring, and cache effects.

5.5 Harder shapes, partial compaction, and where the saving inverts

The stress suite confirms the intended boundary behavior (fig. 18). In Demo E, an order-fulfillment exception workflow branches over reads and pagination before a mandatory irreversible write. The live SDK run therefore compacts only the read prefix: model requests fall from 7.0 to 2.0 and total tokens by 66.4%, while all four baseline and compacted episodes pass the registered task contract (1.00/1.00). The measured estimated cost rises 8.3% because deleting turns fragments prompt-cache reuse. This is a fictional deterministic WMS fixture, not a real fulfillment system or business-record study.

The write remains under the ordinary agent, and the current Demo E program is a manually authored straight-line/branch fixture used to test the runtime effect boundary rather than evidence that automatic synthesis can safely infer arbitrary writes. An undeclared MCP surface, an unsupported loop-bearing artifact, and a drifted entry schema all reproduce the baseline model-call count and quality. These refusals are the intended result: the paper uses the suite to establish partial-compaction and fail-closed behavior, not a production write-safety guarantee.

Table 14: GAC against the obvious engineering alternative — one hand-written composite tool performing the same reads — on the deterministic offline stress study. R_{req} is the paired request ratio against the unchanged agent, so lower is better and 1.000 means no saving. The macro is at least as good on three of five workloads. This substrate is simulated: a scripted policy stands in for the model at each boundary, so these numbers cannot settle the live question. They can, and do, settle whether the comparison was run.

Demo	Workload	Macro R_{req}	GAC R_{req}	Lower is better
A	Tier-1 support evidence gathering	0.726	0.755	**macro wins**
B	permissioned RAG knowledge assistant	0.335	0.718	**macro wins**
C	multi-agent incident triage	0.922	0.780	GAC wins
D	multi-tenant MCP operations (negative control)	0.724	1.000	**macro wins**
E	order-fulfillment exception handling	0.661	0.642	GAC wins

5.6 HMDA public-record preflight: a validated extension, not an optimization result

The HMDA checkpoint makes the proposed second domain concrete without overstating what has been run. The frozen pool contains 420 privacy-modified public LAR groups, and an independent validator reproduces all 420 exact gold records; 416/420 (99.05%) expose a variable path for a future trace comparison. The validator executed zero provider calls. Consequently, HMDA contributes a verified data, schema, privacy, and provenance boundary only. It contributes no measured baseline, GRC, or reviewed-macro quality, request, token, latency, cost, determinism, or workflow-reduction result, and it remains outside the primary effectiveness denominator. The SEC row in table 5 is retained as an explicit source-gated negative disposition rather than silently dropping an incomplete three-domain protocol.

5.7 The comparator that matters: a hand-written composite tool

For a workflow whose reads are already known, the obvious engineering answer is not a compiler: it is to write one composite tool that performs them (fig. 19). A request reduction measured

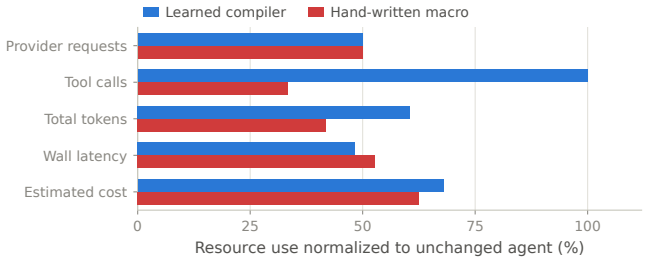


Figure 19: The live comparison that constrains the contribution, on the 30-pair expanded replication. The hand-written macro is at or below the compiled condition on every resource except observed wall latency, and uses one tool call where the compiler keeps three.

only against an unoptimized agent cannot distinguish "compilation works" from "this workflow was easy". Table 14 therefore reports both against the same baseline on the offline stress study, and the result is not in our favour.

On the permissioned-retrieval workload the macro is far better ($R_{\text{req}} = 0.335$ versus 0.718): it collapses six reads into one call, whereas GAC must keep each call separately dispatchable to verify its live-outs. On Tier-1 support the macro is slightly ahead. GAC wins where the *choice* of which reads to make is itself part of the recurrent structure — multi-agent triage (0.780 versus 0.922), where a composite tool cannot remove the coordinator's routing turn, and fulfillment (0.642 versus 0.661), where branch-dependent evidence means a single fixed composite would over-read.

The negative control is the most informative row. A hand-written macro reaches $R_{\text{req}} = 0.724$ on the MCP workload; GAC reaches 1.000, because the tool effects are undeclared and the catalog fails closed. The macro takes a 27.6% saving that GAC refuses. Whether that refusal is prudence or lost value depends entirely on whether those undeclared tools really are read-only — which is exactly the question a human answers by writing the macro and the compiler declines to guess. Neither number is "right"; the pair is the point.

We therefore do not claim that GAC dominates ordinary engineering. The expanded live comparison in table 12 reaches the same conclusion directly: both candidates pass 30/30, but the macro matches the partial compiler's request count and uses fewer tools, tokens, and dollars; only observed wall latency favors the compiler. The earlier aggressive artifact saves one additional provider turn but records a factual miss. Where a hand-written composite is available and its effects are understood, write it. GAC earns its complexity only when the region is not economically maintainable as a manual macro, is branch-dependent, or requires guards a macro would otherwise assume implicitly.

5.7.1 Why the token win and the dollar win are different sizes. The macro's advantage shrinks once prefix reuse is priced, and the reason was recorded all along. In the issue-type family the hand-written macro uses 30.9% fewer total tokens than the compiled condition but is only 8.0% cheaper, because it retains no cache reads at all (0.0% of its input, against 27.8% for the compiled condition and 32.3% for the unchanged agent; table 15). Collapsing three reads into one call shortens the prompt and simultaneously destroys the repeated prefix that made the remaining input cheap. A token-only objective scores that trade as a clear macro win; a cache-aware objective scores it as a near-tie. This is the effect the prompt-cache literature [33, 43] describes for compression, arising here from interface fusion.

Table 15: Prompt-cache structure of the primary families, recovered provider-free from the retained per-record usage of the same runs. "Input" is mean input tokens per episode, "Cached" the share of them served as cache reads, "Cache-write" the mean tokens written per episode. Cache reads are already priced by each study's frozen price table; the shares explain why token and dollar changes diverge.

Family	Condition	Input	Cached	Cache-write
Issue type	unchanged	4062	32.3%	1177
	GAC	2435	27.8%	1165
	hand-written	1633	0.0%	828
PR outcome	unchanged	2592	0.0%	185
	GAC	458	0.0%	0
	hand-written	458	0.0%	0
Backlog	unchanged	2689	0.0%	77
	GAC	454	0.0%	0
	hand-written	454	0.0%	0

The same table exposes a limit on our own cost numbers. The two newer families record zero cache reads in every arm, so their 75.1–75.3% cost reductions are measured against a cache-cold baseline, whereas the issue family's 32.0% is measured against a baseline that serves a third of its input from cache. Part of the width of the reported 32.0–75.3% range is therefore a property of cache warmth rather than of the compiled programs, and a production baseline with a warm prefix should be expected at the low end. We report the shares instead of narrowing the range post hoc, but a cache-controlled replication is the correct fix and we do not have one.

Amortization for the families that ship. Each family's compiler learned from 132 paid discovery episodes: \$0.1146, \$0.0931, and \$0.0988 for issue-type, PR-outcome, and backlog routing respectively, at 528 provider requests each. Against the measured per-episode dollar saving, provider-side break-even is 411 episodes for issue-type routing and 182 and 181 for the two newer families — so the weak two-read artifact needs more than three times its own discovery cohort to repay itself in provider spend alone, before any engineering, review, monitoring, or invalidation cost. The 411/182/181 spread tracks compiled depth, not workload volume, which is the practical form of the depth sensitivity section 9 describes.

The result has a direct mechanistic explanation and a testable hypothesis. The macro is allowed to redesign the application interface: it fuses three reads, returns a task-specific sufficient record, and exposes one tool schema. The compiler is constrained to preserve existing interfaces, provenance, intermediate evidence, and fallback semantics; in the expanded run its groundability proof permits only a two-read prefix and all three logical reads remain necessary. We therefore hypothesize that manual macros dominate on

Table 16: Fresh real-record comparison after adding guarded composite synthesis. "Exposed" counts model-visible interfaces; both conditions execute three source reads. GCS runs the admitted composite before the first provider request. This is a 12-pair exploratory extension designed after the earlier macro result, not a confirmatory replacement for table 12.

Condition	Requests	Exposed	Reads	Tokens	Wall (s)	Cost	Exact
Provider-visible macro	2.0	1.0	3.0	1524.0	2.49	$0.000430	12/12
Pre-model GCS	1.0	1.0	3.0	931.2	1.49	$0.000291	12/12

Table 17: Fresh real-record evaluation after bounded prompt optimization. Values are means over six held-out issues. "Interfaces" counts model-visible tool calls. GEPA retained its seed; the combined row is therefore a GCS replication, not a distinct optimized prompt. Optimization overhead is excluded.

Condition	Requests	Interfaces	Input	Total	Wall (s)	Exact
Unchanged	4.0	3.0	3542.0	3683.5	5.08	6/6
GEPA (seed retained)	4.0	3.0	3530.0	3669.3	4.33	6/6
GCS	1.0	1.0	770.8	861.0	1.61	6/6
GCS + retained seed	1.0	1.0	770.8	868.8	1.78	6/6
Manual pre-model	1.0	1.0	770.8	865.5	1.46	6/6

stable, low-entropy workflows, while guarded compilation earns its cost across branching or changing workflow families for which manual discovery and maintenance do not amortize. This study establishes the structural facts behind that hypothesis, but it does not measure engineering effort, drift response, or the cross-workflow break-even.

5.8 Exploratory GCS: closing the measured macro gap

We implemented the interface transformation suggested by the comparator rather than discarding the guard. The compiler now canonicalizes only application-declared task-semantic arguments, emits the complete three-read program behind one projected interface, retains all internal provenance and verification, and executes it before the provider only when the continuation manifest matches exactly. Provider-free reconstruction of all 132 retained discovery traces dispatches 124, safely falls back on eight, and produces 124/124 exact projections with no projection failure. That replay makes no new model call and supports implementation correctness, not an efficiency claim.

This artifact was calibrated at $\alpha = .10$, not at the registered $\alpha = .05$ (table 3). Its gate admits 88 of 92 calibration groups with zero violations for a simultaneous upper bound of 0.0520, which exceeds .05; at the registered level the family would have retired and there would be no GCS result to report. Everything below is therefore a 10%-selective-risk result, and it is not interchangeable with the three primary families.

The paid comparison uses 12 further public issues excluded from all 424 issue identifiers used by the preceding experiments. Both the provider-visible hand-written macro and GCS pass all 12 exact factual and tool contracts; with no observed discordance, exact McNemar $p = 1$, and the one-sided 95% zero-event bound still permits a 22.1% population discordance rate. Relative to the measured macro, GCS reduces provider requests from 2.0 to 1.0 (−50.0%), total tokens from 1,524.0 to 931.2 (−38.9%), mean wall latency from 2.49 to 1.49 seconds (−40.0%), and estimated cost from $0.000430 to $0.000291 (−32.3%). Paired request, total-token, latency, and cost differences all have two-sided exact signed-rank $p = 0.000488$; output-token reduction alone is uncertain ($p = 0.155$). Each condition exposes one interface and still performs three source reads, so this is decision and context compaction, not elimination of necessary evidence.

The mechanism is placement, not magic: the provider-visible macro spends one request choosing its composite and another producing the answer, whereas GCS supplies the guarded projection to the first request. The follow-up in section 5.9 implements the previously missing manually engineered, equally guarded *pre-model* comparator. The run also covers only five bug and seven other-labelled issues—no question or enhancement cases survived strict prior-cohort exclusion. A retained two-case smoke run had worse GCS latency, reinforcing that the 12-case latency magnitude is provider- and schedule-sensitive. The supported conclusion is therefore precise: automatic guarded interface synthesis matches the measured macro's observed quality and beats its measured provider work on this family, while preserving safe fallback. It is not evidence of an advantage over manual placement.

5.9 Fair placement and bounded prompt optimization

All five conditions pass all six exact factual and tool contracts. The GCS arm executes the same $\alpha = .10$ artifact as section 5.8, so this comparison inherits that looser risk level (table 3). Relative to the unchanged agent, GCS reduces provider requests 75.0%, exposed interfaces 66.7%, input tokens 78.2%, total tokens 76.6%, observed wall time 68.2%, and estimated cost 67.9%; the paired structural and token reductions have exact signed-rank $p = .03125$. These six pairs remain too few for a population equivalence claim.

The fair manual program is the decisive comparator. It also uses one request, one exposed interface, and 770.8 mean input tokens, and passes 6/6. Against it, GCS changes none of those structural metrics. GCS emits 4.5 fewer output and total tokens on average ($p = .25$), while observed wall time and cost differences are also non-significant ($p = .844$ and .625). Automatic discovery, provenance, compatibility pins, and calibrated admission may justify GCS operationally, but this study shows no runtime dominance over a correctly placed hand-authored program. Manual construction and review effort were not measured.

GEPA consumes 14 real task evaluations and three real reflection calls under the fixed budget, proposes three alternatives, and retains the seed. Its held-out arm therefore keeps four requests and three interfaces; total-token reduction versus unchanged is 0.38% ($p = .688$), and estimated cost is 3.6% higher ($p = .688$). Optimization itself consumes 59 provider requests, 63,954 tokens, and an estimated $0.01163, reported outside the held-out arm. Because no prompt change is selected, the combined arm cannot support a composition claim. This is a bounded negative result on one extractive workflow, not a general failure of GEPA. One GCS+seed

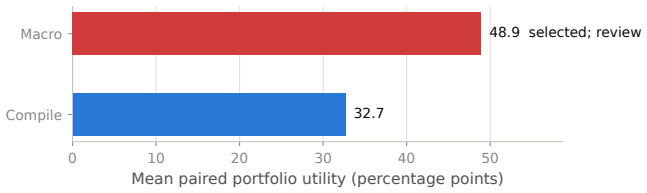


Figure 20: The frozen selection, taken over the 30 independent calibration issues. Both actions clear the risk limits, so the decision is made on mean paired utility alone and the measured macro wins it. Selection routes the action to human review; it does not deploy it.

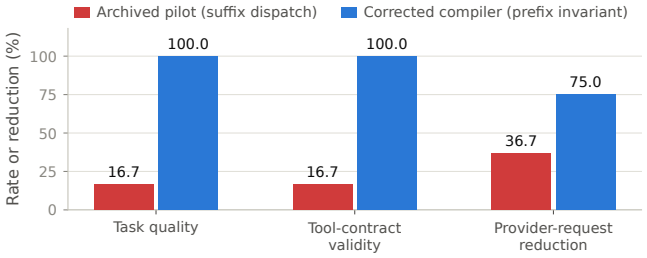


Figure 21: Why position is an invariant and not a tuning parameter. The archived pilot dispatched a compiled region from the wrong boundary and still reduced provider requests, which is exactly what makes it dangerous: efficiency survived while task quality and tool-contract validity collapsed to 16.7%. An optimizer scored on reduction alone would have accepted it.

provider-span record exceeds its surrounding host wall time; the raw span is retained, but provider-span comparisons involving that arm are invalidated.

5.10 RQ5: prospective portfolio selection

Both actions record zero quality failures and zero non-positive-utility groups among 30 independent calibration issues. With the multiplicity split, each one-sided upper bound is 0.1359, below the pilot limit 0.15. Mean utility is 0.327 for compilation and 0.489 for the macro, so the frozen selector recommends the macro for review before seeing a fresh 12-issue cohort (fig. 20). Baseline and macro then pass 12/12 exact contracts; the macro halves provider requests, reduces tool calls from three to one, and lowers total tokens 59.2% and estimated cost 40.6%. This is a prospective action evaluation, not evidence that the selector beats an always-macro policy: calibration and test contain one workflow family, and the static policy would make the same choice.

5.11 RQ4: failed pilot and determinism

The first live pilot compiled suffix-only regions even though the runtime resolves only at entry. It therefore dispatched label/comment reads before their intended position, duplicated or reordered tools, and achieved only 16.7% task/tool-contract validity despite reducing requests (fig. 21). We archived the complete pilot, added the prefix invariant in eq. (2), added a regression test, and excluded every pilot issue from the final cohort. The corrected compiler blocks 116 suffix candidates and restores 100% observed contract validity.

On six repeated test issues, category decisions and tool traces agree perfectly in both conditions. Byte-exact natural-language answers agree in one of six baseline pairs and zero of six compiled pairs. Compaction therefore improves structural determinism of the tool prefix by construction, but not free-form surface determinism. This small sample does not support a claim that it stabilizes final text.

5.12 What did not work, and what each failure rules out

Five results in this section are negative, and they constrain the contribution more sharply than the efficiency numbers do. They are collected here because each one closes a different explanation that a reader would otherwise be entitled to keep open.

Three externally sourced substrates retired, and the fourth exposed why that was not yet informative. Twelve synthesized NESTFUL families, both API-Bank families, and four executed-BFCL families reached held-out replay without a single wrong execution — 24 passes and 12 abstentions, two abstentions, and 3 passes with 3 abstentions respectively — and none reached the 92 zero-violation groups the configured gate requires (section 5.1). This rules out the reading that recurrence plus successful replay is sufficient evidence to rewrite an agent, which is the assumption a recurrence-only macro miner makes. It also fixes the cost of the guarantee: on these corpora the admission requirement, not the synthesizer, is the binding constraint.

The honest weakness of that argument is that all three shortfalls were settled by corpus size — largest supports of 26, 8, and 15 against 92 — before any compilation happened, so none of the three actually tested whether the gate can be wrong. AppWorld does (section 5.2): it reaches 136 groups, admits one artifact at $U_{\hat{\eta}} = 0.0498$, and then still refuses four further candidates — two of them short by three and two eligible calibration groups, at $U = 0.051$ against $\alpha = 0.05$. What that buys is a refusal that was not arithmetically pre-determined. What it does not buy is a demonstrated risk–coverage frontier, because the admitted gate is as step-like as the live ones, or a provenance demonstration, because the admitted program is argument-free.

The admitted artifact does not dispatch on most real agents. Measured against 8,190 released official-baseline AppWorld trajectories, the admitted region is structurally eligible at the entry boundary on essentially every full-code agent trajectory and on 1 of 4,680 ReAct and plan-and-execute trajectories (section 5.2.1). This rules out reading admission as deployability: an artifact can satisfy every provenance, effect, position, contract, and finite-sample condition in this paper and still never fire, because whether an agent presents a compilable prefix at its entry boundary is a property of its calling discipline that no admission condition tests. The 8,190 trajectories repeat tasks across 28 released runs and are therefore descriptive units, not independent samples; the reported 37.9% is an architecture diagnostic rather than a population estimate.

The suffix pilot reduced requests while destroying the task. It cut provider requests 36.7% at 16.7% contract validity (section 5.11).

This rules out efficiency-scored evaluation of this class of optimizer: a reduction metric ranked the broken compiler above no compiler at all.

The gate did not discriminate. Five of six live gates admit all or none, and the single partially selective gate refuses four of its 92 groups — which is precisely what lifts its own bound above the registered 5% (section 9). The AppWorld admission replicates this on a public corpus: its coverage moves from 0 at $\eta = .05$ to 1.00 at $\eta = .08$ and stays there. This rules out any claim that the calibrated score separates risky inputs from safe ones on this evidence; what the mechanism demonstrably provides is a sample-size requirement and a refusal, not a demonstrated risk–coverage frontier.

Bounded prompt optimization selected no change. GEPA spent 59 provider requests and retained its seed (section 5.8). This does not rule out prompt optimization in general — the budget was small and the workload narrow — but it does rule out the specific alternative explanation that the compiler's gains are available from prompt optimization alone on this task.

Hand-written programs matched the compiler. Manual programs reach 90/90 on both new families, and an independently authored pre-model program ties GCS at 6/6 with identical request and interface counts (section 5.7). This rules out runtime dominance as the contribution and leaves automatic discovery under a stated admission rule as the part of the claim the evidence still supports. The drift-robustness protocol targeted this parity directly: a three-arm design isolates the *induced verifier* from the *program* by pairing the compiled artifact with a permissive no-op verifier and pairing the hand-written program with the same nine-family metamorphic perturbation suite implemented in `evaluation/perturb.py` (`paper/supplementary/drift-robustness-ablation-protocol.md`). It has since run in three studies (section 6.4): the verifier and a permissive verifier produced the same wrong outcomes in every arm, and under emptied evidence the guarded arm gave a silent wrong answer on 39 of 60 records, exactly as the permissive arm did.

6 LATER EVIDENCE: A PUBLIC SQL BENCHMARK, FRESH RECORDS, REPLICATIONS, AND RETIRED GATES

The studies above were extended after this article's core experiments were sealed. The ICLR 2027 submission (`paper/iclr/`) is the current statement of the paper and reports them in full; this section records, in this article's terms, what they add and which earlier statements they change. Where this article and the ICLR paper differ, the ICLR paper governs. Every study below was registered in a protocol under `paper/supplementary/` before its provider calls, and none changes a headline number of the three-family study.

6.1 A public benchmark where GAC helps: BIRD

A tool-using SQL agent on `gpt-5.6-luna` answers questions from the BIRD text-to-SQL benchmark [28] (release `dev_20240627`) over read-only SQLite files through three tools: `list_tables`, `get_schema(table_names)`, and `run_query`. The contract is BIRD execution accuracy. Each of the five dev databases with at least 146 questions is a family (16 train, 8 dev, 92 calibration, 30 held out). Two agent designs were registered: a *standard* agent that queries the schema of the tables it judges relevant [26], and a *schema-first* agent whose workflow reads every table before writing a query.

Table 18: BIRD runs. "Std." and "S.-first" count databases admitted per design (the rotated rerun reuses the primary artifacts). Correct counts are on questions completed by all four conditions; bold marks the compiled arm, not a significant improvement. Reductions compare the compiled agent with the unchanged agent; † every primary-row reduction is against the warm repeat run. Cost* prices every input token at the uncached rate.

Run	Model	DBs	Std.	S.-first	n	Correct (base / GAC)	Requests	Tokens	Latency	Cost	Cost*
Primary (fixed order)†	`gpt-5.6-luna`	5	0	5	150	94 / **97**	44.9%	9.8%	29.4%	6.3%	11.2%
E1 rotated rerun	`gpt-5.6-luna`	5	—	—	150	94 / **100**	46.0%	12.3%	33.9%	16.2%	13.9%
E2 second model	`gpt-6-luna`	5	0	5	149	91 / **93**	49.9%	16.8%	38.4%	6.1%	17.5%
E3 second provider	`claude-sonnet-5`	5	0	0	—	—	—	—	—	—	—
E4 train split	`gpt-5.6-luna`	26	0	21	625	401 / **400**	45.9%	12.6%	29.3%	18.1%	14.0%

The two designs separate cleanly. The standard agent passed 7 to 39 distinct table lists per database, so `get_schema.table_names` has no consistent expression and four families retire at synthesis; the fifth retires on calibration support. The schema-first agent's opening, `list_tables` followed by `get_schema` on all tables, is admitted on all five databases at 92/92 zero-violation groups with one candidate at calibration and a clean strict audit. On 150 sealed questions the compiled program dispatched every time and answered 97 correctly against 94 for the unchanged agent (McNemar $p = 0.45$; +2.0 points, 95% interval $[-1.3, +5.3]$), cutting requests 44.9%, tokens 9.8%, latency 29.4%, and cost 6.3% against a warm repeat run of the unchanged agent. The registered rotation of condition order was not implemented in this run, so the first baseline met a colder prompt cache; a rerun with the order rotated reproduces the result (100 against 94 correct, $p = 0.07$, no superiority claimed, 46.0% fewer requests). The interval permits a loss, so neither equivalence nor noninferiority is established.

Three extensions (table 18) bound the result. On a second model family (`gpt-6-luna`) the standard design retires on all five databases and the schema-first opening is admitted on all five, with 49.9% fewer requests and 93 against 91 correct on 149 complete-case questions. On the BIRD training split (26 databases) the standard design retires on all 26 and the schema-first opening is admitted on 21; the five refusals are databases of 11 to 65 tables where the agent occasionally dropped or reordered a table. On the 625 four-arm-complete questions of the 21 admitted databases the compiled agent dispatched every time and answered 400 against 401 ($p = 1.0$), with 45.9% fewer requests and 18.1% lower cost. On a second provider, Anthropic `claude-sonnet-5`, the agent followed neither opening as a fixed step (under the schema-first prompt it listed the tables first in 429 of 655 runs and passed every table in 26), and GAC retired all ten (design, database) pairs: it compiles what an agent does, not what its prompt asks for.

Counting every scheduled question, with each missing run scored incorrect in its own arm, gives 402/630 compiled against 403/630 unchanged on the training split ($p = 1$) and 93/150 against 91/150 on the second model ($p = 0.73$). A conservative paired 95% interval, built from two Clopper–Pearson intervals [13] under i.i.d. question pairs, allows a loss of 3.27 points or a gain of 2.96 on the training split (table 19); deleting any one database moves

Table 19: Retrospective BIRD quality sensitivity on all scheduled questions. Differences are compiled minus unchanged, in percentage points; intervals assume i.i.d. question pairs. "Delete-one" omits each database in turn and is a descriptive range, not a confidence interval. The last column counts databases with positive, zero, or negative differences.

Cohort	Δ (pp)	Conservative 95% interval	Delete-one range	DBs +/0/−
Primary	+2.00	[−4.40, +8.18]	[+0.83, +4.17]	3/1/1
Rotated	+4.00	[−2.53, +10.11]	[+1.67, +5.83]	3/1/1
Second model	+1.33	[−5.44, +7.97]	[−0.83, +3.33]	3/0/2
Train split	−0.16	[−3.27, +2.96]	[−0.50, +0.33]	5/10/6

the training difference only between −0.50 and +0.33 points. Net evaluation savings on that split are positive only if the unrecorded cost of its one failed compiled run is below $0.162952. These are descriptive sensitivities over selected databases, not evidence of generalization to new ones.

6.2 An end-to-end certificate on post-cutoff pull requests

After the main study we acquired every `huggingface/datasets` issue and pull request created after the pinned snapshot's last day (1,095 records) and pre-registered a study on the one family whose fresh pool supports it, PR-outcome. Three disjoint splits were frozen before any call: 60 test records (20 per class), 132 discovery records, and 132 calibration groups. The calibration groups are 84 open and 48 merged pull requests and no closed-unmerged ones, so the certified population excludes that class. The compiler re-derived a byte-identical three-read program, guard, and gate from the fresh discovery traces (131/132 exact), differing from the retained artifact only in its snapshot pin. Dispatched on all 132 calibration groups with the unchanged continuation graded on the exact task contract, it produced 0 misses. Under the single pre-registered rule ($\gamma = \delta = 0.10$, one candidate) the one-sided bound on the *end-to-end* violation rate is 0.0173, below $\alpha = 0.05$; the primary grid gives 0.035 for the same count, and treating the 100 distinct creation days or 85 distinct authors as units gives 0.0228 and 0.0267 (independence of those clusters is unverified). On the 60 fresh test records all four conditions reach 60/60, with requests down 75.0%, tokens 82.7%, latency 80.0%, and cost 76.6%. This is the first certificate in this work whose event includes the model's answer; it is per-candidate, for one family, conditional on i.i.d. groups from one repository's post-cutoff records. Across the 810 held-out GitHub episodes of six specified studies run under a compiled artifact, one produced a compiled-only contract failure.

6.3 A second model family and a second provider

Every headline number above comes from one calibrated model, so the three primary families were rerun on `gpt-6-luna` with the sealed records, splits, and discovery traces, and on a second provider, Anthropic's `claude-sonnet-5`, through a provider adapter around the unchanged harness (tables 20 and 21). Transferring the retained `gpt-6-luna` artifacts keeps the structure (requests unchanged, 30/30 dispatch, pooled reductions 66.6% requests, 63.0% tokens, 60.1% cost) with one compiled-only miss in

Table 20: Same-cohort transfer to `gpt-6-luna`: retained artifacts recompiled under the new model pin (design B) and the pipeline rerun on the second model's own traces (design A). Reductions are against the same-model baseline.

	Exact held-out contract			Reduction (%)		
Family	Base	Compiled	Manual	Requests	Tokens	Cost
Transfer (design B): retained artifacts recompiled under the new pin						
Issue-type routing	29/30	**28/30**	30/30	50.0	39.2	33.0
PR-outcome audit	30/30	**30/30**	30/30	75.0	80.8	76.3
Backlog-attention routing	29/30	**30/30**	30/30	74.8	81.0	75.4
Weighted total ($n = 90$)	88/90	**88/90**	90/90	66.6	63.0	60.1
Re-discovery (design A): the pipeline run on the second model's own traces						
Issue-type routing	30/30	**30/30**	30/30	50.0	38.9	32.8
PR-outcome audit	30/30	**30/30**	30/30	75.0	80.7	76.0
Backlog-attention routing	RETIRE at compile time (113/132 exact discovery traces)					

Table 21: Second provider, design A: Anthropic `claude-sonnet-5` on the sealed cohorts through the unchanged pipeline. Reductions are against the same-provider baseline; the backlog refusal is the 92-group floor binding.

	Exact held-out contract			Reduction (%)		
Family	Base	Compiled	Manual	Requests	Tokens	Cost
Issue-type routing	29/30	**28/30**	28/30	48.3	40.4	39.8
PR-outcome audit	27/30	**27/30**	29/30	75.0	84.9	83.4
Backlog-attention routing	RETIRE at calibration (91 and 90 of 92 groups, $U > \alpha$; 116/132 exact traces)					

90 (issue 6532, an excerpt that pluralized one source word); the certificate is conditional on the pinned model, so that is what an uncertified transfer looks like. Re-discovery from each model's own traces re-derives the identical issue-type and PR-outcome programs and *refuses* backlog routing: on `gpt-6-luna` only 113 of 132 discovery traces were exact, below the 116 the split needs, and on `claude-sonnet-5` the two candidates retire at calibration with 91 and 90 of 92 groups, the 92-group floor binding on a second provider. On Sonnet there are four compiled-only misses, every one an excerpt that drops Markdown link markup or normalizes whitespace, the issue-6602 mechanism. We claim one calibrated model family, a second on which the pipeline re-derived two of three artifacts and refused the third, and no cross-model certificate.

6.4 Where the gate and the guards earned nothing

The learned gate is a support threshold, and an amended issue gate retired. Beyond the cross-repository gate-frontier null of section 8.3.1, an amended end-to-end issue gate study (`paper/supplementary/graded-issue-end-to-end-gate-protocol.md`) also retired. A pinned 474-issue cohort reserved 200 development, 184 calibration, and 90 untouched test issues, enriched for link-bearing comments. The first attempt stopped after 288 ordinary fallbacks because the historical artifact's manifest did not match the current runtime; its checkpoint and estimated cost are retained. A compatible candidate was then re-derived from 80 disjoint historical discovery issues before any new-cohort compiled result. In the amended run the candidate dispatched on all 200 development and 184 calibration issues, with 4 and 2 final factual-task errors. The frozen eleven-point grid admitted no threshold at $\alpha = 0.05$ and $\delta = 0.10$: at $\eta = 0.02$, 64/184 groups had no errors but

$U = 0.0708$; at $\eta = 0.50$, 153/184 had two errors and $U = 0.0546$. The gate retired before the 90-issue test, so no held-out quality or savings comparison follows. The support-only ablation accepts 184/184 with the same two errors, but it disables the risk budget. This is a negative result: the certificate mostly confirms that enough examples were seen.

The 90-record link pilot is withdrawn as a compiled-effect result. The pilot discussed in section 8.3.1 named a `compiled` policy manifest while the retained artifact was registered under `base`, so exact registry resolution found no artifact and the pilot retained factual grades but no dispatch telemetry. Its Markdown-link contrast is a difference between stochastic model and tool runs, not between a compiled and an unchanged agent, and is withdrawn.

The run-time guards prevented no wrong answers under corruption. The drift-robustness protocol, listed as not executed in an earlier draft of this article, ran as three pre-registered studies. On the deterministic demonstration substrate (144 paired groups) and on the 89 paired primary GitHub records, the induced verifier and a permissive verifier both produced zero wrong outcomes, so the suite does not separate them; the verifier's measurable effect was abstention, including on perturbations that left the answer unchanged. The third, continuation-graded study (1,080 live cells) took the pre-declared adverse branch: when the tool layer returned emptied lists, an emptied discussion or assignee list lies inside the hull the verifier had learned, so the guarded arm dispatched and the model answered faithfully from the corrupted evidence, producing a silent wrong answer on 39 of 60 records as graded, and the permissive arm produced exactly the same 39. Where corruption left the hull, dispatch fell back in both arms and the fallback agent, fed the same corrupted tools, answered wrong on 30 of 30. On these records the verifier prevented no silent wrong answer. This is a statement about this contract on these corruptions with the calibrated model, not about production safety.

6.5 What the later evidence changes

Taken together, the later studies sharpen rather than overturn the conclusions. The central claim, that repeated read-only prefixes can become guarded programs that preserve held-out contracts, is supported on three GitHub families, a fresh post-cutoff cohort, and BIRD, each within the limits stated. The claims that the learned gate adds measured risk–coverage value, that the run-time checks protect against corrupted tool outputs, and that a certificate on one model transfers to another are *not* supported. NESTFUL and AppWorld remain the two public benchmarks with no measured benefit. The ICLR paper's checked-evidence assessment of the full body of evidence is 79/100, borderline, a diagnostic judgment rather than a reviewer score (`paper/iclr/notes/review_scorecard.md`).

7 RELATED WORK

Agent execution and scheduling. ReAct interleaves reasoning with environment actions [52]. LLMCompiler generates a dependency plan for the current request and schedules independent calls in parallel, reporting up to 3.7× latency and 6.7× cost improvement over ReAct [25]. GAC instead learns from repeated executions and deletes historical model boundaries; it currently executes synthesized calls sequentially. The methods are complementary.

Workflow and agent optimization. DSPy and MIPRO optimize instructions and demonstrations in declarative LM programs [24, 39]. RouteLLM selects a model based on preference data [36]; AgentSlimming prunes or replaces multi-agent graph nodes and reports token reductions up to 78.9% [9]. FlowCompile explores model, reasoning-budget, and workflow configurations for a declared graph and reports up to 6.4× speedup [27]. These optimize parameters or declared topology, whereas GAC infers a value-grounded replacement from observed traces.

GEPA uses execution and evaluation traces as natural-language feedback for reflective, instance-wise Pareto prompt evolution [2]. Across its six reported tasks it improves over GRPO with up to 35× fewer rollouts and produces instructions up to 9.2× shorter than MIPROv2 prompts. GEPA therefore prevents us from claiming trace-driven agent optimization itself as novel. It changes residual prompts rather than deleting model boundaries, making it complementary to region compilation. Our bounded factorial comparison covers unchanged, GEPA-only, GCS, combined, and manual pre-model conditions on six held-out records. Official GEPA 0.1.4 retains its seed, so the combined arm is a GCS replication rather than evidence of interaction. This budget-limited negative result does not contradict GEPA's broader task results.

AWO is the closest trace-based system: it identifies recurring tool-call sequences and bundles them into meta-tools, reducing LLM calls by up to 11.9% [1]. Agent Workflow Memory induces reusable workflows offline or online and reports large relative success gains on Mind2Web and WebArena [47]; it optimizes *what* an agent reuses rather than proving a reuse admissible, and it is the learned-workflow comparator a future head-to-head most needs. Agentic Compilation generates a deterministic JSON workflow from a single model call and replays it without further inference, reporting 80–94% zero-shot compilation success on web automation [12]; it shares our compile-versus-rerun motivation and differs in admitting the compiled plan without provenance, effect, or finite-sample risk conditions. A recent survey separates reusable templates, run-specific realized graphs, and traces [53]; in that taxonomy GAC optimizes traces under a declared-effect contract. EvoC2F compiles a typed Plan IR carrying dependency, effect, resource, idempotency, and retry annotations, then admits trajectory-derived macro-skills through tests, contracts, and regression checks [49]. Agent JIT Compilation generates and validates code plans for current web tasks, selects low-cost candidates, and schedules execution under tool pre/post-state invariants [51]. Both are closer compiler comparators than prompt-only optimizers. GAC differs by mining recurrent cross-execution regions from value-level traces and attaching a conditional group-level contract bound; the present experiments do not establish superiority to either system. Agentic plan caching adapts reusable plan templates and reports 50.31% mean cost reduction [54]. GAC adds explicit typed provenance, effect and position barriers, hard compatibility contracts, verifier/staging semantics, and an exact selective admission rule. Our result should not be read as a head-to-head improvement: we did not reimplement AWO on the GitHub scenario.

Table 22: Adjacent optimizers by what they consume, what they rewrite, which safety conditions are hard rather than learned, and what licenses deployment. The last column is where GAC differs: every other row admits on a metric, a profile, or a test, none on a finite-sample selective bound.

System	Source	Rewrite target	Hard safety	Admission
LLMCompiler	Current-query scheduling	Parallel tool calls	No	No
DSPy / MIPRO	Prompt/program parameters	Metric optimization	No	No
GEPA	Execution + evaluation traces	Reflective prompt evolution	No	Held-out metric + Pareto
AgentSlimming	Multi-agent graph	Node pruning/quantization	No	Baseline rule
AWO	Trace tool sequences	Deterministic meta-tools	No	Empirical
Agent JIT	Task description	Validated code + schedule	Tool pre/post invariants	Candidate validation
EvoC2F	Plan IR + trajectories	DAG compiler + macro-skills	Effects + resources	Tests + contracts
FlowCompile	Declared workflow	Configuration Pareto set	No	Profiled
COVENANT	Declared policy	Workflow CFG	Effects	Runtime checks
GAC (ours)	Observed value flow	Decision-eliding program	Provenance + effects	Exact selective bound

COVENANT compiles natural-language policies into workflow graphs and checks proposed actions against the declared procedure [46]. It uses policy as source; GAC uses executions as candidate evidence and must therefore abstain more aggressively. Combining declared workflow authority with trace-derived specialization is promising.

Tool-use evaluation. BFCL covers serial, parallel, abstaining, and stateful function calling [40]; API-Bank provides runnable APIs and tool-use dialogues [29]; ToolSandbox evaluates stateful conversational interactions [31]; τ^2-Bench adds dual control by agent and user [5]; ToolBench scales tool retrieval [41]; AgentBench spans interactive environments [30]; GAIA and BrowseComp test general assistant and persistent browsing capabilities [35, 48]; and SWE-bench grounds agents in real software issues [22]. These primarily test whether an agent can choose and execute actions or produce a final answer. NESTFUL exposes nested producer references, while API-Bank retains recorded call results directly [6]. BFCL and AppWorld become suitable for the same post-trace question only after their official gold artifacts are executed on pinned backends to obtain the missing intermediate results. No model runs in any of those four compiler rows. The artifact retains a revision-pinned supplementary interoperability ledger for the other benchmarks; we do not turn absent trajectories into compiler failures. Stateful and write-bearing compilation remains future work until transactional staging exists.

Trace-based specialization, guards, and deoptimization. The architecture is, structurally, a tracing JIT for agent executions, and the correspondence is close enough that the vocabulary is nearly shared: hot-trace detection becomes family mining, guard synthesis becomes the hard guard, region compilation becomes bounded synthesis, and deoptimization to the interpreter becomes fallback to the baseline agent. Dynamo introduced transparent trace regions with guards and fallback [4], and trace-based type specialization [16] and meta-tracing [8] developed the guard-and-recompile discipline we borrow. The problem of section 3.8.1 — restoring a consistent state after a specialized region is abandoned — is exactly dynamic deoptimization [19], and that literature's answer, explicit deoptimization points, is the same mechanism we defer to as "control points with continuation tokens". Effect systems supply the discipline behind treating effect declarations as a trusted computing base [32], and selective prediction long predates learn-then-test: the reject option is Chow's [11]. What is not inherited is what an agent guard must additionally certify: that arguments are grounded in observable state, that declared effects permit speculation, and that the residual error rate carries a finite-sample bound. We take the framing as the clearer statement of the contribution.

Behavioral equivalence and caching economics. Counterfactual trace auditing pairs with-skill and without-skill traces, aligns their phases, and shows that unchanged aggregate pass rates can hide many substantive behavioral differences [55]. That is exactly the weakness of our registered-contract oracle, and it describes the instrument a stronger equivalence check would use. On economics, prompt-cache strategy has been evaluated across three providers on long-horizon agentic tasks, reporting 41–80% API-cost reductions and showing that naive full-context caching can even increase latency [33]; cache-aware prompt compression models the compression/caching crossover directly [43]. Our section 5.5 observation that fewer tokens can cost more is therefore *consistent with* that literature rather than a new discovery. What our data adds is the same effect arising from *turn removal and route fragmentation* rather than from compression, and the consequence that a workflow optimizer's objective must price prefix reuse.

The deployed counterpart of that literature is context-compression middleware. Headroom, for example, sits between agent and provider and compresses tool outputs, logs, files, and retrieved chunks, reporting 60–95% token reduction on JSON payloads, 15–20% on coding tasks, and 73% on a GitHub triage workload, with a `CacheAligner` whose stated job is to flag content that would bust a provider KV-cache prefix [18]. The comparison is worth stating precisely because the token numbers are superficially similar to ours on a superficially similar workload. Such systems shrink the payload of calls that still happen; they do not directly remove a model boundary; downstream model behavior can still change the total number of requests. GAC removes the boundary and leaves the payload alone. The two therefore compose rather than compete — one could compress the reads that a compiled prefix still performs. We ran that comparator rather than only naming it, adding a live Headroom v0.5.18 ablation to both live GitHub families (section 5.3): Headroom attempted every eligible payload and applied *zero* transformations, a null engagement with these

payload shapes rather than a compression result. It nonetheless retires the specific concern that our reduction is really Headroom's reduction under another name — on this workload the two systems visibly do not touch the same bytes — though it says nothing about longer or differently shaped contexts. Their accuracy evidence is also of a different kind: measured benchmark deltas over lossy compression, with no abstention path and no finite-sample bound, which is the same distinction that separates GAC from every other row of table 22.

Program analysis and selective risk. Canonical trace families resemble process-mining variants [45]; bounded synthesis relates to partial evaluation [23], likely-invariant mining [15], and programming by example [17]. Risk-controlled admission builds on exact binomial inference [13] and learn-then-test calibration [3]. The guarantee remains conditional on i.i.d. or conditionally i.i.d. admitted group indicators and a frozen candidate; provenance and effects are separate hard constraints, not learned away by calibration.

8 DISCUSSION

8.1 What is novel, and what is not

Core novelty. The contribution is not agent compilation; it is *admissibility for trace-derived specialization.* Deterministic workflows [12], reusable trace patterns [1, 47], and effect-aware plans all have clear precedent. A classical JIT guard checks types and shapes. An agent guard must additionally show that arguments are grounded in observable state, declared effects permit speculation, runtime position is safe, and observed contract risk is bounded. GAC combines those requirements in one compile-or-retire path: typed value provenance, hard effect/permission/compatibility barriers, bounded readable synthesis, verifier and staging semantics, and per-candidate exact finite-sample admission. Each element has predecessors; their composition defines the new safety argument. The statistical component remains the weakest empirically because the observed gate is all-or-none rather than a demonstrated risk–coverage discriminator (section 9).

GCS adds an interface-level specialization over this guarded program: a bounded projection of verified live-outs, a signed task-semantic argument contract, and an exact continuation pin allow the region to execute before the first provider request. The individual ideas are not new—manual macros, projection pushdown, and partial evaluation are standard—but their composition preserves the compiler's internal effect checks, provenance, staging, and fallback rather than treating a generated meta-tool as an opaque trusted shortcut. The 12-pair result supports that mechanism on one family. The later six-pair comparison finds structural parity with an independently authored pre-model program, so the scientific claim is automatic guarded specialization, not superior runtime efficiency once a human has already placed the same program correctly.

The portfolio adds a second, narrower composition: selection across transformation *classes* rather than configurations within one declared graph, with separate exact bounds on task failure and non-positive paired utility, compatibility invalidation, and a human-review condition. FlowCompile already constructs reusable accuracy–latency configuration sets and downstream routing; GEPA evolves residual prompts; AWO and Agent JIT generate reusable or task-specific execution structures. We therefore do not claim that portfolio search itself is new. The distinct object here is an evidence-gated choice among baseline, compiler, and reviewed application-interface redesign. Its current evidence is only one family and does not demonstrate that this choice rule beats a fixed macro policy.

The NESTFUL result is as important as the live saving. A system that emitted all 32 recurrent families would look more productive, yet none has the 92 zero-violation group records required by the configured calculation. Treating those records as independent is a load-bearing sampling assumption, not a property established by the benchmark. Refusal exposes the data requirement rather than moving it into an undocumented heuristic.

8.2 What the experiments changed about the design

Seven findings altered the method after it was first built, and each is stated here as the observation that forced the change rather than as advice.

Per-episode saving does not determine whether compilation pays. Provider-side break-even is 411, 182, and 181 future episodes for the issue-type, PR-outcome, and backlog-attention families, and these exclude engineering, review, monitoring, and invalidation cost entirely. The issue-type figure is the largest because its admitted program is only two reads deep, so the family with the safest artifact is also the slowest to repay it. Amortization, not reduction, is the quantity that separates a workflow worth compiling from one that is not.

Token reduction and cost reduction came apart. The hand-written macro uses 30.9% fewer tokens than the compiled arm but is only 8.0% cheaper, because it reads 0.0% of its input from prompt cache against the compiled arm's 27.8%. Deleting turns changes the number and shape of cache reads and writes, so the two quantities measure different things and a study reporting only tokens would have overstated the saving.

No amount of calibration data substitutes for an effect declaration. Demo D exposes undeclared MCP effects and is correctly refused; the refusal comes from the declaration, not from any statistical check, and the run is indistinguishable from baseline (3.0 model calls in both arms, tokens +0.1%). A misdeclared external write is therefore outside what the admission bound can protect, which places the effect catalog in the trusted computing base rather than in the learned part of the system.

Compiler and runtime must share control-point semantics, and the failure is silent otherwise. The archived pilot emitted suffix-only regions against a runtime that resolves at entry. It still reduced provider requests 36.7% while task quality and tool-contract validity fell to 16.7% (fig. 21) — an efficiency-scored optimizer would have accepted it. Making position an explicit invariant blocked 116 suffix candidates and restored 100% observed contract validity.

Interface and execution position are separable optimizations. Exposing a macro leaves a model turn to select it; a continuation-pinned pre-model composite removes that turn. GCS reduces provider requests 50.0% against the measured provider-visible macro while both pass 12/12, which isolates position as the operative variable rather than interface width.

Learned-optimizer overhead needs a separate ledger, because it can be unbounded by the benefit. Bounded GEPA consumes 59 provider requests, 63,954 tokens, and an estimated $0.01163 across 14 task evaluations and three reflections, then retains its seed. With no change selected there is no benefit against which to amortize that cost, so folding optimization spend into the held-out arm would have hidden a null result.

Opaque identifiers behave as nominal values, not measurements. The archived pull-request pilot projected nullable merge timestamps and hulled empirical numeric ranges over identifiers, which is correct for bounded quantities and wrong for IDs: it rejects valid unseen values while admitting nothing safer. Retaining type and provenance with an `any` hull keeps the guard sound without that false rejection.

8.3 Open questions

Two of these are forced by section 5.5 rather than chosen. First, eq. (5) counts tokens and calls, and Demo F shows that is the wrong objective: a route whose traffic share is too small to amortize its own cache write is a net loss at +123.0% cost even though its prompt is strictly shorter. What the objective should charge for *cache-prefix fragmentation*, and whether that term can be estimated before synthesis rather than measured after it, is unresolved. Second, the break-even in section 5.4 is computed against the list input price, which overstates headroom for any workload whose baseline is already cache-dominated; the right comparison is the *cached* baseline price, and we do not know how much of the reported saving survives it.

Portfolio optimization beyond the pilot. The implemented portfolio layer selects among measured actions and can recommend a macro for review; it does not synthesize arbitrary macro code, execute cache-only or model-routing candidates, estimate developer effort, or learn a policy across workflow families. A study that could decide the selection question needs families deliberately spanning stable bundles, branching prefixes, cache-dominated routes, unsafe effects, and low-support abstention. It would have to measure construction, review, maintenance, invalidation, and drift costs and compare against always-macro, always-compile, cache-only, FlowCompile-style configuration selection, and a learned contextual policy. Only then can the central selection claim be stronger than "the pilot correctly chose the already stronger macro."

One further question is structural rather than empirical, and corollary 2 answers half of it. Section 9 shows that the per-candidate bound does not compose: the compiler searches candidate families, but the guarantee is stated for one fixed candidate, and a two-candidate correction already moves the requirement from 92 to 106 zero-violation groups. Freezing selection before calibration closes that gap exactly, at the cost of never exploring a second candidate once the first reaches calibration — which is why the primary GitHub families, whose dominance search keeps the higher-removal survivor of two admitted candidates, do not use it. What remains open is a procedure that keeps that exploration — letting several candidates compete for admission and retaining the best — while still pricing the search at a compiler-wide δ, rather than trading exploration away for the guarantee or paying the full Bonferroni cost for every family a workload happens to produce.

8.3.1 A prospective gate-frontier protocol, pre-registered and partially executed. Five of six live gates in this paper admit all or none, so no registered $\alpha = .05$ result yet shows a genuine risk–coverage frontier rather than a support threshold (section 9). Closing that gap needs new evidence, not new arithmetic, so we commit the design before spending on it. `paper/supplementary/prospective-gate-frontier-protocol.md` pre-registers a time-forward cohort across at least three repositories or domains, at least 300 sealed held-out paired cases (chosen so that zero compiled-only failures give a one-sided upper bound near 1%, well below the current small-cohort bounds), and three arms compared under identical records, model, cache policy, and ordering: the unchanged agent, the learned gate of section 2, and a support-only frequency gate. Compilation uses frozen candidate selection throughout, so any admitted gate there inherits corollary 2 rather than a per-candidate certificate. Thresholds, features, splits, hypotheses, and stopping rules are fixed and hashed before acquisition; the protocol also states, and defers, a model/provider breadth plan and the feasibility bar an executable workflow-compiler comparator would have to clear before being run at all. The document was committed *unexecuted*, before any repository or provider had been acquired under it, so that if the evidence were later gathered, the design could not be revised in light of what it showed. The cohort has since been sealed and partially executed; the observed result is reported below at the end of this subsection, using exactly the design, arms, and decision rule committed above.

The comparator feasibility bar has been checked, and it returns no-go rather than open (`awo-comparator-feasibility-spike.md`): AWO [1], the closest published candidate, has no confirmed public implementation to build against, so a from-scratch reimplementation could not be verified to match the system it claims to compare against. Building one anyway is exactly the "forced or mismatched comparator" this protocol already declines in advance; the gate is exercised, not skipped, and the workstream closes on that finding rather than on a fabricated baseline.

A cheap, live pilot for that study was run before committing the full budget (`gate-frontier-pilot-protocol.md`), on 90 records stratified by whether an issue's evidence comments contain a Markdown-style link — the one mechanism this codebase has on record for a compiled artifact's returned excerpt to diverge from its source, having already caused one recorded violation. Its reported contrast (4/60 violations in the Markdown-link stratum against 0/60, one-sided $p = .059$) is *withdrawn* as a compiled-versus-unchanged result: the pilot's named compiled arm used a `compiled` policy manifest while the retained artifact was registered under `base`, so exact registry resolution found no artifact and the pilot retained factual grades but no dispatch telemetry. The contrast is a difference between stochastic model and tool runs. The independently recorded excerpt error on issue 6602 still motivates the link stratum, but its predictive value is unestablished. A corrected, amended end-to-end issue gate study followed and retired before its held-out test (section 6.4).

Table 23: Gate-frontier study outcomes by repository. "Nonzero coverage in sweep" lists every value the frozen Λ grid produces for the learned gate before threshold selection; the support-only ablation ($\alpha = 1$) produces the identical sweep and admission on every repository.

Repository	Held-out	Exact	Nonzero coverage in sweep	Admitted	Req. ↓
huggingface/datasets	60	60/60	0.1087 (upper .375, rejected), 1.0	1.0	44.4%
pandas-dev/pandas	60	60/60	1.0	1.0	44.4%
psf/requests	60	59/60	1.0	1.0	44.4%
streamlit/streamlit	60	60/60	1.0	1.0	44.4%
pytorch/pytorch	—	RETIRE	none (upper 1.0 at every η)	—	—

Observed results. Executed against all five repositories: four admit a candidate and complete 240 of the pre-registered 300 held-out triples (580/580 exact discovery traces); `pytorch/pytorch` again retires at compile time. This cohort is drawn from the same frozen snapshots with the same selection seed as the core study and excludes none of its records (it shares 561/580 discovery, 442/460 calibration, and 130/150 held-out records with it), so at twice the core scale it is a larger sealed re-execution, not an independent replication, and the `pytorch/pytorch` retirement is a near-replay of the one in section 5.3.1. Table 23 gives the per-repository account; the full numeric ledger is in `prospective-gate-frontier-protocol.md`. Accounting is by episode: 240 records × 3 arms, 719 of 720 completed; exact task contracts under intention-to-treat are 240/240 (unchanged agent), 240/240 (learned gate), and 239/240 (support-only), the one incomplete episode being `psf/requests` record 6708 under the support-only arm, which raised the 120 s provider timeout and, the protocol fixing no retry rule, was not re-attempted; the record passed in the other two arms. The `psf/requests` block was executed once, as the single-repository pipeline check under the sealed selection, and reused by the five-repository run through the driver's resume path. Held-out dispatch was 160/240 for the learned gate: every open-class pull request abstained on the induced verifier's `pr.state` hull and ran the unchanged agent. The learned gate and the support-only comparator — $\alpha = 1$ in an otherwise byte-for-byte identical compile pass, so only the risk budget differs — are statistically indistinguishable on every efficiency metric (44–52% reductions, both arms). That is exactly what table 23's coverage column explains: every admitting repository deploys the identical coverage-1.0 threshold, and the one sweep that shows a third value (`huggingface/datasets`) rejects it on the risk bound, not on coverage. No held-out wrong dispatch occurred on either gate. Per the decision rule fixed in advance, this is the pre-declared null, not a frontier: at twice the prior cross-repository scale, against a comparator built to isolate the risk budget specifically, the exact-$\alpha = .05$ gate remains a support threshold — as it must at $|\mathcal{K}| = 92$, where the registered bound certifies only $n_\eta = 92$, so the pre-registered criterion of three admissible nonzero coverage levels was unattainable whatever q ranks, and where held-out q is constant within each repository because the entry state is a single record number. The study audits support-threshold behavior at full coverage under two risk budgets; it does not test the ranking quality of q.

9 LIMITATIONS AND THREATS TO VALIDITY

The threats below are grouped by the kind of inference each one endangers, so that a reader can tell which results a given caveat actually touches. Each is stated at the strength the evidence forces, including the four that undercut a headline number.

9.1 Internal validity

Whether the reported effect follows from what was manipulated rather than from how the runs were arranged or configured.

The original latency result is confounded by condition order. The fixed-prefix harness runs the entire baseline batch and then the entire compiled batch, without randomizing or counterbalancing order within pairs. Provider load, connection reuse, and prompt-cache warmth therefore differ systematically between conditions. The request-count reduction is structural and unaffected, but the −85.0% wall-latency figure should be read as an observation under this ordering rather than a clean causal estimate; the same confound is the most likely explanation for the refusing conditions of section 5.5 billing more than their baselines on identical token counts. Both natural-order studies counterbalance all six three-condition permutations; the expanded run assigns each exactly five primary records. This removes ordinal imbalance but cannot eliminate provider noise or cache interference.

Removing model turns can remove guardrail evaluations. Moderation, policy, and guardrail checks in agent frameworks commonly run *at* model boundaries. A region that deletes three of four boundaries therefore risks deleting three of four guardrail evaluations, and neither our effect catalog nor the read-only policy covers that: a guardrail is not a tool call, so it is invisible to both. Our demonstrations expose no guardrails, so the gap is untested rather than benign. A deployment must either re-run guardrails inside the permission facade for every elided boundary, or declare guardrail presence an explicit barrier condition alongside handoffs and approvals. Only the second is available here, because the first is neither implemented nor tested; a misdeclared external write remains undetectable by statistics — detecting it needs differential execution in a sandbox or declaration-versus-observation diffing, neither of which we implement.

Registry integrity is configured off in the reported run. Artifact records are mutable Python objects, the reported experiment sets lifecycle and approval fields directly, and the archived artifact carries an empty signature because signing was not enabled. The immutability and signature-verification semantics described in section 3 are therefore capabilities of the registry, not properties this experiment demonstrates.

The compressed-baseline comparator ran, and did not engage. Every headline resource number in this paper compares a compiled prefix against an *uncompressed* baseline agent, because deployed context-compression middleware reports token reductions of the same order on a comparable GitHub triage workload by an entirely different mechanism [18]. The live Headroom v0.5.18 ablation of section 5.3 checks this directly rather than leaving it asserted (`paper/supplementary/headroom-ablation-protocol.md`): Headroom applied none of its eligible payloads on either family, a null engagement at this payload shape rather than

a measured composition or a measured floor, and the committed protocol says the finding does not extend to larger or differently structured contexts. What it licenses is narrower than "the best available reduction on these axes" but still real — on this workload, over these payloads, compression and compaction visibly do not compete for the same bytes, so GAC's reduction is not a relabeling of what a compression pass would already have captured. The provider-request reduction was never exposed to this threat in the first place, because compression never removes a call.

Baseline scope. The natural study runs both the unchanged SDK loop and a live hand-written composite tool. In the expanded partial-compilation study, the macro and compiler each use two provider requests and pass 30/30, but the macro uses one local tool call rather than three and is better on tokens and estimated cost. In the earlier aggressive study, GAC saves one more provider request and more tokens but records a factual miss. The offline suite independently gives the macro the lower request ratio on three of five workloads. The compiler itself remains compile-or-retire; the new portfolio layer selects the measured macro and emits a review-required recommendation. Separately, GCS packages an admitted read program behind a synthesized projection and beats the measured provider-visible macro on 12 exploratory pairs; it does not synthesize arbitrary application logic. The six-pair follow-up gives an independently authored program the same pre-model position and finds equal requests, interfaces, input tokens, and exact quality. Official GEPA retains its seed under a 14-task-evaluation, three-reflection budget. The portfolio does not generate the macro, and on a single family its decision remains indistinguishable from an always-macro rule. AWO, AWM, Agent JIT, EvoC2F, LLMCompiler, FlowCompile, and plan caching remain unexecuted. The evidence establishes an intervention and engineering trade-off, not state-of-the-art superiority.

9.2 Construct validity

Whether the measures capture the things they are named after.

Factual quality is narrow and depth-sensitive. The original five-boolean oracle does not check summary factuality and accepts fluent fabrication. The natural studies replace it with exact snapshot fields and source-supported excerpts scored independently of tool order, but this remains an automatic extractive contract rather than human semantic adjudication. The expanded two-read artifact passes 30/30 while the earlier three-read artifact catches one compiled-only Markdown-link alteration. Thus preservation is observed at one depth and explicitly unsupported at the other. Both audited oracle corrections show that even strict-looking metrics can drift; raw online labels and correction records are retained. In the earlier run the corrected eligibility rule would swap one discovery trace, but the evaluated artifact remains the one actually executed. In the expanded run the semantic regrade changes only literal tool-contract labels, not compiler inputs, provider outputs, or measurements. The post-hoc continuation replay repairs the retained exact-source miss with a task-specific deterministic renderer, but it is not a live evaluation and does not replace human semantic adjudication or a separately calibrated continuation-risk frontier.

Semantic scope. Recorded output equality and empirical contracts do not prove full semantic equivalence. The closed DSL intentionally misses legitimate transformations and loops. Tool-effect truth, permissions, freshness, and isolation are application-supplied. The current version is read-only, does not parallelize synthesized calls, and task-semantic argument contracts are application-supplied trusted declarations. The narrow SDK adapter bypasses streaming, hosted tools, MCP, handoffs, and loops.

9.3 Statistical conclusion validity

Whether the inferences drawn from the numbers are licensed by the design that produced them.

The selective gate did not discriminate in this run. On the sealed artifact, every grid threshold below 0.14 admits zero calibration groups and every threshold at or above 0.14 admits all 92, with zero violations throughout. The score therefore behaved as an all-or-none support test rather than as a ranking of safe against unsafe inputs, and six of its seven feature weights are exactly zero (only `hull_margin` is non-zero).

The cause is diagnosable rather than mysterious, and it is a property of the data, not of the estimator. The score is fitted on development-group *unproductive* outcomes, and the artifact passed all eight development replays — so the logistic model was fitted with *zero* positive examples on eight observations over seven features. A single-class fit yields a near-constant score, and a near-constant score is exactly what produces the observed step at $\eta = 0.14$: all 92 calibration entries land in one interval. Reporting the positive count is therefore mandatory for any such gate, and we report it here as zero. Two honest routes exist: build a development set containing genuine unproductive outcomes and publish a risk–coverage curve at several α, or replace the learned score with an explicitly one-class construction (distance-to-hull, or a conformal nonconformity measure over entry features) and describe it as such. Until one of those is done, contribution (v) is demonstrated only as a sample-size counter: on NESTFUL the gate refuses everything because $26 < 92$, and here it admits everything because the score is constant. We cannot claim a demonstrated risk–coverage frontier from this evidence. Establishing one needs calibration and test sets containing natural error: hard-but-supported inputs, out-of-domain entries, source and schema drift, stale versions, ambiguous provenance, and partial tool failures.

Both natural-order artifacts repeat the same pattern. In the earlier run, thresholds below 0.14 admit no calibration records and those at or above 0.14 admit all 45, with zero registered violations and upper bound 0.0992 at $\alpha = .10$. In the expanded replication the step moves to 0.11, below which none of 92 records are admitted and at or above which all are, with zero registered violations and upper bound 0.0498. Two fitted weights are non-zero there, but they are numerically identical to fourteen significant figures (0.80093206584596 for both `hull_margin` and `provenance_ambiguity`), which is the signature of two perfectly collinear features in a degenerate fit rather than of a score that has learned to weigh them. The observed admission decision is still all-or-none. Natural planning and more calibration data therefore do not repair the missing risk–coverage discrimination, and the count of non-zero weights is not a useful progress measure for it.

One artifact is a partial exception, and it is the one whose bound is loosest. The guarded-composite gate of section 5.8 admits 88 of 92 calibration groups at its selected threshold, so its admission decision is not all-or-none: four groups are refused on entry-observable evidence. That is the only observed instance in this paper of the gate doing the job the method claims for it, it is a coverage of 0.957 rather than a curve, and it is precisely why that artifact's bound is 0.0520 instead of 0.0498 (section 3.6.1). A mechanism visible only at $n = 4$ refusals under a looser α does not establish a risk–coverage frontier; it does show that the degeneracy is a property of these development sets rather than of the estimator.

Statistical scope. The pilot and final cohort are separated, but the study was not externally preregistered. Secondary tests are unadjusted and provider latency is noisy. The optimizer comparison contains only six pairs; its exact signed-rank minimum is therefore .03125, and one provider-span record fails the host-wall consistency check. Clopper–Pearson is exact but conservative. Three selective-risk levels appear in this paper and are not interchangeable: $\alpha = .05$ for the three primary families and the prescribed-prefix ablation, $\alpha = .10$ for the earlier three-read artifact and for the guarded-composite artifact behind every GCS and comparator number, and a 15% pilot limit for the portfolio (table 3). The pooled 3.3% preservation bound in section 5.3 spans three separately calibrated artifacts and so is a statement about the observed 90 pairs, not a per-artifact certificate. The gate's guarantee applies to the recorded group-level violation definition only when admitted group indicators are i.i.d. or conditionally i.i.d.; drift, clustered tenants, adaptive artifact selection, or incomplete outcomes invalidate a naive interpretation. Each artifact is calibrated individually; the implementation lacks both candidate-family multiplicity control within a compile and a global guarantee when many artifacts are selected.

9.4 External validity and replication

How far the results carry beyond the corpus, model, and snapshot that produced them, and what a repetition would and would not recover.

The natural task still constrains the evidence shape. The original conformance prompt names three tools in an exact order, so its recurrence is constructed. The natural follow-ups remove tool names and order, and compiler eligibility no longer requires an exact trace, but the requested title, state, labels, and evidence excerpt still make the three sources useful. All 132 expanded discovery agents (and all 80 in the earlier study) converge on one order. This is stronger evidence of naturally selected recurrence than the original study, but it does not establish discovery in open-ended plans where tools may be skipped, substituted, branched, or revisited for reasons not fixed by an output schema.

Empirical scope. The richer workflow-family studies remain one repository/domain and one calibrated model configuration (the later replications of section 6.3 add a second model family and a second provider, without a cross-model certificate): the confirmatory compiler study contains 30 primary issues, the prospective portfolio pilot contains 12, the exploratory GCS comparison contains 12 further pairs, and the learned/manual comparator contains six five-condition blocks. A separate PR-outcome-core extension adds five repositories, 580 exact discovery traces, 120 completed time-forward held-out pairs, and one compile-time retirement, while a balanced rerun adds three repositories, 360 exact discovery traces, and 180 completed time-forward held-out pairs with full `open/merged/closed_unmerged` coverage under the same task. Both extensions still use a narrower two-read task than the main workflow-family evidence. The GCS cohort contains only bug and other-labelled issues; the comparator contains bug, enhancement, and other but no question-labelled issue. All GitHub evidence uses a frozen public snapshot, not the live GitHub API, so network/service behavior, concurrent mutation, rate limits, and authentication are absent. The exact gates have 92, 45, and again 92 calibration records depending on the artifact, while the portfolio has only 30 selection groups and uses a looser 15% risk limit. The fresh portfolio test contains no question-labelled issue after strict prior-record exclusion. There is no human study of productivity or user experience and no multi-domain production canary.

The AppWorld substrate widens the tool surface without widening the efficiency claim. It adds a second domain, 68 typed APIs across nine simulated apps, 46 scenarios, and 79 distinct fictional supervisors, and it is scored against an independent third-party oracle rather than a contract this paper registered. But no model runs on it, so it carries no request, token, latency, or cost number, and its group definition inherits the same independence conditional as the live studies: three variants of an AppWorld scenario share a supervisor and a database, and scenario-level grouping would give 49 groups, below the 92 the bound needs. Its dispatch measurement is likewise bounded — it establishes structural eligibility at the entry boundary, not ϕ, because the released baseline runs never used the artifact's pinned manifest, and it cannot speak to savings because those logs retain API calls rather than model boundaries. The 8,190 trajectories also repeat benchmark tasks across 28 released runs, models, and architectures. Their pooled eligibility rate is therefore descriptive and intentionally has no independent-sample confidence interval.

The artifact includes a prospective three-domain extension harness for vulnerability evidence, SEC filing facts, and privacy-modified public HMDA records. The HMDA checkpoint is now reported explicitly: 420/420 independent exact-gold reconstructions pass and 416/420 groups expose a variable path, but zero provider calls have run. The SEC pool is absent because the required compliant source contact is not configured; consequently the three-domain protocol is not frozen and human macro approvals are absent. These artifacts demonstrate data-pipeline and control-plane feasibility only and remain excluded from every effectiveness result in this paper.

Portfolio scope. The weighted utility is a declared operator preference, not a universal scientific metric. Its current dimensions omit construction time, review effort, maintenance, drift, cache fragmentation, and authorization changes. Both measured actions pass both binary gates, so exact risk controls admission but does not explain the choice; the utility weights do. The prospective result validates the chosen arm against baseline, not the selector against alternative selection algorithms. A claim that the portfolio advances the state of the art requires multi-family decisions, closer

learned baselines, sensitivity to the weights, and time-forward evaluation under drift.

Reproducibility scope. The original experimental workspace did not retain prior `.git` history. The public artifact begins from a later initial snapshot, so pre-snapshot commit ancestry and CI state cannot be reconstructed. Dataset revisions and file hashes are pinned, raw results are retained, and the full local suite passes on Python 3.14.4; nevertheless, provider responses and latency will not be byte-identical on rerun. API access and the named model are required to repeat the live study.

10 CONCLUSION

Guarded agentic compaction treats repeated traces as candidate evidence, not permission to rewrite an agent. It compiles only value-grounded, read-only prefixes that satisfy explicit compatibility, verification, and finite-sample admission requirements; otherwise it keeps the original agent.

The results separate discovery and admission from runtime efficiency, and separate both from deployability. Across the three real-record GitHub families, the compiler preserves 90/90 exact held-out contracts and reduces provider requests by 50–75%. The time-forward extension reproduces the pattern on a narrower task in four repositories and retires the fifth. The historical three-read artifact still has a factual miss, and fairly placed hand-written programs match the compiler where they share an interface. AppWorld is the one external corpus whose sample size lets the gate decide rather than arithmetic: it admits a small argument-free region and refuses four more at the margin, and that admitted region proves structurally eligible on essentially every full-code agent trajectory and essentially no ReAct one — so an artifact can satisfy every condition in this paper and still never fire. NESTFUL, API-Bank, and executed BFCL gold plans provide the complementary result: recurrent, executable traces still retire when the configured evidence requirement is unmet. Thus the contribution is automatic, evidence-gated discovery of a safe partial program, not a claim that generated code universally outruns a well-placed manual macro.

The later studies (section 6) add one public benchmark where the pattern pays off — BIRD's schema-first SQL agent compiles on 26 of 31 databases — and an end-to-end certificate on 132 fresh pull requests, and they take two claims away: the learned gate adds no measured risk–coverage value over a support-only comparator, and the run-time checks prevented no wrong answers under corrupted tool outputs.

That separation suggests an evaluation criterion for this class of system: measure a workflow optimizer by the evidence it requires, the effects and positions it refuses, and the state to which it falls back—not only by the turns it removes. The criterion is proposed, not validated; applying it to systems other than this one is open. Deciding between the remaining candidate explanations would require a richer time-forward, cross-repository comparison on full workflow tasks, a non-degenerate risk–coverage frontier, measured manual construction and maintenance cost, cache effects, drift, and closer executable workflow-learning baselines.

CODE AVAILABILITY

Code, data manifests, and research artifacts are available at https://github.com/rrahimi-uci/guarded-agentic-compaction. What was studied is guarded read-only specialization: no experiment here compiled a write, bypassed an approval, or touched a regulated decision, so the results say nothing about those cases in either direction.

REFERENCES

[1] Sami Abuzakuk, Anne-Marie Kermarrec, Rishi Sharma, Rasmus Moorits Veski, and Martijn de Vos. 2026. Optimizing Agentic Workflows using Meta-tools. *arXiv preprint arXiv:2601.22037* (2026). https://arxiv.org/abs/2601.22037

[2] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations*. https://arxiv.org/abs/2507.19457

[3] Anastasios N. Angelopoulos, Stephen Bates, Emmanuel J. Candès, Michael I. Jordan, and Lihua Lei. 2022. Learn then Test: Calibrating Predictive Algorithms to Achieve Risk Control. In *International Conference on Learning Representations*. https://openreview.net/forum?id=TNcOrox3tQ

[4] Vasanth Bala, Evelyn Duesterwald, and Sanjeev Banerjia. 2000. Dynamo: A Transparent Dynamic Optimization System. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 1–12. https://doi.org/10.1145/349299.349303

[5] Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-Bench: Evaluating Conversational Agents in a Dual-Control Environment. *arXiv preprint arXiv:2506.07982* (2025). https://arxiv.org/abs/2506.07982

[6] Kinjal Basu, Ibrahim Abdelaziz, Kiran Kate, Mayank Agarwal, Maxwell Crouse, Yara Rizk, Kelsey Bradford, Asim Munawar, Sadhana Kumaravel, Saurabh Goyal, Xin Wang, Luis A. Lastras, and Pavan Kapanipathi. 2025. NESTFUL: A Benchmark for Evaluating LLMs on Nested Sequences of API Calls. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 33538–33547. https://doi.org/10.18653/v1/2025.emnlp-main.1702

[7] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael I. Jordan. 2021. Distribution-Free, Risk-Controlling Prediction Sets. *J. ACM* 68, 6 (2021). https://doi.org/10.1145/3478535

[8] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijałkowski, and Armin Rigo. 2009. Tracing the Meta-Level: PyPy's Tracing JIT Compiler. In *Proceedings of the 4th Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems (ICOOOLPS)*. 18–25. https://doi.org/10.1145/1565824.1565827

[9] Yulang Chen, Haoxuan Peng, Jinyan Liu, Zichen Wen, Dongrui Liu, and Linfeng Zhang. 2026. AgentSlimming: Towards Efficient and Cost-Aware Multi-Agent Systems. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. 30064–30086. https://doi.org/10.18653/v1/2026.acl-long.1387

[10] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474. https://doi.org/10.1561/1900000006

[11] C. K. Chow. 1970. On Optimum Recognition Error and Reject Tradeoff. *IEEE Transactions on Information Theory* 16, 1 (1970), 41–46. https://doi.org/10.1109/TIT.1970.1054406

[12] Jagadeesh Chundru. 2026. Agentic Compilation: Mitigating the LLM Rerun Crisis for Minimized-Inference-Cost Web Automation. *arXiv preprint arXiv:2604.09718* (2026). https://arxiv.org/abs/2604.09718

[13] C. J. Clopper and E. S. Pearson. 1934. The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. *Biometrika* 26, 4 (1934), 404–413. https://doi.org/10.1093/biomet/26.4.404

[14] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman and Hall/CRC. https://doi.org/10.1201/9780429246593

[15] Michael D. Ernst, Jake Cockrell, William G. Griswold, and David Notkin. 2001. Dynamically Discovering Likely Program Invariants to Support Program Evolution. *IEEE Transactions on Software Engineering* 27, 2 (2001), 99–123. https://doi.org/10.1109/32.908957

[16] Andreas Gal, Brendan Eich, Mike Shaver, David Anderson, David Mandelin, Mohammad R. Haghighat, Blake Kaplan, Graydon Hoare, Boris Zbarsky, Jason Orendorff, Jesse Ruderman, Edwin W. Smith, Rick Reitmaier, Michael Bebenita, Mason Chang, and Michael Franz. 2009. Trace-Based Just-in-Time Type Specialization for Dynamic Languages. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 465–478. https://doi.org/10.1145/1542476.1542528

[17] Sumit Gulwani. 2011. Automating String Processing in Spreadsheets Using Input-Output Examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. 317–330. https://doi.org/10.1145/1926385.1926423

[18] Headroom Labs. 2026. Headroom: Compressing Tool Outputs, Logs, and Retrieved Context Before They Reach the LLM. Software repository, https://github.com/headroomlabs-ai/headroom. Apache-2.0; self-reported 60–95% token reduction on JSON payloads and 73% on a GitHub triage workload. Accessed 7 August 2026.

[19] Urs Hölzle, Craig Chambers, and David Ungar. 1992. Debugging Optimized Code with Dynamic Deoptimization. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 32–43. https://doi.org/10.1145/143095.143114

[20] Hugging Face. 2025. GitHub Issues Dataset Card. Hugging Face Datasets. https://huggingface.co/datasets/helmo/github-issues Revision e344be7b84d199661a9956036991e1fc25715a47.

[21] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. https://aclanthology.org/2023.emnlp-main.825/

[22] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *The Twelfth International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/hash/edac78c3e300629acfe6cbe9ca88fb84-Abstract-Conference.html

[23] Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. 1993. *Partial Evaluation and Automatic Program Generation*. Prentice Hall. https://studwww.itu.dk/people/sestoft/pebook/

[24] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2024/file/f1cf02ce09757f57c3b93c0db83181e0-Paper-Conference.pdf

[25] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 24370–24391. https://proceedings.mlr.press/v235/kim24y.html

[26] LangChain. 2026. Build a SQL agent. https://docs.langchain.com/oss/python/langchain/sql-agent Official documentation; accessed October 8, 2026.

[27] Junyan Li, Zhang-Wei Hong, Maohao Shen, Yang Zhang, and Chuang Gan. 2026. FlowCompile: An Optimizing Compiler for Structured LLM Workflows. *arXiv preprint arXiv:2605.13647* (2026). https://arxiv.org/abs/2605.13647

[28] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/83fc8fab1710363050bbd1d4b8cc0021-Abstract-Datasets_and_Benchmarks.html

[29] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3102–3116. https://doi.org/10.18653/v1/2023.emnlp-main.187

[30] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2023. AgentBench: Evaluating LLMs as Agents. *arXiv preprint arXiv:2308.03688* (2023). https://arxiv.org/abs/2308.03688

[31] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. 2025. ToolSandbox: A Stateful, Conversational, Interactive Evaluation Benchmark for LLM Tool Use Capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*. https://doi.org/10.18653/v1/2025.findings-naacl.65

[32] John M. Lucassen and David K. Gifford. 1988. Polymorphic Effect Systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 47–57. https://doi.org/10.1145/73560.73564

[33] Elias Lumer, Faheem Nizar, Akshaya Jangiti, Kevin Frank, Anmol Gulati, Mandar Phadate, and Vamse Kumar Subbiah. 2026. Don't Break the Cache: An Evaluation of Prompt Caching for Long-Horizon Agentic Tasks. *arXiv preprint arXiv:2601.06007* (2026). https://arxiv.org/abs/2601.06007

[34] Quinn McNemar. 1947. Note on the Sampling Error of the Difference between Correlated Proportions or Percentages. *Psychometrika* 12 (1947), 153–157. https://doi.org/10.1007/BF02295996

[35] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2023. GAIA: A Benchmark for General AI Assistants. *arXiv preprint arXiv:2311.12983* (2023). https://arxiv.org/abs/2311.12983

[36] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs from Preference Data. In *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5503a7c69d48a2f86fc00b3dc09de686-Abstract-Conference.html

[37] OpenAI. 2026. Agents SDK Guide. OpenAI Developer Documentation. https://developers.openai.com/api/docs/guides/agents Accessed 2026-08-03.

[38] OpenAI. 2026. API Pricing. OpenAI Developer Documentation. https://developers.openai.com/api/docs/pricing Accessed 2026-08-03.

[39] Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 9340–9366. https://doi.org/10.18653/v1/2024.emnlp-main.525

[40] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. 48371–48392. https://proceedings.mlr.press/v267/patil25a.html

[41] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2023. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. *arXiv preprint arXiv:2307.16789* (2023). https://arxiv.org/abs/2307.16789

[42] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodik, Sanjit Seshia, and Vijay Saraswat. 2006. Combinatorial Sketching for Finite Programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*. 404–415. https://doi.org/10.1145/1168857.1168907

[43] Yan Song. 2026. Cache-Aware Prompt Compression: A Two-Tier Cost Model for LLM API Caching. *arXiv preprint arXiv:2607.15516* (2026). https://arxiv.org/abs/2607.15516

[44] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 16022–16076. https://doi.org/10.18653/v1/2024.acl-long.850

[45] Wil M. P. van der Aalst. 2016. *Process Mining: Data Science in Action* (second ed.). Springer. https://doi.org/10.1007/978-3-662-49851-4

[46] Jincheng Wang, Min Zheng, and Tao Wei. 2026. COVENANT: Natural-Language Workflow Compilation for Aligned Agent Execution. *arXiv preprint arXiv:2607.25400* (2026). https://arxiv.org/abs/2607.25400

[47] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[48] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. 2025. BrowseComp: A Simple Yet Challenging Benchmark for Browsing Agents. *arXiv preprint arXiv:2504.12516* (2025). https://arxiv.org/abs/2504.12516

[49] Lei Wei, Qi Liu, Ruiyang Huang, Xiao Peng, Sicong Xie, Lanbo Lin, Chenhao Jiang, Yuanwu Xu, Tianyuan Yang, Jiayao Liu, Li Cai, Zhaolu Kang, and Bin Wang. 2026. EvoC2F: Compiling Tool Orchestration for Efficient and Evolvable LLM Agents. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. https://proceedings.mlr.press/v306/wei26q.html

[50] Frank Wilcoxon. 1945. Individual Comparisons by Ranking Methods. *Biometrics Bulletin* 1, 6 (1945), 80–83. https://doi.org/10.2307/3001968

[51] Caleb Winston, Ron Yifeng Wang, Azalia Mirhoseini, and Christos Kozyrakis. 2026. Agent JIT Compilation for Latency-Optimizing Web Agent Planning and Scheduling. In *Proceedings of the 43rd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 306)*. https://proceedings.mlr.press/v306/winston26b.html

[52] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*. https://openreview.net/forum?id=WE_vluYUL-X

[53] Ling Yue, Kushal Raj Bhandari, Ching-Yun Ko, Dhaval Patel, Shuxin Lin, Nianjun Zhou, Jianxi Gao, Pin-Yu Chen, and Shaowu Pan. 2026. From Static Templates to Dynamic Runtime Graphs: A Survey of Workflow Optimization for LLM Agents. *arXiv preprint arXiv:2603.22386* (2026). https://arxiv.org/abs/2603.22386

[54] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. 2025. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. In *Advances in Neural Information Processing Systems*, Vol. 38. https://proceedings.neurips.cc/paper_files/paper/2025/hash/9549f7d06700f0966d5f938f1d11022a-Abstract-Conference.html

[55] Xiaolin Zhou, Jinbo Liu, Li Li, Ryan A. Rossi, and Xiyang Hu. 2026. Counterfactual Trace Auditing of LLM Agent Skills. *arXiv preprint arXiv:2605.11946* (2026). https://arxiv.org/abs/2605.11946